Frankfurt School of Finance & Management

# Dissecting Characteristics via Machine Learning for Stock Selection

**David Duemig**
Frankfurt School of Finance and Management

**Prof. Dr. Wolfgang Schmidt**
Frankfurt School of Finance and Management

**Prof. Dr. Jan Nagler**
Frankfurt School of Finance and Management

## Abstract

We conduct a comparative analysis of methods in the machine learning repertoire, including penalized linear models, generalized linear models, boosted regression trees, random forests, and neural networks, that investors can deploy to forecast the cross-section of stock returns. Gaining more widespread use in economics, machine learning algorithms have demonstrated the ability to reveal complex, nonlinear patterns that are difficult or largely impossible to detect with conventional statistical methods and are often more robust to the effects of multi-collinearity among predictors. We provide new evidence that machine learning techniques can improve the economic value of cross-sectional return forecasts.

**Key words:** Machine Learning, Deep Learning, Big Data, Cross-section of Expected Stock Returns, Characteristic-based Anomalies, Ridge, (Group) Lasso, Elastic-net, Random Forest, Gradient Boosting, Neural Networks

# Table of Contents


**1 INTRODUCTION ..... 3**

1.1 WHAT IS MACHINE LEARNING? ..... 5

1.2 WHAT MACHINE LEARNING CAN(NOT) DO ..... 6

**2 LITERATURE ..... 7**

**3 DATA AND METHODOLOGY ..... 9**

3.1 DATA ..... 9

3.2 METHODOLOGY ..... 10

**4 MODELS ..... 15**

4.1 BENCHMARK ..... 15

4.2 PENALIZED LINEAR ..... 17

4.3 TREE-BASED MODELS ..... 22

4.4 NEURAL NETWORKS ..... 27

**5 RESULTS ..... 34**

5.1 PREDICTIVE SLOPE ..... 34

5.2 PORTFOLIOS ..... 35

**6 ROBUSTNESS CHECKS ..... 42**

**7 CONCLUSION ..... 43**

**GLOSSARY ..... 45**

**REFERENCES ..... 49**

**APPENDIX ..... 52**

# 1 Introduction

The implications of machine learning for quantitative finance are becoming both increasingly apparent and controversial. There is a growing discussion over whether machine learning tools can and should be applied to predict stock returns with greater precision. Broadly speaking, models that can be used to explain the returns of individual stocks draw on stock and firm characteristics, such as the market price of financial instruments and companies' accounting data. These characteristics can also be used to predict expected returns out-of-sample.

Finance researchers claim to have identified hundreds of these characteristics or anomalies that predict cross-sectional stock returns. However, how robust are these findings? For example, Harvey, Liu, & Zhu (2016) mention data mining and McLean & Pontiff (2016) notice post-publication decline.[1] Moreover, Hou, Xue, & Zhang (2017) show that many anomalies vanish when excluding micro stocks and applying value-weighting.

Given the vast breadth of potential factors or what Cochrane (2011) calls a multidimensional challenge: "[W]hich characteristics really provide independent information about average returns? Which are subsumed by others?" Cochrane (2011) speculates, "To address these questions in the zoo of new variables, I suspect we will have to use different methods."

This thesis investigates whether advanced modeling approaches can effectively synthesize and dissect the high-dimensional set of factors into a return prediction model that yields more precise predictions than other, more traditional, approaches. In particular, the study examines whether out-of-sample predictions can be improved by considering potential nonlinearities in the relationship between expected returns and characteristics as well as nonlinear interactions among factors.

To date, the majority of studies[2] use simple linear models to develop predictive models, undoubtedly because of their familiarity – these econometric techniques have been used for decades – and relative simplicity. However, financial researchers who have relied on traditional factor models have faced greater challenges since the financial crisis of 2008 (Rasekhschaffe & Jones, 2019). The most feasible explanations as to why include the tiny signal-to-noise ratio of financial data, the presence of multi-collinearity, and important nonlinear interactions between predictors that conventional methods overlook. Moreover, complex interaction effects among factors become more pronounced during times of market stress, making it unrealistic for

[1] However, Jacobs & Mueller (Forthcoming) show that there is no post-publication decline outside the U.S.
[2] See, for example, Haugen & Baker (1996), Hanna & Ready (2005), and Lewellen (2015).

researchers to rely on simple linear regression models to estimate complicated relationships between potential predictors and expected returns.

As a result, more and more market players are looking beyond traditional quantitative approaches to stock selection. Furthermore, a growing body of academic literature shows that more sophisticated tools in the domain of machine learning, which allow for nonlinear predictor interaction, can improve the prediction of stock returns. To evaluate the potential benefits of these tools, we offer a comparative analysis of promising practices in the machine learning toolkit, with reference to the familiar setting of forecasting stock returns in the cross section.

At the broadest level, our research finds that machine learning techniques can improve the economic value of cross-sectional return forecasts relative to traditional forecasting methods. We identify the strongest performing methods (tree-based methods and neural networks) and attribute their predictive gains in accordance with Gu, Kelly, & Xiu (2018) most notably to the inclusion of nonlinear predictor interactions that are overlooked by traditional statistical techniques.

## 1.1 What Is Machine Learning?

Given the broad potential applications of machine learning, there has been an understandable eagerness to utilize these techniques in a variety of applied settings.[3] But before exploring the stock selection applications of this emerging field in depth, it is necessary to first briefly review the basics of machine learning. In the broadest sense, machine learning is the science of getting computers to act without being explicitly programmed (Samuel, 1959). In a typical scenario, we deal with either a quantitative (e.g., stock price) or categorical (e.g., up/down movement) outcome measurement (or response variable) that we wish to predict according to a set of features (e.g., size, value, momentum). Both the outcome measurement and the set of features can be observed in a training set of data. Using this data, we develop a predictive model that allows us to forecast the outcome of new, unseen measurements in a test set of data. This exercise is referred to as a supervised learning problem, so called because the outcome measurement is present to guide the learning process.

Machine learning techniques have a long history of development and application.[4] This begs the question: why are these techniques only now gaining popularity? The main reasons behind the rise of machine learning can be traced back to the following developments:

- Digitalization has increased the amount of readily-available data.
- Heightened computing power and the significant drop in storage costs have allowed for the greater exploitation of this data (e.g., the ability to train large neural networks on either a CPU or GPU).
- Significant algorithmic innovations in recent years (e.g., one of the major breakthroughs in neural networks has been the switch from a sigmoid activation function to a rectified linear unit activation function).

In large part, machine learning is changing virtually every aspect of our lives. Today's algorithms accomplish tasks that, until recently, only expert humans could perform.[5] With respect to real-time applications in finance, we believe that this is an exciting time to adopt a disruptive technology that will transform how investment decisions are made on a broad scale.

[3] Prominent examples are image recognition (used in self-driving cars), voice recognition (as seen with Alexa or Siri), and recommendation engines (such as Amazon).
[4] Frank Rosenblatt invented the Perceptron, a neural network that could classify images, in 1957.
[5] For instance, the machine learning algorithm, Deep Blue, beat the best human chess master, Garry Kasparov, in 1997. The algorithm used brute-force computational speed to calculate thousands of possible moves and counter moves. More recently, the deep learning neural network, AlphaZero, used pattern recognition techniques to become the world chess champion. Unlike Deep Blue, which was pre-programmed to evaluate the value of different positions, AlphaZero did not receive domain knowledge. Nevertheless, the network was able to teach itself how to become a chess master in just four hours by playing against itself.

## 1.2 What Machine Learning Can(not) Do

In stock selection, quantitative researchers seek to describe and understand differences in expected returns across assets; fundamentally this is a problem of prediction. As machine learning techniques are, to a great extent, specialized in predictive tasks, they offer a robust means to address this problem. Moreover, machine learning techniques have the potential to help us better understand which factors are the most influential and how they relate to future returns.

As mentioned above, finance researchers have compiled an astonishing list of factors that show significant predictive power for future stock returns. Conventional statistical models experience problems when the number of predictors approaches the number of observations or when multi-collinearity among factors is present. In this high-dimensional problem setting, machine learning have repeatedly demonstrated strength by reducing degrees of freedom and eliminating redundant variations between predictors.

Uncertainty regarding the relationship between potential predictors and expected returns further complicates the success of modeling. There is no *a priori* reason why stock-level characteristics should be linearly related to stock returns. Whereas linear models are relatively simple to describe or implement, and may provide further advantages with respect to interpretation and inference, they demonstrate significant limitations in terms of their predictive prowess. If nonlinearities are present and important, challenges arise on defining their form in the model, and accounting for interactions among predictors. These phenomena can quickly increase the number of potential model varieties. Compared to conventional econometric prediction methods, machine learning offers a much wider range of functional form specifications. It is this flexibility that allows practitioners to uncover complex patterns and hidden relationships, including nonlinear relationships, which are often difficult or impossible to detect with linear models.[6]

However, this potential improvement in forecasting performance can come at a price. More specifically, machine learning algorithms do not readily identify fundamental associations between stock prices and predictor variables. This makes it difficult to interpret results and to investigate whether behavioral explanations may be the mechanisms driving the predictions.

The rest of this thesis is organized as follows: A comparison with related literature is provided in Section 2, followed by an explanation of our data and methodology in Section 3. In Section 4, we describe the models that we use in our analysis. Our main results are presented in Section 5. In Section 6 we conduct extensive robustness checks and conclusions are drawn in Section 7.

[6] However, with more flexibility comes a higher risk of overfitting the data. In Section 2, we describe refinements in implementation that emphasize stable out-of-sample performance while limiting the risk of overfitting.

# 2 Literature

The capital asset pricing model (CAPM) of Sharpe (1964), Lintner (1965), and Mossin (1966) uses asset betas with respect to the market portfolio to explain the cross section of expected returns. Since its original development, however, researchers have identified variables, such as size (Banz, 1981), the book-to-market ratio (Stattman, 1980 and Rosenberg, Reid, & Lanstein, 1985), leverage (Bhandari, 1988), earnings-to-price ratios (Basu, 1983), and past returns (Titman & Jegadeesh, 1993), that provide additional and independent information for expected returns. Sorting stocks into portfolios based on these features has often resulted in the CAPM being ineffective because the spread in betas could not explain the spread in returns. Fama & French (1992) summarize the results, and suggest a three-factor model (1993) – consisting of the factors market return, size, and value – that collectively explained these discrepancies. Yet over the next 20 years, considerable research sought to discover characteristics and factor exposures that the three-factor model could not explain. Subrahmanyam (2010) synthesized the outcome of this research and Harvey, Liu, & Zhu (2016) listed over 300 published papers that examine the cross-section of expected returns.[7] Moreover, Fama & French (2015) revamped their 3-factor model and added two new factors to analyze stock returns: Profitability and Investment.

For these reasons, the goal of this thesis is not to break new ground in terms of identifying new factors. Instead, we consider multiple characteristics simultaneously and focus on the construction of out-of-sample cross-sectional return forecasts. More specifically, we are predominantly concerned with the question of whether a model that uses existing characteristics can predict returns that surpasses the forecasting abilities of traditional approaches.

A common approach to predict differences in expected returns across stocks is to perform cross-sectional regressions of stock returns on lagged stock characteristics in the spirit of Fama & MacBeth (1973), as exemplified by Haugen & Baker (1996), Hanna & Ready (2005), and Lewellen (2015). Although traditional methods have many important applications, they fall short in what Cochrane (2011) calls the multidimensional challenge. In other words, they are not suited to handle the vast sea of spurious predictor variables that the literature has accumulated over decades. Moreover, they make strong functional-form assumptions. In the zoo of variables, more advanced machine learning tools can improve upon traditional models in terms of their capacity to handle the plethora of predictor variables, as well as account for nonlinearities and interaction effects among these predictors.

[7] Figure A.1 in the Appendix documents the number of discovered factors over time.

Research on the application of machine learning in finance is relatively active, yet most previous studies have focused on the implementation of specific algorithms. More recently, financial researchers have utilized machine learning to analyze the cross section of stock returns. Moritz & Zimmermann (2016) applied tree-based models to portfolio sorting. Harvey & Liu (2018) proposed a bootstrap procedure to select from a large group of candidate factors. Giglio & Xiu (2017) and Kelly, Pruitt, & Su (2017) used dimension reduction techniques to estimate the risk premia of common factors, whereas Kozak, Nagel, & Santosh (2017) used shrinkage methods to construct a stochastic discount factor that summarized the joint explanatory power of a large number of cross-sectional stock return predictors. Freyberger, Neuhierl, & Weber (2018) deployed the adaptive group lasso to select characteristics and estimate how they affect expected returns nonparametrically. Feng, Polson, & Xu (2018) proposed a deep factor alpha model to establish a deep learning representation for the characteristics-based factor model. Meanwhile, Han, He, Rapach, & Zhou (2018) applied lasso and elastic-net in the construction of combination forecasts of cross-sectional returns. Rasekhschaffe & Jones (2019) combined forecasts generated from different machine learning algorithms and training windows. Chinco, Clark-Joseph, & Ye (2019) applied lasso to make rolling one-minute-ahead return forecasts. Simonian, Wu, Itano, & Narayanam (2019) show how to use random forests to produce factor models that improve the description of expected return behavior relative to traditional forecasting methods.

In a comparative study, Gu, Kelly, & Xiu (2018) investigated the effectiveness of machine learning techniques in the context of asset pricing, creating forecasts for individual stock returns based on a variety of company and macro variables. Their analysis used total returns as the dependent variable rather than market excess returns, thus they jointly predicted the cross-section of expected returns and the equity premium. This comparative analysis of methods in the machine learning repertoire found that machine learning offers an improved description of expected return behavior relative to traditional forecasting methods. They attributed the predictive gain of machine learning methods to their ability to account for nonlinearities and interaction effects among variables, as well as their robustness to handle multi-collinearity among predictors. We come to similar conclusions, but focus exclusively on the cross-section of returns, irrespective of the equity risk premium. For this reason, we only use individual firm characteristics and exclude macro variables. In agreement with Rasekhschaffe & Jones (2019), we believe this approach reduces noise and better avoids the risk of overfitting. With this in mind, we address the questions of whether machine learning can help to more efficiently predict the pure cross-section of expected stock returns (i.e., without simultaneously considering the

equity premium). In doing so, we explore a wide range of machine learning tools and provide a framework that allows for nonlinear behavior and/or interaction effects among characteristics.

# 3 Data and Methodology

In this section, we describe the data, define our predictor set, and detail the methodology used in this study.

## 3.1 Data

Our sample included small-, mid-, and large-cap stocks from the S&P Developed BMI (USD) with an average of 4177 stocks per month starting in October 1989 and ending in December 2018. We identified all constituents of the S&P Developed BMI (USD) for each month. We excluded stocks from our analysis when they lacked an observation for one of the characteristics.[8] All characteristics, except returns, were winsorized at the 1st and 99th percentiles of their monthly observations. It is common practice to standardize the monthly observations for each characteristic so that they demonstrate zero mean and unit standard deviation. However, such standardization may obscure information about the distribution of characteristics. This information is informative to predict returns, as a higher dispersion in book-to-market ratios, for example, may lead to higher value premiums on average. Therefore, predictors were standardized only when necessary, e.g., when applying regularization techniques such as lasso, ridge and elastic-net.[9]

We collated monthly stock returns, taking into account dividends, stock splits, and other capital adjustments. We were concerned with forecasting cross-sectional returns relative to the cross-sectional mean return and not with forecasting the cross-sectional mean return per se, therefore, we further demeaned the return variable. Return forecasts were derived from lagged firm characteristics by relating a firm's month-t return to its characteristics available at the end of month t – 1.[10]

For convenience, Table A.1 reports the acronyms and definitions used in our predictor set of 21 characteristics. Summary statistics for monthly returns and the predictors are provided in Table A.2. We obtained fundamental data from, in order of preference, Compustat quarterly, Compustat annual, Worldscope quarterly, Worldscope semi-annual, and Worldscope annual. To

[8] An alternative would be to replace the missing values with the characteristic's cross-sectional mean.
[9] Results for standardized and raw predictors for all models used in this study are available on request.
[10] To achieve this we did not lag the characteristics, but shifted the return variable forward.

avoid forward-looking bias, we lagged Compustat data by three months and Worldscope data by six months. Market data, on the other hand, were assumed to be known immediately.

## 3.2 Methodology

We followed Gu, Kelly, & Xiu (2018) and describe stock returns as an additive prediction error model:

$$r_{i,t+1} = E_t(r_{i,t+1}) + \epsilon_{i,t+1} \tag{3.1}$$

where

$$E_t(r_{i,t+1}) = g^*(x_{i,t})$$

$$\epsilon_{i,t+1} \sim N(0, \sigma^2).$$

Stocks were indexed by $i = 1, \dots, N_t$, and months by $t = 1, \dots, T$. The main goal in this thesis was to isolate a representation of $E_t(r_{i,t+1})$ as a function of predictor variables that maximizes the out-of-sample performance for realized returns $r_{i,t+1}$. Predictors were denoted as the *M*-dimensional vector $x_{i,t}$ and the conditional expected return, which represents a flexible function of these predictors, was denoted by $g^*(\cdot)$.

In the familiar empirical setting of predicting stock returns cross-sectionally, we compared a representative set of potentially well-suited methods from the machine learning repertoire, including penalized linear models, generalized linear models, boosted regression trees, random forests, and neural networks The aim was to determine whether a model exists that provides an approximation for the unknown function $g^*(\cdot)$ that produces stock predictions whose accuracy exceed traditional approaches. For our comparison, we examined individual stocks from the S&P Developed BMI (USD) over a sample period spanning October 1989 to December 2018. Our set of predictors included 21 characteristics for each stock.

As a benchmark, we calculated monthly return forecasts based on traditional cross-sectional regressions in the spirit of Fama & MacBeth (1973). We also computed pooled time-series cross-sectional OLS regressions as a variation of this benchmark. The first estimation of our benchmark model used 10 years of data (October 1989 to September 1999) and considered the 10-year rolling average slopes for the Fama & MacBeth model and a 10-year window for the pooled OLS. Monthly cross-sectional stock returns one year ahead (October 1999 to September 2000) were then predicted. Next, we (i) *rolled* the estimation and prediction period forward by one year, or (ii) *expanded* the estimation period and *rolled* the prediction period forward by one year. We repeated this procedure until the end of the sample. Figure 1 illustrates both approaches.

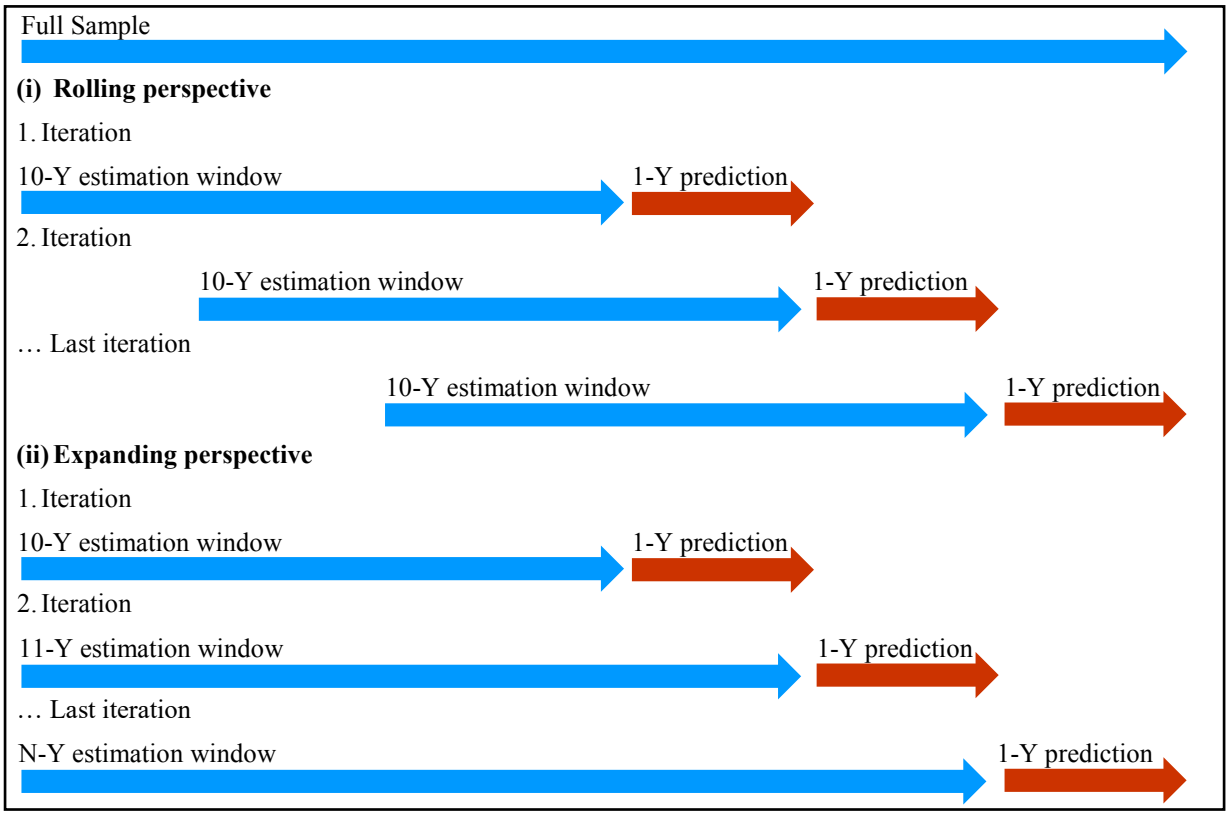



*Figure 1.* Sample splitting for benchmark models. Figure 1 shows the two backtesting procedures used in this study. The rolling approach leverages more recent observations by rolling the estimation window forward at each iteration. The expanding perspective uses all available information by expanding the estimation window at each iteration; hence, more stable predictions are expected with this procedure.

Compared to the expanding perspective (ii), the advantage of a rolling scheme (i) is that more recent observations were leveraged during modeling. However, we would expect more stable forecasts with an expanding scheme.

A slightly modified procedure was used to minimize overfitting during the application of the machine learning techniques. Overfitting occurs when a model attempts to parameterize noise rather than signal from the data. A model that overfits the data enjoys spectacular in-sample performance but shows poor forecasting power with regard to unseen or out-of-sample data. Although machine learning techniques can reveal subtle patterns in past data, overfitting presents a major challenge for predicting data for the future. More precisely, the largest challenge of machine learning does not involve the modeling of in-sample data with extreme fidelity, but discovering universal patterns that also generalize to unseen data out-of-sample.

Minimizing overfitting in machine learning is best achieved through various regularization methods based on the selection of so-called *hyper-parameters* (often referred to as *hyper-parameter tuning*). These control model complexity and therefore are key performance determinants of machine learning methods. Some hyper-parameters used in this study included the penalty parameter in lasso, ridge and elastic-net, the number of algorithm iterations in tree-

based methods, or the dropout rate in neural networks. There was little theoretical guidance on how to tune these hyper-parameters, so we followed the most common approach used in the literature: the validation set approach.[11] The goal of the validation set approach is to iteratively search through a list of hyper-parameters that minimize the validation error.[12] Put another way, hyper-parameter tuning involves searching for model complexity that tends to produce reliable out-of-sample results. Using such an approach involved dividing our sample into three parts – a *training set*, a *validation set*, and a *test set* – which maintained the temporal ordering of our data. The first set was needed to train or estimate our model for a specific set of tuning parameters. Using the validation set, we then evaluated the performance of our trained model by using the fitted model to predict the response measurements. In our implementation, we applied a grid search algorithm to conduct an exhaustive search through a set of potential tuning parameters that minimized the validation error.[13]

To provide an unbiased assessment of our final model, we withheld a separate test set not used for estimation or tuning. Apart from an additional validation set, the procedure shown in Figure 1 remained unchanged. Figure 2 illustrates the modifications.

[11] See James, Witten, Hastie, & Tibshirani (2013, pp. 176-178) for a review.
[12] All our models shared the basic goal of minimizing mean-squared prediction error (MSE).
[13] In Section 4, we will discuss the grid of hyper-parameters for each model.

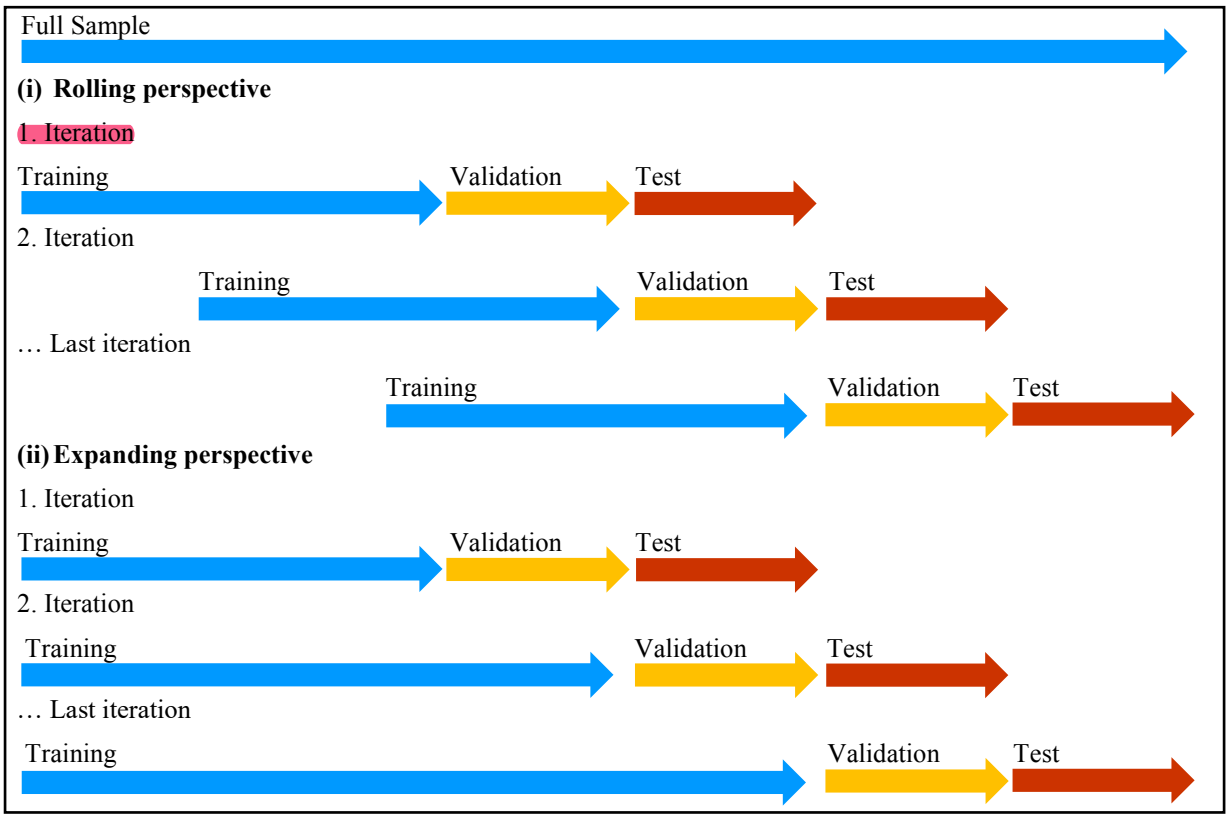



*Figure 2*. Sample splitting for machine learning models. Figure 2 depicts the two modified backtesting procedures used in this study. To allow for an unbiased assessment of the final machine learning models, a separate test set was withheld from estimation or tuning. Apart from an additional validation set, the logic described in Figure 1 remained unchanged.

For illustrative purposes, we describe the procedure with only one training/validation split per iteration. However, in our implementation, we applied more than one split per iteration. The best way to understand the intuition of this approach is to visualize it. Figure 3 displays the splitting scheme for the first iteration with three training/validation splits.

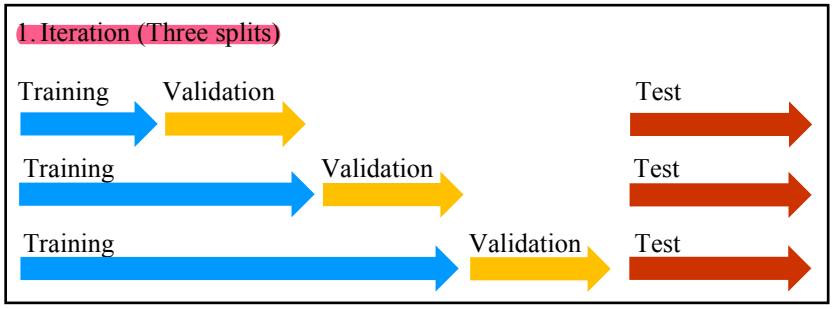



*Figure 3*. Sample splitting for machine learning models (continued). This figure details the backtesting procedures shown in Figure 2, with Figure 3 being the inner loop of the procedures described in Figure 2 (the outer loop). Using the hyper-parameter values that achieved the best average performance across all folds reduces the dependency on one specific estimation period.

We divided the training/validation set into three folds when the validation set was ahead of the training split. The training sets increased gradually by the same amount, with the magnitude depending on the number of splits. We chose a constant validation set of 12 months because we were interested in predicting one year ahead (test set of one year). We then refitted the model

with the full dataset (10 years for all but the last iteration)[14], using the hyper-parameter values that achieved the best average performance across all folds.

To assess our cross-sectional return forecast on the test set, we followed Lewellen (2015) and examined the slope coefficients in Fama & MacBeth's (1973) regressions of actual on predicted cross-sectional returns. A slope coefficient equal to one implied a good model fit. For an additional perspective, we also evaluated the economic value of our cross-sectional return forecasts by forming monthly spread portfolios that go long (short) the top (bottom) decile of companies with the highest (lowest) predicted returns, as seen in Lewellen (2015) or Green, Hand, and Zhang (2017). Because both the conventional and machine learning methods shared the same set of characteristics, the performance difference related to how the information was digested. This allowed us to address the main question of whether a model exists that dissects the high-dimensional set of existing factors into a return prediction model that demonstrate a predictive performance that surpasses traditional approaches.

Finally, as a strong robustness check, we reran our entire process on a completely different sample set, the FTSE Developed Index. The results of this test are presented in the Appendix.

[14] The last iteration used the remaining data.

# 4 Models

Here, we describe the models used in our analysis, including penalized linear models, generalized linear models, boosted regression trees, random forests, and neural networks.[15]

## 4.1 Benchmark

We followed Lewellen (2015) to provide a benchmark for our machine learning models and employed cross-sectional multiple regressions estimated via ordinary least squares (OLS), building on the procedure presented by Fama & MacBeth (1973). That is, at the end of each month we regressed stock returns on characteristics measured at the end of the previous month. This model assumes that conditional expectations $g^*(\cdot)$ can be approximated by a linear function of predictor variables $x_{i,t}$ and the parameter vector $\theta$:

$$g(x_{i,t};\theta) = x'_{i,t}\theta. \tag{4.1}$$

Stocks were indexed by $i = 1, \dots, N_t$, and months by $t = 1, \dots, T$. We used the standard least squares objective function to estimate the model:

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\left(r_{i,t} - g(x_{i,t};\theta)\right)^2. \tag{4.2}$$

Minimizing $\mathcal{L}(\theta)$ yielded the OLS estimator. Table A.3 reports average slopes, *t*-statistics, and sample sizes for 350 monthly cross-sectional multiple regressions from October 1989 to November 2018. The *t*-statistics were based on the time-series variability of the slope estimates, incorporating a Newey-West adjustment with four lags to account for possible autocorrelation in the slopes. Figure A.2 presents the rolling/expanding average of slope estimates over time.

To derive cross-sectional stock return forecasts from Fama and MacBeth regressions, we multiplied each stock's characteristics by either the prior 10-year rolling average (rolling perspective) or the expanding average (expanding perspective) of slopes and added them. Table 1 summarizes the distribution (average, standard deviation, 10th and 90th percentiles) of the resulting monthly return forecasts.

[15] For a more in-depth review of these models, see James, Witten, Hastie, & Tibshirani (2013), and Hastie, Tibshirani, & Friedman (2009), or see Marcos (2018) for a more practical treatment.

| | Rolling perspective | Expanding perspective |
|---|---|---|
| *Avg* | -0.19 | -0.07 |
| *Std* | 0.66 | 0.76 |
| *p10* | -0.97 | -0.94 |
| *p90* | 0.55 | 0.76 |

*Table 1*. Cross-sectional stock forecasts, 1999 to 2018. Table 1 reports the distribution (average, standard deviation, 10th and 90th percentiles) of 1-month forecasts derived from current firm characteristics and 10-year rolling or expanding slope estimates. All numbers equal time-series averages of monthly cross-sectional parameters. Estimates of expected one-month returns derived from Fama-MacBeth regressions vary substantially across stocks.

The forecasts produced considerable dispersion in expected cross-sectional stock returns (standard deviation of 0.66 using 10-year rolling slope estimates and 0.76 using expanding slope estimates). Examining the 10th and 90th percentiles, the estimates implied a spread of about 1.5% (rolling perspective) and 1.7% (expanding perspective) between expected monthly high and low returns.

As a variation of this benchmark, we also employed pooled time-series cross-sectional OLS regressions, which we briefly describe next.

As with the Fama & MacBeth (1973) procedure, the pooled time-series cross-sectional OLS model assumed that stock-level characteristics are linearly related to realized stock returns (see Equation 4.1). Thus, neither OLS model allowed for nonlinear effects or accounted for interactions among predictors. To estimate the pooled OLS, we again used the standard least squares objective function:

$$\mathcal{L}(\theta) = \frac{1}{NT}\sum_{i=1}^{N}\sum_{t=1}^{T}\left(r_{i,t+1} - g\left(x_{i,t};\theta\right)\right)^2. \tag{4.3}$$

Minimizing $\mathcal{L}(\theta)$ yielded the pooled OLS estimator. Figure A.3 shows rolling/expanding slope estimates over time. To derive cross-sectional stock return forecasts from the pooled time-series cross-sectional OLS regressions, we multiplied each stock's characteristics by either the prior 10-year (rolling perspective) or the expanding (expanding perspective) slope estimates and added them up. Table 2 summarizes the distribution (average, standard deviation, 10th and 90th percentiles) of the resulting monthly return forecasts.

| | Rolling perspective | Expanding perspective |
|---|---|---|
| *Avg* | 0.06 | 0.07 |
| *Std* | 0.74 | 0.75 |
| *p10* | -0.75 | -0.76 |
| *p90* | 0.85 | 0.88 |

*Table 2*. Cross-sectional stock forecasts, 1999 to 2018. Table 2 reports the distribution (average, standard deviation, 10th and 90th percentiles) of 1-month forecasts derived from current firm characteristics and 10-year rolling or expanding slope estimates. All numbers equal time-series averages of monthly cross-sectional parameters. Estimates of expected 1-month returns derived from pooled time-series cross-sectional OLS regressions vary substantially across stocks.

Forecasts based on the pooled OLS suggested a similar dispersion in expected cross-sectional stock returns (standard deviation of 0.74 using 10-year rolling slope estimates and 0.75 using expanding slope estimates). When we considered the 10th and 90th percentiles, we found that the estimates indicated a spread of about 1.60% (rolling perspective) and 1.64% (expanding perspective) between the expected monthly high and low returns.

The advantage of our benchmark models was that their objective function offered analytical estimates and thus avoided costly optimizations and calculations.

## 4.2 Penalized Linear

In the OLS approach discussed in Section 4.1, we minimized the loss functions given in Equations 4.2 and 4.3 to obtain the OLS parameter estimates:

$$\hat{\theta} = (X'X)^{-1}(X'R).^{16} \tag{4.4}$$

There were two critical characteristics of estimators to be considered: bias and variance. *Bias* is the difference between the expected estimator and the true population parameter. It measures the accuracy of the estimates:

$$Bias\left(\hat{\theta}\right) = E\left(\hat{\theta}\right) - \theta. \tag{4.5}$$

*Variance*, on the other hand, measures the spread, or uncertainty, in these estimates. It is given by:

$$Var\left(\hat{\theta}\right) = \sigma^2(X'X)^{-1}, \tag{4.6}$$

where the unknown error variance $\sigma^2$ can be estimated from the residuals as:

$$\hat{\sigma}^2 = \frac{e'e}{n-m}, \tag{4.7}$$

$$e = R - X\hat{\theta}. \tag{4.8}$$

[16] The notation is derived in the Appendix.

The expected mean squared error (MSE), $E(e)$, can be decomposed into the sum of three fundamental quantities: error due to variance, error due to bias, and the remainder (the unexplainable part):

$$E(e) = \underbrace{\left(E\left(X\hat{\theta}\right) - X\theta\right)^2}_{Bias^2} + \underbrace{E\left(X\hat{\theta} - E\left(X\hat{\theta}\right)\right)^2}_{Variance} + \underbrace{\sigma^2}_{Irreducible\ Error}. \tag{4.9}$$

Equation 4.9 shows that we needed to select a statistical model that simultaneously achieved low variance and low bias to minimize the expected error. The relationship between bias, variance, and MSE given in Equation 4.9 and illustrated in Figure A.4 is referred to as the *bias-variance trade-off*. The OLS estimator has the desired property of being unbiased. However, it can exhibit high variance, specifically when: (1) the predictor variables are highly correlated with each other[17] and (2) the number of predictor variables $M$ is large.[18] High variance indicated that the model begins to overfit noise rather than extracting signal, which implies that a small change in the data could cause a large modification in the least square coefficient estimates. The most common machine learning solution to alleviate this problem is to reduce variance at the expense of an increase in bias. This approach is called regularization and can be beneficial for the predictive performance of a model.[19] The statistical model for our regularized linear model was the same as the simple linear model in Equations 4.2 and 4.3, except that we used standardized predictors[20] and included a new term in the loss function:

$$\mathcal{L}(\theta;\cdot) = \underbrace{\left(r_{i,t} - g\left(z_{i,t};\theta\right)\right)^2}_{Loss\ Function} + \underbrace{\phi(\theta;\cdot)}_{Penalty}. \tag{4.10}$$

Adding a penalty deteriorates a model's in-sample performance in order to improve out-of-sample stability. This will be the case when penalization manages to reduce the model's fit to noise while preserving its fit to signal. Three well-known choices for the penalty function $\phi(\cdot)$ are *ridge regularization*, *lasso regularization* and *elastic net regularization*.

**Ridge** (Hoerl & Kennard, 2000) uses $l_2$ regularization, which adds a penalty equal to the sum of squared values of the coefficients:

$$\phi(\theta;\lambda) = \lambda \sum_{j=1}^{M} \theta_j^2. \tag{4.11}$$

This penalizes the size of parameter estimates in order to shrink them towards zero. Thus, $l_2$ regularization can be seen as a *shrinkage* method that helps to prevent coefficients from

[17] A large positive coefficient on one variable can be canceled by a similarly large negative coefficient on its correlated cousin.
[18] This is reflected in the formula for variance given above.
[19] As the validation mean squared error is a function of the squared bias plus the variance, see Equation 4.9.
[20] The standardization procedure is explained in the Appendix.

becoming unduly large in magnitude. However, ridge will not set any of the coefficients to exactly zero (unless $\lambda = \infty$) and thus always result in models that include all $M$ predictors.[21]

The **lasso** (Tibshirani, 1996) is an alternative approach that overcomes this disadvantage. Lasso uses $l_1$ regularization that adds a penalty equal to the sum of absolute values of the coefficients:

$$\phi(\theta; \lambda) = \lambda \sum_{j=1}^{M} |\theta_i|. \tag{4.12}$$

As a result, for sufficiently large values of $\lambda$, many coefficients are exactly equal to zero. In this sense, lasso performs variable *selection*, resulting in models that are easier to interpret.[22]

To determine the parameter $\lambda$, which is a tuning parameter that controls the relative impact of the $l_2$ and $l_1$ penalties, we chose a grid of $\lambda$ values and computed the validation error for each value of $\lambda$. Subsequently, the model was re-fit using all of the available observations and the selected value of the tuning parameter, as described in Section 3.2.

**Elastic-net** regularization (Hastie & Zou, 2005) incorporates penalties from both $l_1$ and $l_2$ regularization**:**

$$\phi(\theta; \lambda, a) = \lambda \left( \frac{1-a}{2} \sum_{j=1}^{M} \theta_j^2 + a \sum_{j=1}^{M} |\theta_i| \right), \tag{4.13}$$

where $a$ is the mixing parameter between ridge $(a = 0)$ and lasso $(a = 1)$.
The two hyperparameters $\lambda$ and $a$ were optimized by selecting the combination of $\lambda$ and $a$ that minimized the validated sum of squared residuals.

Minimizing $\mathcal{L}(\theta) + \phi(\theta; \cdot)$ given in Equation 4.10 yielded the penalized estimator. In the following we focus on the elastic-net penalty, but the results obtained with lasso and ridge are presented in the Appendix. Figure A.5 presents rolling/expanding slope estimates over time, whereas Figures A.6 and A.7 display the selected/number of features over time. Finally, Figure A.8 illustrates the slope averages.

Table 3 summarizes the distribution (average, standard deviation, 10th and 90th percentiles) of the resulting monthly return forecasts, which can be derived in a similar fashion to the approaches described in Section 4.1.

[21] This can create a challenge in model interpretation, especially if the number of predictor variables $M$ is large.
[22] For a detailed explanation of why lasso regularization, unlike ridge regularization, results in coefficient estimates that are exactly zero, see James, Witten, Hastie, & Tibshirani (2013, pp. 221-222).

| | Rolling perspective | Expanding perspective |
|---|---|---|
| *Avg* | 0.05 | 0.06 |
| *Std* | 0.71 | 0.72 |
| *p10* | -0.72 | -0.73 |
| *p90* | 0.82 | 0.85 |

*Table 3*. Cross-sectional stock forecasts, 1999 to 2018. Table 3 reports the distribution (average, standard deviation, 10th and 90th percentiles) of 1-month forecasts derived from current firm characteristics and 10-year rolling or expanding penalized slope estimates. All numbers equal time-series averages of monthly cross-sectional parameters. Estimates of expected 1-month returns derived from penalized pooled time-series cross-sectional OLS regressions vary substantially across stocks.

## Generalized Linear

So far, this thesis has focused on linear models that are easy to describe and implement and have advantages in terms of interpretation and inference. However, standard linear models can have significant limitations in terms of predictive power. This is because the linearity assumption is almost always an approximation, sometimes an inaccurate one. In this and the following Sections, we relax the linearity assumption and introduce popular non-parametric models of $g^*(\cdot)$, each complemented by regularization methods to mitigate overfitting.

The generalized linear model is the first and most straightforward nonparametric approach considered. This model augments/replaces the vector of inputs $z$ with new additive terms, which are nonlinear transformations of $z$, and then uses linear models in this new space of derived input features. The model we implemented adapted the simple linear model by adding a K-term spline series expansion of the predictors. More specifically, instead of approximating the conditional expectations $g^*(\cdot)$ by a linear function of predictor variables $z$ and the parameter vector $\theta$, we now have:

$$g\big(z;\ \theta, b(\cdot)\big) = \sum_{j=1}^{M} b(z_j)'\theta_j, \tag{4.14}$$

where $\theta_j$[23] differ for each $b(z_j)$ – the points where the coefficients change are called *knots* – and $b(\cdot) = (b_1(\cdot),\ b_2(\cdot), \dots,\ b_K(\cdot))'$ is a vector of basic functions.

Equation 4.14 can be considered a standard linear model with predictors $b_1(z_j), b_2(z_j)$, …, $b_K(z_j)$. Therefore, we can use least squares to estimate the unknown regression coefficients. Generalized linear models are thus the closest nonlinear counterparts to the linear approaches discussed in Section 4.1. There are many potential choices for $b(\cdot)$. We used a *cubic spline,* which is a common choice for a basis function. The most direct way to represent a cubic spline

[23] The parameters are now a $K \times M$ matrix where $\theta = (\theta_1, \theta_2, \dots, \theta_M)$.

is to begin with a basis for a polynomial (namely, $z, z^2, z^3$) and then add one *truncated power basis* function per knot.[24] A truncated power basis function is defined as

$$h(z,\xi) = (z - \xi)_+^3 = \begin{cases} (z-\xi)^3 & if\ x > \xi \\ 0 & otherwise \end{cases}, \tag{4.15}$$

where $\xi$ is the knot. A cubic spline is continuous and has continuous first and second derivatives at the knots.

Unfortunately, splines can have high variance at the outer range of the predictors; that is, when $z$ takes on either a very small or very large value. A *natural cubic spline,* which we also considered in our implementation, makes the function linear beyond the boundary knots.[25] This additional constraint means that natural splines generally produce more stable estimates at the boundaries. Despite this resulting in bias near the boundaries, assuming the function is linear near the boundaries (where less information is available) is often considered reasonable (Hastie, Tibshirani, & Friedman, 2009, p. 145). A natural cubic spline with $K$ knots is represented by $K$-basis functions. Starting from the truncated power series basis (Equation 4.15) and imposing additional boundary constraints, we arrive at:

$$N_1(z) = 1, N_2(z) = z, N_{K+2}(z) = d_k(z) - d_{k-1}(z), \tag{4.16}$$

where

$$d_k(z) = \frac{(z - \xi_k)_+^3 - (z - \xi_K)_+^3}{(\xi_K - \xi_k)}$$

Each of these basis functions have zero, second and third derivatives for $z \geq \xi_K$.

Because $K$-term spline series expansion of the predictors quickly multiplies the number of model parameters, we use penalization to control degrees of freedom. Our choice of penalization function was specialized for the spline expansion setting and is known as the *group lasso*. The group lasso seeks to estimate the function $g^*(\cdot)$ nonparametrically while setting functions for a given predictor to zero if the characteristic does not help to predict expected returns. Therefore, the procedure achieves variable *selection* and takes the form:

$$\phi(\theta; \lambda, K) = \lambda \sum_{j=1}^{M} \left( \sum_{k=1}^{K} \theta_{j,k}^2 \right)^{1/2} \tag{4.17}$$

As its name suggests, the group lasso selects either all $K$-spline terms associated with a given characteristic or none of them. We embed this penalty in the general objective of Equation 4.10.

[24] Because the space of spline functions of a particular order and knot sequence is a vector space, there are many equivalent bases for representing them. While the truncated power basis is conceptually simple, it is not attractive numerically: powers of large numbers can lead to severe rounding problems. In our implementation, we thus used a B-spline basis, which allows for efficient computations even when the number of knots $K$ is large.

[25] The region where $z$ is smaller than the largest knot or bigger than the largest knot.

Minimizing this equation yielded the group penalized estimator. The model consisted of two tuning parameters: $K$ and $\lambda$. $K$ represented a smoothing parameter[26] and $\lambda$ controlled the relative impact of the group penalty. As before, the two hyperparameters were optimized by selecting the combination of $\lambda$ and $K$ that minimized the validated sum of squared residuals.

In the following we focus on the results derived with natural cubic splines. Figures A.9 and A.10 show the selected/number of features over time Table 4 summarizes the distribution (average, standard deviation, 10th and 90th percentiles) of the resulting monthly return forecasts.

| | **Rolling perspective** | **Expanding perspective** |
|---|---|---|
| *Avg* | 0.04 | 0.05 |
| *Std* | 0.54 | 0.58 |
| *p10* | -0.59 | -0.63 |
| *p90* | 0.72 | 0.77 |

*Table 4*. Cross-sectional stock forecasts, 1999 to 2018**.** Table 4 reports the distribution (average, standard deviation, 10th and 90th percentiles) of 1-month forecasts derived from current firm characteristics and 10-year rolling or expanding slope estimates. All numbers equal time-series averages of monthly cross-sectional parameters. Estimates of expected 1-month returns derived from generalized and penalized pooled time-series cross-sectional OLS regressions vary substantially across stocks.

The generalized linear model captures individual predictors' nonlinear impact on expected returns but does not allow for interactions among characteristics.[27] This shortcoming can be remedied by adding certain interactions as additional regressors.[28] However, while expanding univariate predictors with $K$-basis functions multiplies the number of parameters by a factor of $K$, multi-way interactions increase the parameterization combinatorially. Without *a priori* assumptions for which interactions to include, the generalized linear model becomes computationally infeasible. Although the model assumptions are somewhat restrictive, it provides desirable econometric advantages and is far less restrictive than assuming linearity right away. In the following sections, we examine tree-based methods and neural networks that have shown utility in accounting for potentially complex multi-way predictor interactions.

## 4.3 Tree-Based Models

In this Section, we describe tree-based methods that can be applied to our regression problem. These methods have been widely applied to incorporate multi-way predictor interactions. They involve segmenting the predictor space into several rectangles and subsequently fitting a simple

[26] As $K$ increases, the precision of the approximation increases, but so do the number of parameters we have to estimate, hence the variance.
[27] For example, the predictive power of the book-to-market ratio for expected returns does not vary with firm size.
[28] For instance, we could interact every characteristic with size to see if small firms are actually different.

model in each of these regions. As the set of splitting rules for segmenting the predictor space can be summarized in a tree, these approaches are called *decision tree* methods.

Specifically, a decision tree grows in a sequence of steps, with a new branch at each step segmenting the data remaining from the previous step based on one of the predictor variables. This sequential branching divides the space of predictors into rectangular partitions and approximates the unknown function $g^*(\cdot)$ with the mean of the training observations within each partition.

Tree-based methods are simple and useful for interpretation, but they cannot compete with other supervised approaches in terms of predictive power. Therefore, we introduced random forest and boosting methods to produce multiple trees that are combined into a single consensus forecast. The combination of a large number of trees often leads to a significant improvement in predictive power at the cost of a more limited interpretation.

Figure 4 shows an example with two predictors: momentum and size. The split at the top (or root) of the tree results in two large branches. In this case, the observations were first sorted by momentum, with those above the breakpoint of 0.5 (right-hand branch) being assigned to Category 3. Observations below the breakpoint 0.5 (left-hand branch) were further sorted by size, with those below 0.3 being assigned to Category 2 and those above 0.3 to Category 1.

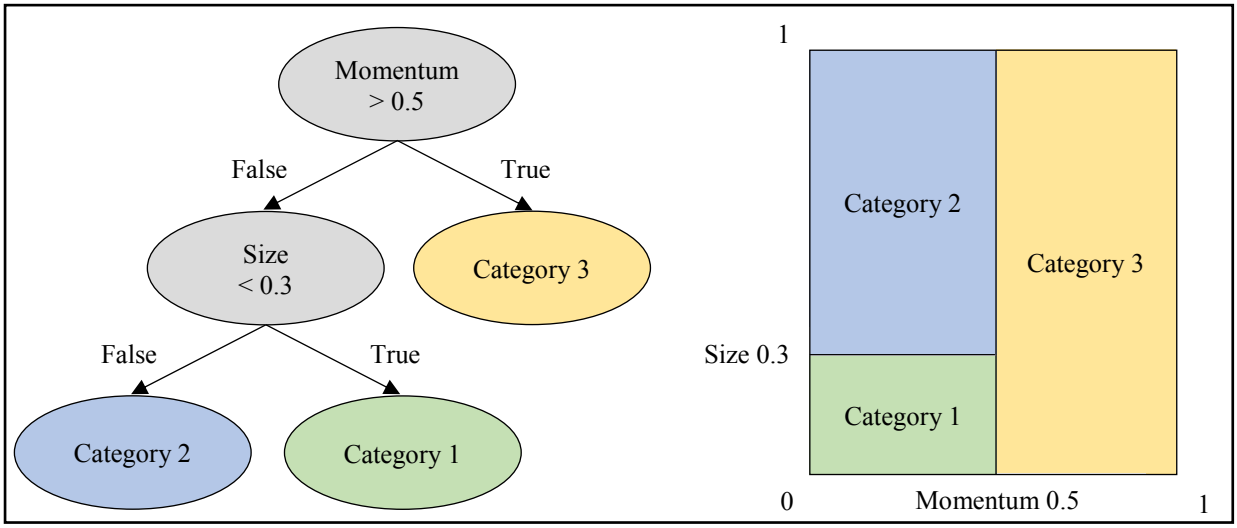



*Figure 4*. Decision tree with two internal nodes (Momentum and Size) and three terminal nodes or leaves (Categories 1–3). Figure 4 illustrates a regression tree (left) and its equivalent representation (right) for two characteristics (momentum and size). The terminal nodes, or leaves, of the tree are colored in blue, green, and yellow. The points along the tree where the predictor space is split (colored in grey) are referred to as internal nodes. The segments of the tree that connects the nodes are called *branches*. Based on their values for momentum and size, the sample of individual stocks is divided into three categories.

Following the tree analogy, Categories 1–3 are known as *terminal nodes* or *leaves* of the tree; the points at which the predictor space is split (momentum and size) are referred to as *internal nodes;* and the segments of the trees connecting the nodes are called *branches*.

When growing a tree, the goal is to find categories $C_1, \ldots, C_k$ that minimize the RSS, given by:

$$\sum_{k=1}^{K} \sum_{x_{i,t} \in C_k} (r_{i,t+1} - \theta_{C_k})^2, \tag{4.18}$$

where $\theta_{C_k}$ denotes the mean response for the training observations within the *k-th* category. Unfortunately, it is computationally infeasible to consider every possible partition of the feature space into $K$ categories. Therefore, we proceeded with a top-down, greedy approach known as *recursive binary splitting*. The approach starts at the top of the tree, where all observations belong to a single region, and then successively divides the predictor space, with each split indicated by two new branches further down in the tree. This method is 'greedy' because rather than looking ahead and choosing a split that ultimately leads to a superior tree at a later stage, a split is made at each step of the tree-building process based on information in that particular step. The splitting stops when the number of leaves or the depth of the tree reaches a predefined threshold value, which can be adaptively selected using our validation set approach described in Section 3.2.[29]

Once the partitions have been determined, the predicted value of the response for a given test observation is the mean value of the training observations in the region to which that test observation belongs. More formally, the prediction of a tree with $K$ leaves, and depth $L$ can be represented as follows:

$$g(x_{i,t}; \theta, K, L) = \sum_{k=1}^{K} \theta_k 1_{\{x_{i,t} \in C_k(L)\}}, \tag{4.19}$$

where $C_k(L)$ represents one of the $K$ partitions and $\theta_k$ the mean of outcome variables within $C_k(L)$. For the example in Figure 4, the prediction equation can be expressed as follows:

$$\begin{aligned} g(x_{i,t}; \theta, 3, 2) = \theta_1 1_{\{Momentum_{i,t} \geq 0.5\}} + \theta_2 1_{\{Momentum_{i,t} < 0.5\}} 1_{\{Size_{i,t} < 0.3\}} \\ + \theta_3 1_{\{Momentum_{i,t} < 0.5\}} 1_{\{Size_{i,t} \geq 0.3\}} \end{aligned} \tag{4.20}$$

Decision trees have a number of advantages over the more traditional approaches described in Section 4.1. For example, trees can be explained easily, handle both categorical and numerical data, approximate nonlinearities, and capture multi-way predictor interactions without *a priori* specification. However, their flexibility is also their limitation. That is, trees are among the

[29] For details, see James, Witten, Hastie, & Tibshirani (2013, pp. 303-307).

prediction methods most prone to overfitting. In other words, a small change in the data can cause a large change in the final estimated tree. By aggregating many decision trees and using random forest and boosting methods (both of which will be introduced next), predictive power can be significantly improved.

A **random forest** (Breiman, 2001) is an ensemble method that combines predictions from multiple trees into a single consensus forecast. It is a variant of a more general procedure known as *bagging* or *bootstrap aggregation,* which averages many noisy but approximately unbiased models, thereby reducing variance and stabilizing the predictive power of a statistical model.[30]

Trees are ideal candidates for bagging because they can capture complex interaction structures in the data and, if grown sufficiently deep, have relatively low bias. In other words, since trees are notoriously noisy, they benefit greatly from the averaging procedure.

Using random forests further improves the variance reduction of bagging by reducing the correlation between the trees. As with bagging, a number of decision trees are created, but at each potential split in the tree, a random sample of $m$ predictors from the full set of $M$ predictors are drawn as split candidates.[31] Thus, the algorithm does not consider all available predictors at each split in the tree. This has a clever justification: if there is, for example, a very strong predictor in the dataset and a number of moderately strong predictors, then most of the trees will use this strong predictor in the top split, leading to a significant correlation among the final predictions. Random forests circumvent this problem by requiring that each split only considers a subset of predictors, which reduces the correlation between trees and further improves the variance reduction compared to conventional bagging methods. The number and depth of the trees are tuning parameters that are optimized via the validation set approach described in Section 3.2.

Table 5 summarizes the distribution (average, standard deviation, 10$^{th}$ and 90$^{th}$ percentiles) of the resulting monthly return forecasts.

[30] More formally, given a set of $N_t$ independent observations $X_1, \dots, X_{N_t}$, each with variance $\sigma^2$, the variance of the mean $\bar{X}$ of the observations is given by $\sigma^2/N_t$.
[31] Note that if a random forest considers all predictors on each split, this will result in bagging.

| | Rolling perspective | Expanding perspective |
|---|---|---|
| *Avg* | 0.05 | 0.07 |
| *Std* | 0.60 | 0.60 |
| *p10* | -0.70 | -0.69 |
| *p90* | 0.75 | 0.77 |

*Table 5*. Cross-sectional stock forecasts, 1999 to 2018. Table 5 reports the distribution (average, standard deviation, 10th and 90th percentiles) of 1-month forecasts derived from current firm characteristics and 10-year rolling or expanding coefficients. All numbers equal time-series averages of monthly cross-sectional parameters. Estimates of expected 1-month returns derived from random forests vary substantially across stocks.

**Boosting** works similarly, except the trees are grown sequentially.[32] Instead of fitting a single large decision tree to the data, which potentially amounts to overfitting, the boosting approach learns slowly by recursively combining predictions from several over-simplified trees. The theory behind boosting suggests that many weak learners as an ensemble can comprise a single strong learner with greater stability than a single complex tree.

In this case, the procedure begins by fitting a shallow tree with depth $L = 3$. This oversimplified tree was doomed to be a weak predictor with significant bias in the training sample. Subsequently, a second simple tree (with the same shallow depth $L$) was fit using the residuals of the first tree rather than the outcome $r_{i,t+1}$. Forecasts from these two trees were combined to form an ensemble prediction of the outcome, but the forecast component from the second tree was shrunken by a factor $\lambda \in (0,1)$ to prevent the model from overfitting the residuals. At each new step, a shallow tree was fitted to the residuals from the previous trees, and its residual forecast was added to the total with a shrinkage weight of $\lambda$. This procedure was repeated until the number of predefined trees was exhausted. By fitting small trees to the residuals, the model slowly improved in areas where previously it's performance was poor. In contrast to bagging, the construction of each tree therefore depended strongly on the trees that have already been grown. The final output was an additive model of shallow trees with three tuning parameters (tree depth, number of trees and shrinkage parameter) which we selected adaptively using the validation set approach.

Table 6 summarizes the distribution (average, standard deviation, 10th and 90th percentiles) of the monthly return forecasts resulting from this model.

[32] Boosting was originally described by Schapire (1990) and Freund (1995) for classification problems. Friedman, Hastie, & Tibshirani (2000) and Friedman (2001) extended boosting to context beyond classification.

| | Rolling perspective | Expanding perspective |
|---|---|---|
| *Avg* | 0.09 | 0.14 |
| *Std* | 1.21 | 1.32 |
| *p10* | -0.67 | -0.71 |
| *p90* | 0.69 | 0.85 |

*Table 6*. Cross-sectional stock forecasts, 1999 to 2018. Table 6 reports the distribution (average, standard deviation, 10th and 90th percentiles) of 1-month forecasts derived from current firm characteristics and 10-year rolling or expanding coefficients. All numbers equal time-series averages of monthly cross-sectional parameters. Estimates of expected 1-month returns derived from boosted regression trees vary substantially across stocks.

As described above, random forests and boosting methods can improve accuracy over prediction with a single tree at the expense of interpretability. Although it is much harder to interpret these methods, it is possible to obtain an overall summary of the predictor importance using the RSS. Specifically, the total amount by which the RSS was reduced due to the split over a specific predictor, averaged over all trees, can be computed to estimate overall predictor variable importance. A graphical representation of this feature importance, both for random forests and boosting, is depicted in Figures A.11 - A.14.

## 4.4 Neural Networks

Artificial neural networks were the last nonlinear model investigated. An artificial neural network is a machine learning technique inspired by the way biological neural networks process information in the human brain. The term *neural network* comes from the work of neuroscientist Warren S. McCulloch and logician Walter Pitts, who together developed the first conceptual model of an artificial neural network. In their work, they describe the concept of a neuron, a single cell in a network of cells that receives inputs, processes them, and generates outputs. Neural networks have created much excitement in machine learning research and industry thanks to many groundbreaking results in speech recognition, computer vision, and text processing. Although there are many different network architectures, our analysis focused on traditional *feedforward* networks. They consist of an *input layer* of predictors, one or more *hidden layers*, and an *output layer* that aggregates hidden layers into a final prediction.

Specifically, our goal was to input an $M$-dimensional vector of predictors $x_{i,t}$ into a function $g^*(x)$ that outputs forecasts for the realized returns $r_{i,t+1}$. One of the simplest possible neural networks is to define $g^*(x)$ as a single *neuron* in the network. Figure 5 depicts an example of such a neural network with three predictors: momentum, size, and value.

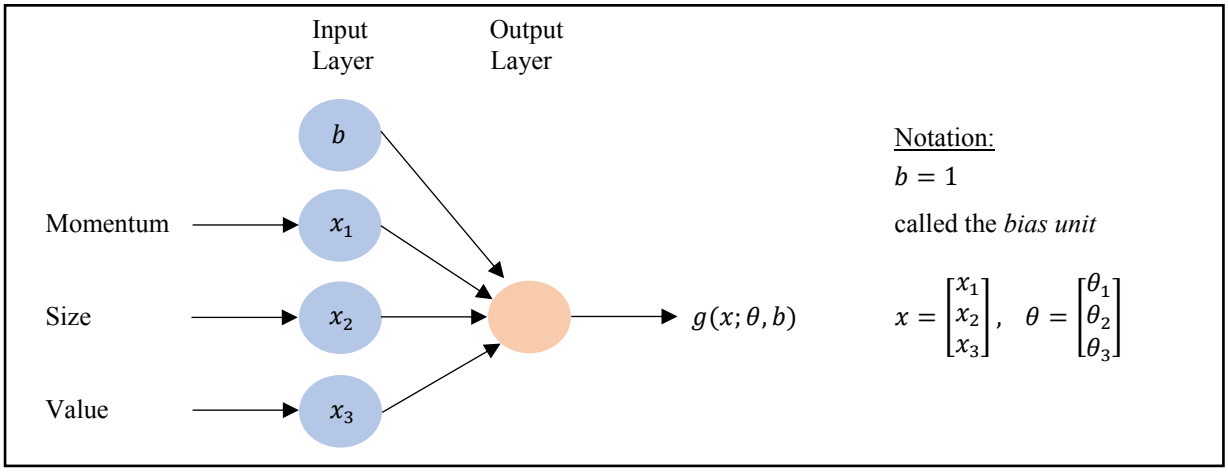



*Figure 5*. Simple neural network without a hidden layer. Figure 5 shows an oversimplified neural network without a hidden layer, which corresponds to a simple linear regression. The blue circles denote the input layer, and the red circle denotes the output layer. Each arrow is associated with a weight parameter and the output layer aggregates the weighted signals into a forecast.

The number of units in the input layer equals the number of predictors $x_1, x_2, x_3$ plus an additional bias unit $b$ corresponding to the intercept term. Each predictor is amplified by a three-dimensional vector $\theta$, with one weight parameter per predictor. The output layer aggregates the weighted signals into the forecast $\sum_{u=1}^{3} x_u \theta_u + b$, which in the case of no hidden layer (Figure 5) corresponds to a simple linear regression.

A more complex neural network takes the single neuron described above and "stacks" it so that one neuron passes its output as input to the next neuron, resulting in a more sophisticated function. The power of a neural network is that it only requires the input characteristics $x_{i,t}$ and the corresponding outputs $r_{i,t+1}$, while it figures out everything in the middle itself. This process of learning intermediate interactions is referred to as *end-to-end learning*. Following the example above, in Figure 6 we inputted a set of features $x_1, x_2, x_3$ into our network and connected those characteristics to three neurons, so-called *hidden units*.[33] The aim of this neural network is to automatically identify three relevant interactions, so the characteristics predict the realized returns efficiently. All that is required for the neural network is a sufficient number of training examples ($x_{i,t}$, $r_{i,t+1}$). Often, the neural network discovers complex relationships that are very useful for predicting the output, but these relationships can be difficult for humans to understand as they have no "common" meaning.[34] The following figure shows an example with one hidden layer containing four neurons:[35]

[33] The term *hidden layer* is "hidden" since we do not have the ground truth training value for the hidden units. This contrasts with the input and output layers, of which we know the ground truth values ($x_{i,t}$, $r_{i,t+1}$).
[34] For this reason, neural networks are sometimes called black boxes because it can be difficult to understand the relationships they have identified.
[35] In this example we have a single hidden layer, but it is possible to have a more complex neural network with multiple hidden layers.

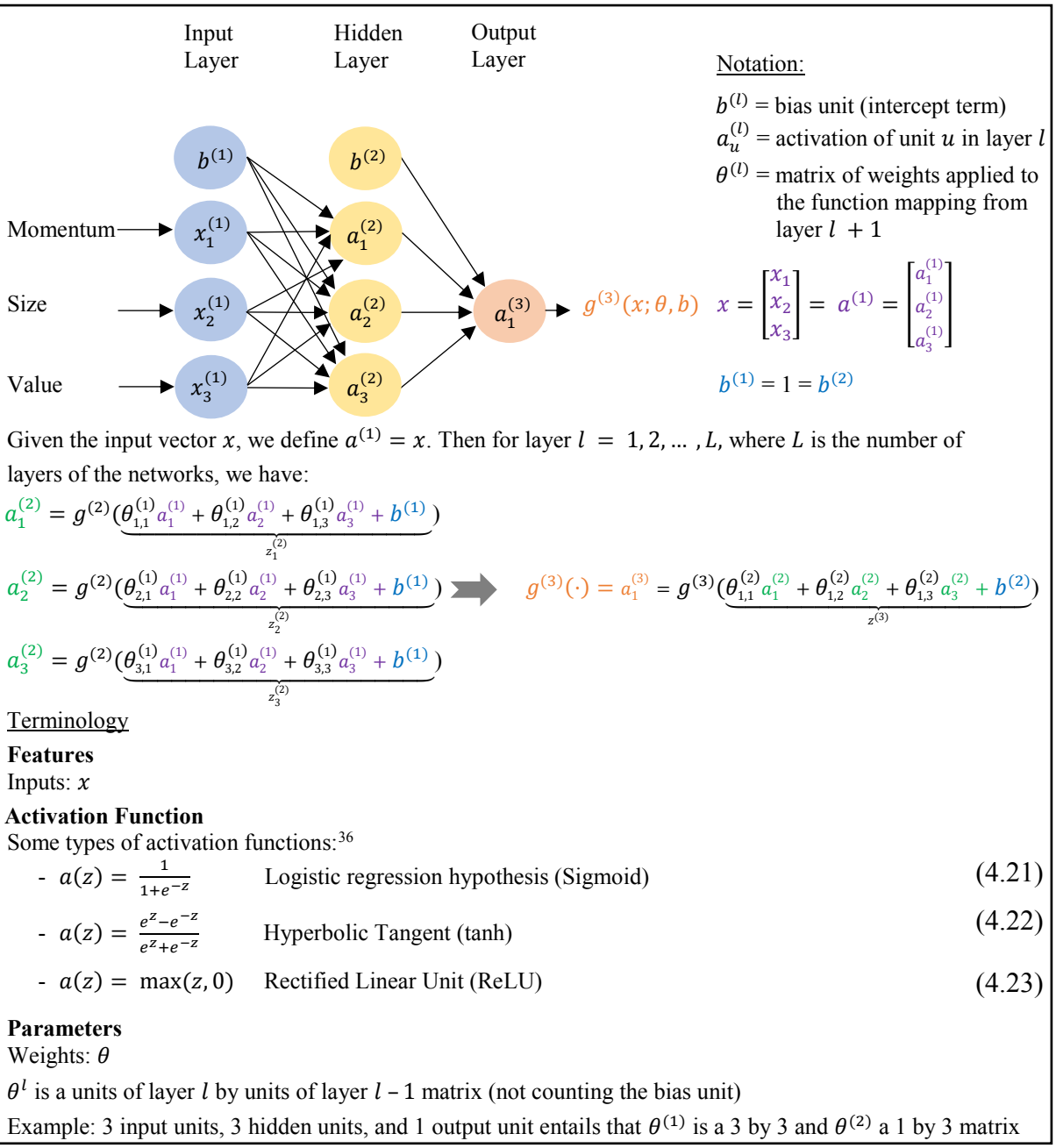



*Figure 6.* A simple neural network with one hidden layer.[37] The blue circles indicate the input layer, the yellow circles the hidden layer, and the red circle the output layer. Each arrow is associated with a weight parameter. In this neural network, a nonlinear activation function transforms the inputs before passing them on to the output.

The vectorized implementation of these calculations is called **forward-propagation**. Forward-propagation computes the value of the output layer, $g(x;\theta,b)$, in terms of the values of input features, $x_0, \dots, x_3$, propagated through the layers of a neural network. Specifically, each neuron draws information linearly from all input units, as in the simple network shown in Figure 5. Next, each neuron applies a nonlinear *activation function* to its aggregated signal before

[36] A visualization of these functions can be found in Figure A.15.
[37] Note that bias units do not have inputs or connections going into them because they always output the value +1.

transmitting its output to the next layer. Finally, the results of the individual neurons are aggregated linearly into a final output prediction.[38]

There are several architectural options for building a neural network, including the number of hidden layers, the number of neurons in each layer, and which units to connect. Selecting a successful network architecture via our validation set approach is a difficult task. Therefore, we considered a variety of different network architectures with up to three hidden layers. Our shallowest neural network (NN1) had a single hidden layer of 21 neurons. NN2 had two hidden layers with 21 and 10 neurons, respectively. The last network in our implementation had three hidden layers with 21, 10, and 5 neurons, respectively. All architectures were fully connected so that each unit received input from all units of the previous layer.

The neural network was parameterized by applying the well-known **back-propagation** algorithm. Specifically, given a fixed training set $\{(x_{i,t}, \mathrm{r}_{\mathrm{i,t+1}}), \dots, (x_{N_t,T}, \mathrm{r}_{N_t,\mathrm{T+1}})\}$ of $N_t$ training examples, we trained our neural network using *mini batch gradient descent*. For a single training example $(x, r)$, the cost function with respect to that single example was defined for every $t$ as follows:

$$\mathcal{L}(\theta, b; x, r) = \frac{1}{2}\|g(x; \theta, b) - r\|^2. \tag{4.24}$$

Equation 4.24 represents the (one-half) squared-error cost function. Consequently, given a training set of $N$ examples, the total cost function can be defined as:

$$\begin{aligned}\mathcal{L}(\theta, b; \cdot) &= \left[\frac{1}{\mathrm{N}}\sum_{i=1}^{N}\mathcal{L}(\theta, b; x_i, \mathrm{r}_i)\right] + \frac{\lambda}{2}\sum_{l=1}^{L-1}\sum_{u^{(l)}=1}^{U^{(l)}}\sum_{u^{(l+1)}=1}^{U^{(l+1)}}\left(\theta^{(l)}_{u^{(l+1)}u^{(l)}}\right)^2 \\ &= \left[\frac{1}{\mathrm{N}}\sum_{i=1}^{N}\left(\frac{1}{2}\|g(x_i; \theta, b) - \mathrm{r}_i\|^2\right)\right] + \frac{\lambda}{2}\sum_{l=1}^{L-1}\sum_{u^{(l)}=1}^{U^{(l)}}\sum_{u^{(l+1)}=1}^{U^{(l+1)}}\left(\theta^{(l)}_{u^{(l+1)}u^{(l)}}\right)^2\end{aligned} \tag{4.25}$$

The first term in Equation 4.25 represents an average sum of squares, and the second term (also called a weight decay) incorporates regularization, which tends to reduce the size of the weights to help prevent overfitting. The weight decay parameter $\lambda$ controls the relative importance of these two terms. Typically, weight decay is not applied to the bias terms $b^{(l)}$, as reflected in Equation 4.25. However, applying weight decay to the bias units usually only makes a small difference to the final network.

[38] We assume that the nonlinearities $g^{(l)}$ for all layers except the last layer $L$ are the same. This is because we apply a regression in the output layer (i.e., use $g(x) = x$). Thus, we distinguish $g^{(L)}$ from $g^{(l)}$, and assume that $g^{(l)}$ is used for all layers but layer $L$.

Our goal was to minimize $\mathcal{L}(\theta, b;\cdot)$ as a function of $\theta$ and $b$. To train our neural network, we initialized parameters $\theta^{(l)}_{u^{(l)},u^{l+1}}$ and $b^{(l)}$ with a small, random value close to zero[39] and then applied an optimization algorithm such as mini batch gradient descent.[40] Because $\mathcal{L}(\theta, b;\cdot)$ is a non-convex function, the gradient descent algorithm is susceptible to converging on local optima rather than the global optima, but in practice it usually performs quite well. One iteration of gradient descent updated the parameters $\theta, b$ as follows:

$$\theta^{(l)}_{u^{(l)},u^{l+1}} = \theta^{(l)}_{u^{(l)},u^{l+1}} - \alpha \frac{\partial}{\partial \theta^{(l)}_{u^{(l)},u^{l+1}}} \mathcal{L}(\theta, b;\cdot)$$
$$b^{(l)} = b^{(l)} - \alpha \frac{\partial}{\partial b^{(l)}} \mathcal{L}(\theta, b;\cdot) \tag{4.26}$$

where $\alpha$ represents the learning rate. The crucial step was to compute the partial derivatives in equation 4.26, and the following equations describe the back-propagation algorithm, meant to provide an efficient way to compute these partial derivatives. Once $\frac{\partial}{\partial \theta^{(l)}_{u^{(l)},u^{l+1}}} \mathcal{L}(\theta, b; x, r)$ and $\frac{\partial}{\partial b^{(l)}} \mathcal{L}(\theta, b; x, r)$ – the partial derivatives of the cost function $\mathcal{L}(\theta, b; x, r)$ defined with respect to a single example $(x, r)$ – were calculated, the derivative of the overall cost function $\mathcal{L}(\theta, b)$ could be computed as follows:

$$\frac{\partial}{\partial \theta^{(l)}_{u^{(l)},u^{l+1}}} \mathcal{L}(\theta, b) = \left[ \frac{1}{\mathrm{N}} \sum_{i=1}^{N} \frac{\partial}{\partial \theta^{(l)}_{u^{(l)},u^{l+1}}} \mathcal{L}(\theta, b; x_i, \mathrm{r}_i) \right] + \lambda \theta^{(l)}_{u^{(l)},u^{l+1}}$$
$$\frac{\partial}{\partial b^{(l)}} \mathcal{L}(\theta, b) = \frac{1}{\mathrm{N}} \sum_{i=1}^{N} \frac{\partial}{\partial b^{(l)}} \mathcal{L}(\theta, b; x_i, \mathrm{r}_i) \tag{4.27}$$

The two lines above differ slightly because weight decay is applied to $\theta$, but not $b$. The intuition behind the back-propagation algorithm can be described as follows: Using a training example $(x, r)$, we first performed a "forward pass" to calculate all activations throughout the network, including the final output $g(x; \theta, b)$. Next, for each node $u$ in layer $l$, we computed an "error term" $\delta^{(l)}_u$, which measured how much this node was "responsible" for errors in our output. For an output node, we could directly calculate the difference between the activation of the network and the true target value, defining $\delta^{(L)}_u$ (where layer $L$ is the output layer). For the hidden units,

[39] For example, according to a $Normal(0, \epsilon^2)$ distribution for some small $\epsilon$.
[40] Note that it is crucial to initialize the parameters randomly rather than setting them all to 0; that is, if all parameters start at identical values, then all hidden layer units learn the same input function. More formally, $\theta^{(l)}_{u^{(l)},u^{l+1}}$ would be the same for all values of $u$, such that $a^{(2)}_1 = a^{(2)}_2 = a^{(2)}_3 = \cdots$ for any input $x$. Thus, random initialization serves the purpose of *symmetry breaking*.

on the other hand, we had to compute $\delta_u^{(l)}$ based on a weighted average of the error terms of the nodes that used $a_u^{(l)}$ as an input.[41]

Neural networks contain numerous tuning parameters, and a detailed explanation of these parameters would go beyond the scope of this thesis. Table 7 provides a brief overview of the most important parameters.

| **Hyperparameter** | **Description** |
|---|---|
| Activation function | Process the inputs flowing into each neuron. We considered the functions given in Equations 4.21 – 4.23. |
| Regularization | *Dropout*<br>Removes certain connections in every iteration. Thus, the hidden units cannot depend on any particular feature too much.<br>*L1/L2 Regularization.*<br>Curbs high weight values so that the model is not dependent on a single feature.<br>*Early Stopping*<br>Stops training at the point when performance on a validation dataset starts to degrade. |
| Learning rate | Determines how fast the back-propagation algorithm performs gradient descent. |
| Optimizer | Updates weights of every layer after each iteration. Popular choices are SGD, RMSProp, and Adam.<br>We adopted the Adam optimizer, which is straightforward to implement, computationally efficient, and well suited for problems that are large in terms of data and/or parameters (Kingma & Ba, 2015). |
| Batch size | The number of training examples shown to the network before the weight matrix is updated. |
| Number of epochs | The number of times or iterations the entire training data is shown to the model. |

*Table 7.* Important hyperparameters in neural networks. Table 7 provides an overview of the most important hyperparameters for neural networks.

[41] For details of the back-propagation algorithm please refer to table A.4.

As above, we adaptively chose the combination of hyperparameters according to the validation set approach.

Table 8 summarizes the distribution (average, standard deviation, 10th and 90th percentiles) of the resulting monthly return forecasts for NN1:

| | **Rolling perspective** | **Expanding perspective** |
|---|---|---|
| *Avg* | 0.09 | 0.06 |
| *Std* | 0.90 | 0.92 |
| *p10* | -0.88 | -0.96 |
| *p90* | 1.07 | 1.09 |

*Table 8*. Cross-sectional stock forecasts, 1999 to 2018. Table 8 reports the distribution (average, standard deviation, 10th and 90th percentiles) of 1-month forecasts derived from current firm characteristics and 10-year rolling or expanding coefficients. All numbers equal time-series averages of monthly cross-sectional parameters. Estimates of expected 1-month returns derived from neural networks vary substantially across stocks.

# 5 Results

In this section, we present the main results of our models derived in Section 4 for an rolling scheme (results for an expanding scheme can be found in the Appendix). To assess the cross-sectional return forecast, we followed Lewellen (2015) and analyzed the slope coefficients in Fama & MacBeth (1973) regressions of realized on predicted returns. Our primary goal was to determine whether our predictions corresponded with realized returns. More specifically, were the models capable to forecast realized returns with a slope of one, demonstrating that they provided reliable estimates?

As seen in Lewellen (2015) and Green, Hand, & Zhang (2017), we also evaluated the economic value of our cross-sectional return forecasts by creating monthly spread portfolios that go long (short) the top (bottom) decile of companies with the highest (lowest) predicted returns.

Both conventional and machine learning methods shared the same set of characteristics, therefore, differences in performance reflect the manner in which the information was digested. Noting these differences can help determine whether a model exists that can effectively synthesize and dissect the high-dimensional set of factors into a return prediction model that provides more accurate predictions in comparison to traditional approaches.

## 5.1 Predictive Slope

Following Lewellen (2015), we analyzed the cross-sectional return forecast by estimating Fama-MacBeth regressions that relate realized returns to forecasted returns. Specifically, for each month $t$, the following cross-sectional regression was estimated:

$$r_{i,t+1} = a_t^{FM} + b_t^{FM} g\left(x_{i,t}\right) + \epsilon_{i,t+1}, \tag{5.1}$$

where stocks were indexed by $i = 1, \dots, N_t$, months were indexed by $t = 1, \dots, T$, and $g\left(x_{i,t}\right)$ denoted a prediction of $r_{i,t+1}$. Subsequently, the time-series average of monthly slope coefficient estimates in Equation 5.1 were computed as follows:

$$\hat{b}^{FM} = \frac{1}{T}\sum_{t=1}^{T} \hat{b}_t^{FM}, \tag{5.2}$$

where $\hat{b}_t^{FM}$ represented the OLS estimate of $b_t^{FM}$ in Equation 5.1. To account for possible autocorrelation in the $\hat{b}_t^{FM}$ estimates, the *t*-statistic for $\hat{b}^{FM}$ incorporated a Newey & West (1987) correction with four lags.

The magnitude of $\hat{b}^{FM}$ reflects how well the cross-sectional return forecasts capture the cross-sectional dispersion in expected returns (Lewellen, 2015); this provides a direct measure of

overfitting (Han, He, Rapach, & Zhou, 2018). Similar to a Mincer & Zarnowitz (1969) regression, $\hat{b}^{FM} = 1$ indicates that the cross-sectional return forecasts are unbiased. In other words, a unit increase in forecasted returns corresponds, on average, to a unit increase in realized returns. In the case of $\hat{b}^{FM} < 1$, the cross-sectional predictions tend to exaggerate the cross-sectional dispersion in expected returns. This implies that an increase in predicted returns, on average, corresponds to a lower increase in realized returns, suggesting overfitting (Han, He, Rapach, & Zhou, 2018). Finally, if $\hat{b}^{FM} > 1$, an increase in predicted returns, on average, corresponds to a higher increase in realized returns. Due to the inherently large unpredictable nature of monthly equity returns, forecasts characterized by $\hat{b}^{FM} > 1$ are expected to perform better than forecasts characterized by $\hat{b}^{FM} < 1$ (Han, He, Rapach, & Zhou, 2018).

Table 9 reports Fama-Macbeth regression results for out-of-sample cross-sectional return forecasts based on the 21 characteristics described in the Appendix.

| **Method** | **Coefficient** | **S.E.** | ***t*-stat** |
|---|---|---|---|
| Fama & MacBeth | 0.61 | 0.10 | 5.82 |
| Pooled OLS | 0.51 | 0.10 | 5.26 |
| Elastic-net | 0.54 | 0.10 | 5.29 |
| Lasso | 0.54 | 0.10 | 5.28 |
| Ridge | 0.51 | 0.10 | 5.26 |
| GLM | 0.77 | 0.12 | 6.35 |
| Random forest | 0.88 | 0.11 | 8.10 |
| Boosted Regression Tree | 0.69 | 0.10 | 7.19 |
| NN1 | 0.51 | 0.06 | 8.17 |
| NN2 | 0.51 | 0.06 | 7.95 |
| NN3 | 0.60 | 0.07 | 8.68 |

*Table 9*. Fama-MacBeth regression results. Table 9 reports the predictive ability (slope, standard error, *t*-statistic) of 1-month return forecasts derived from current firm characteristics and 10-year rolling coefficients. All numbers equal time-series averages of monthly cross-sectional parameters. Predictive slopes come from (out-of-sample) Fama-MacBeth regressions of 1-month returns on the return forecasts; *t*-statistics are based on the time-series variability of the predictive slopes, incorporating a Newey-West correction with 4 lags. Estimates of expected 1-month returns derived from all models described in section 4 have significant out-of-sample predictive power for actual 1-month returns.

## 5.2 Portfolios

We also assessed cross-sectional return forecasts by building portfolios. The performance metric of portfolios, which are composed of cross-sectional return forecasts, indirectly evaluate the forecasts by measuring their value as input in asset allocation considerations.

To construct portfolios, we calculated one-month-ahead out-of-sample stock return predictions for each method at the end of every month. Subsequently, we sorted the stocks into deciles based

on the forecasts of each model. More formally, given the weight of asset $i$ in portfolio $p$ ($w_{i,t}^{p}$) and a model-based out-of-sample prediction for asset $i$ ($g^{*}(\cdot)$), the portfolio return forecast was calculated as follows:

$$g^{p}(\cdot) = \sum_{i=1}^{n} w_{i,t}^{p} \times g\left(x_{i,t}\right). \tag{5.3}$$

We reconstituted portfolios each month using equal weights, the most natural weighting because the objective functions minimized equal-weighted prediction errors. Finally, we built zero-net-investment spread portfolios that buy (sell) the highest (lowest) expected return stocks (decile 1 minus decile 10).

Table 10 reports the average predicted monthly returns (Pred), the average realized monthly returns (Avg), their realized standard deviations (Std), the information ratios (IR), the sortino ratios (SR), the most extreme negative monthly returns (Max 1M Loss), the maximum drawdowns (Max DD), the maximum drawdown durations (Max DD DUR), the average percentage changes in holdings (TO),[42] and the *t*-statistics (*t*-stat) of equally-weighted decile portfolios sorted on out-of-sample return forecasts.

We defined the maximum drawdown of a strategy as follows:

$$Max\ DD = \max_{0 \le t_1 \le t_1 \le T} \left(Y_{t_1} - Y_{t_2}\right), \tag{5.4}$$

where $Y_t$ is the cumulative return from date 0 to $t$. The average monthly percentage change in holdings was defined as:

$$Turnover = \frac{1}{T}\sum_{t=1}^{T}\left(\sum_{i}\left|\Delta w_{i,t}\right|\right), \tag{5.5}$$

where $w_{i,t}$ represents the weight of stock $i$ in portfolio at time $t$.

[42] We show the two-sided turnovers. The turnovers for the zero-net-investment spread portfolios (decile 1 portfolios and decile 10 portfolios) are divided by two to reflect turnovers of only one of the two portfolios.

| Fama & MacBeth | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 0.92 | 0.62 | 2.11 | 1.01 | 1.73 | 12 | 16 | 244 | 1.38 | 4.09 |
| 2 | 0.40 | 0.34 | 1.06 | 1.10 | 1.92 | 3 | 9 | 489 | 1.59 | 4.52 |
| 3 | 0.19 | 0.23 | 0.93 | 0.87 | 1.47 | 3 | 9 | 942 | 1.64 | 3.41 |
| 4 | 0.04 | 0.18 | 0.92 | 0.68 | 1.06 | 3 | 10 | 518 | 1.67 | 2.77 |
| 5 | -0.10 | 0.08 | 0.89 | 0.30 | 0.46 | 3 | 11 | 1306 | 1.68 | 1.15 |
| 6 | -0.23 | 0.02 | 0.93 | 0.06 | 0.08 | 4 | 13 | 2557 | 1.67 | 0.27 |
| 7 | -0.37 | -0.08 | 0.88 | -0.33 | -0.44 | 4 | 24 | 6270 | 1.66 | -1.51 |
| 8 | -0.54 | -0.31 | 0.86 | -1.27 | -1.40 | 4 | 71 | 6909 | 1.62 | -5.31 |
| 9 | -0.78 | -0.50 | 1.04 | -1.66 | -1.64 | 4 | 115 | 6454 | 1.54 | -6.98 |
| Low (L) | -1.45 | -0.76 | 3.23 | -0.81 | -1.01 | 12 | 184 | 5813 | 1.28 | -3.27 |
| H-L | 2.37 | 1.37 | 3.78 | 1.26 | 2.37 | 13 | 22 | 91 | 1.31 | 5.32 |
| Pooled OLS | | | | | | | | | | |
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.35 | 0.57 | 2.67 | 0.74 | 1.48 | 10 | 18 | 275 | 1.07 | 2.60 |
| 2 | 0.68 | 0.26 | 1.00 | 0.91 | 1.58 | 3 | 10 | 792 | 1.36 | 3.51 |
| 3 | 0.44 | 0.21 | 1.01 | 0.73 | 1.16 | 3 | 12 | 426 | 1.48 | 2.73 |
| 4 | 0.28 | 0.14 | 0.94 | 0.53 | 0.78 | 5 | 11 | 335 | 1.54 | 2.06 |
| 5 | 0.13 | 0.09 | 0.92 | 0.33 | 0.47 | 4 | 11 | 306 | 1.58 | 1.38 |
| 6 | 0 | 0 | 0.92 | -0.01 | -0.01 | 4 | 20 | 2894 | 1.55 | -0.03 |
| 7 | -0.15 | -0.11 | 0.86 | -0.44 | -0.54 | 4 | 28 | 3500 | 1.54 | -1.82 |
| 8 | -0.32 | -0.21 | 0.85 | -0.87 | -1.00 | 3 | 54 | 6634 | 1.47 | -3.32 |
| 9 | -0.56 | -0.38 | 1.05 | -1.24 | -1.35 | 4 | 91 | 6482 | 1.36 | -5.05 |
| Low (L) | -1.28 | -0.77 | 2.85 | -0.93 | -1.13 | 10 | 184 | 6849 | 1.05 | -3.95 |
| H-L | 2.62 | 1.34 | 3.80 | 1.22 | 2.52 | 13 | 20 | 182 | 1.05 | 4.92 |
| Elastic-net | | | | | | | | | | |
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.29 | 0.58 | 2.71 | 0.75 | 1.51 | 10 | 19 | 275 | 1.07 | 2.65 |
| 2 | 0.65 | 0.27 | 0.97 | 0.96 | 1.70 | 3 | 7 | 1309 | 1.35 | 3.70 |
| 3 | 0.42 | 0.21 | 1.03 | 0.72 | 1.11 | 5 | 12 | 426 | 1.47 | 2.69 |
| 4 | 0.26 | 0.16 | 0.93 | 0.59 | 0.88 | 4 | 11 | 398 | 1.55 | 2.23 |
| 5 | 0.13 | 0.12 | 0.92 | 0.44 | 0.63 | 4 | 11 | 427 | 1.56 | 1.85 |
| 6 | -0.01 | -0.02 | 0.95 | -0.09 | -0.12 | 4 | 18 | 2894 | 1.55 | -0.36 |
| 7 | -0.14 | -0.08 | 0.84 | -0.34 | -0.44 | 3 | 26 | 6027 | 1.54 | -1.50 |
| 8 | -0.31 | -0.26 | 0.91 | -0.97 | -1.10 | 4 | 63 | 6634 | 1.48 | -3.35 |
| 9 | -0.55 | -0.40 | 1.04 | -1.33 | -1.43 | 4 | 96 | 6512 | 1.36 | -5.19 |
| Low (L) | -1.22 | -0.77 | 2.87 | -0.92 | -1.12 | 10 | 184 | 6849 | 1.06 | -3.95 |
| H-L | 2.51 | 1.35 | 3.87 | 1.21 | 2.50 | 14 | 20 | 182 | 1.06 | 4.90 |

| Lasso | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.28 | 0.59 | 2.74 | 0.74 | 1.52 | 10 | 19 | 275 | 1.07 | 2.63 |
| 2 | 0.64 | 0.27 | 0.98 | 0.96 | 1.70 | 2 | 7 | 1159 | 1.35 | 3.75 |
| 3 | 0.42 | 0.21 | 1.03 | 0.70 | 1.09 | 5 | 12 | 426 | 1.47 | 2.71 |
| 4 | 0.26 | 0.17 | 0.93 | 0.64 | 0.97 | 4 | 12 | 398 | 1.54 | 2.35 |
| 5 | 0.12 | 0.10 | 0.92 | 0.38 | 0.55 | 4 | 10 | 336 | 1.56 | 1.60 |
| 6 | -0.01 | -0.03 | 0.94 | -0.10 | -0.14 | 4 | 19 | 2894 | 1.55 | -0.42 |
| 7 | -0.14 | -0.08 | 0.86 | -0.32 | -0.42 | 3 | 26 | 6027 | 1.54 | -1.42 |
| 8 | -0.31 | -0.26 | 0.89 | -0.99 | -1.11 | 4 | 63 | 6634 | 1.48 | -3.44 |
| 9 | -0.54 | -0.40 | 1.05 | -1.32 | -1.41 | 4 | 96 | 6482 | 1.36 | -5.12 |
| Low (L) | -1.20 | -0.77 | 2.88 | -0.92 | -1.12 | 10 | 184 | 6849 | 1.06 | -3.94 |
| H-L | 2.49 | 1.35 | 3.89 | 1.21 | 2.50 | 14 | 20 | 182 | 1.06 | 4.91 |
| Ridge | | | | | | | | | | |
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.35 | 0.57 | 2.67 | 0.74 | 1.48 | 10 | 18 | 275 | 1.07 | 2.60 |
| 2 | 0.68 | 0.26 | 1.00 | 0.91 | 1.58 | 3 | 10 | 792 | 1.36 | 3.51 |
| 3 | 0.44 | 0.21 | 1.01 | 0.73 | 1.16 | 3 | 12 | 426 | 1.48 | 2.73 |
| 4 | 0.28 | 0.14 | 0.94 | 0.53 | 0.78 | 5 | 11 | 335 | 1.54 | 2.06 |
| 5 | 0.13 | 0.09 | 0.92 | 0.33 | 0.47 | 4 | 11 | 306 | 1.58 | 1.38 |
| 6 | 0 | 0 | 0.92 | -0.01 | -0.01 | 4 | 20 | 2894 | 1.55 | -0.02 |
| 7 | -0.15 | -0.11 | 0.86 | -0.44 | -0.54 | 4 | 29 | 3500 | 1.54 | -1.82 |
| 8 | -0.32 | -0.21 | 0.85 | -0.87 | -1.01 | 3 | 54 | 6634 | 1.47 | -3.33 |
| 9 | -0.56 | -0.38 | 1.05 | -1.24 | -1.35 | 4 | 91 | 6482 | 1.36 | -5.05 |
| Low (L) | -1.28 | -0.77 | 2.85 | -0.93 | -1.13 | 10 | 184 | 6849 | 1.05 | -3.95 |
| H-L | 2.62 | 1.34 | 3.80 | 1.22 | 2.52 | 13 | 20 | 182 | 1.05 | 4.92 |
| GLM | | | | | | | | | | |
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.04 | 0.61 | 2.46 | 0.86 | 1.52 | 11 | 22 | 275 | 0.99 | 3.22 |
| 2 | 0.57 | 0.38 | 1.19 | 1.12 | 2.33 | 2 | 6 | 151 | 1.39 | 4.96 |
| 3 | 0.36 | 0.31 | 0.86 | 1.24 | 2.40 | 3 | 3 | 182 | 1.51 | 5.70 |
| 4 | 0.21 | 0.23 | 0.93 | 0.84 | 1.46 | 2 | 7 | 306 | 1.55 | 3.40 |
| 5 | 0.08 | 0.10 | 0.82 | 0.43 | 0.70 | 2 | 13 | 2863 | 1.56 | 1.50 |
| 6 | -0.04 | -0.04 | 0.91 | -0.15 | -0.20 | 5 | 25 | 3014 | 1.55 | -0.59 |
| 7 | -0.16 | -0.12 | 0.90 | -0.48 | -0.57 | 5 | 30 | 3773 | 1.53 | -1.83 |
| 8 | -0.29 | -0.33 | 0.91 | -1.26 | -1.37 | 4 | 76 | 6482 | 1.47 | -4.82 |
| 9 | -0.47 | -0.52 | 1.13 | -1.58 | -1.60 | 4 | 119 | 6482 | 1.34 | -6.42 |
| Low (L) | -0.88 | -0.81 | 2.50 | -1.13 | -1.27 | 10 | 191 | 6849 | 1.07 | -4.75 |
| H-L | 1.92 | 1.42 | 3.70 | 1.33 | 2.81 | 10 | 21 | 305 | 1.03 | 5.74 |

| Random Forest | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.01 | 0.95 | 2.42 | 1.37 | 2.89 | 10 | 21 | 273 | 1.19 | 5.20 |
| 2 | 0.63 | 0.54 | 1.23 | 1.52 | 3.19 | 5 | 13 | 242 | 1.47 | 5.14 |
| 3 | 0.45 | 0.33 | 0.89 | 1.29 | 2.56 | 2 | 5 | 151 | 1.59 | 5.54 |
| 4 | 0.30 | 0.17 | 0.77 | 0.76 | 1.19 | 3 | 5 | 396 | 1.66 | 3.09 |
| 5 | 0.16 | 0.08 | 0.81 | 0.36 | 0.52 | 3 | 13 | 549 | 1.67 | 1.34 |
| 6 | 0.02 | -0.04 | 0.85 | -0.15 | -0.20 | 4 | 18 | 4351 | 1.67 | -0.57 |
| 7 | -0.13 | -0.27 | 0.93 | -1.00 | -1.11 | 5 | 62 | 6575 | 1.67 | -3.70 |
| 8 | -0.30 | -0.45 | 1.00 | -1.56 | -1.56 | 5 | 101 | 6879 | 1.62 | -5.79 |
| 9 | -0.54 | -0.53 | 1.11 | -1.64 | -1.63 | 5 | 121 | 6514 | 1.50 | -5.87 |
| Low (L) | -1.08 | -0.99 | 2.47 | -1.39 | -1.51 | 11 | 226 | 6757 | 1.08 | -5.24 |
| H-L | 2.09 | 1.94 | 3.56 | 1.89 | 4.56 | 10 | 26 | 305 | 1.13 | 7.43 |
| Boosted Regression Trees | | | | | | | | | | |
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.85 | 1.03 | 3.36 | 1.06 | 2.92 | 8 | 20 | 151 | 1.16 | 3.68 |
| 2 | 0.55 | 0.39 | 1.23 | 1.11 | 1.90 | 5 | 12 | 274 | 1.45 | 4.39 |
| 3 | 0.36 | 0.26 | 1.06 | 0.86 | 1.39 | 4 | 11 | 669 | 1.58 | 3.40 |
| 4 | 0.22 | 0.18 | 1.06 | 0.58 | 0.85 | 6 | 15 | 518 | 1.61 | 2.14 |
| 5 | 0.09 | 0.07 | 1.07 | 0.24 | 0.34 | 6 | 21 | 3012 | 1.62 | 0.82 |
| 6 | -0.04 | -0.04 | 1.04 | -0.13 | -0.17 | 6 | 25 | 2769 | 1.63 | -0.51 |
| 7 | -0.17 | -0.22 | 1.05 | -0.72 | -0.85 | 7 | 53 | 4962 | 1.60 | -2.68 |
| 8 | -0.33 | -0.36 | 1.01 | -1.23 | -1.30 | 6 | 81 | 6879 | 1.57 | -4.74 |
| 9 | -0.53 | -0.53 | 1.21 | -1.52 | -1.52 | 5 | 122 | 6482 | 1.45 | -5.63 |
| Low (L) | -1.21 | -0.97 | 2.60 | -1.30 | -1.41 | 10 | 225 | 6849 | 1.10 | -5.23 |
| H-L | 3.05 | 2.00 | 3.56 | 1.95 | 6.21 | 8 | 15 | 151 | 1.11 | 6.97 |
| NN1 | | | | | | | | | | |
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.66 | 0.70 | 2.44 | 0.99 | 1.87 | 8 | 17 | 394 | 1.21 | 4.23 |
| 2 | 0.86 | 0.46 | 1.10 | 1.45 | 3.15 | 3 | 10 | 333 | 1.49 | 5.98 |
| 3 | 0.56 | 0.27 | 0.82 | 1.14 | 2.04 | 3 | 6 | 455 | 1.59 | 4.77 |
| 4 | 0.35 | 0.19 | 0.92 | 0.70 | 1.12 | 3 | 14 | 485 | 1.64 | 2.51 |
| 5 | 0.17 | 0.11 | 0.90 | 0.42 | 0.65 | 4 | 13 | 853 | 1.64 | 1.48 |
| 6 | 0 | -0.04 | 0.97 | -0.13 | -0.17 | 4 | 18 | 5632 | 1.66 | -0.51 |
| 7 | -0.18 | -0.16 | 0.93 | -0.61 | -0.72 | 5 | 39 | 6757 | 1.66 | -2.80 |
| 8 | -0.39 | -0.36 | 0.89 | -1.39 | -1.42 | 5 | 81 | 6694 | 1.61 | -6.02 |
| 9 | -0.68 | -0.47 | 1.06 | -1.55 | -1.57 | 5 | 107 | 6514 | 1.50 | -6.13 |
| Low (L) | -1.46 | -0.88 | 2.25 | -1.36 | -1.48 | 10 | 197 | 6909 | 1.23 | -4.69 |
| H-L | 3.12 | 1.58 | 2.90 | 1.89 | 4.42 | 9 | 15 | 92 | 1.20 | 7.78 |

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | NN2 | | | | | |
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.48 | 0.77 | 2.39 | 1.12 | 2.31 | 7 | 16 | 92 | 1.16 | 4.18 |
| 2 | 0.74 | 0.42 | 1.10 | 1.34 | 2.51 | 4 | 4 | 61 | 1.48 | 6.24 |
| 3 | 0.47 | 0.26 | 0.98 | 0.92 | 1.58 | 4 | 8 | 518 | 1.59 | 3.66 |
| 4 | 0.27 | 0.18 | 0.91 | 0.68 | 1.12 | 4 | 15 | 518 | 1.62 | 2.56 |
| 5 | 0.08 | 0.03 | 0.90 | 0.13 | 0.17 | 4 | 16 | 703 | 1.66 | 0.50 |
| 6 | -0.10 | -0.05 | 0.97 | -0.17 | -0.22 | 4 | 22 | 3014 | 1.66 | -0.59 |
| 7 | -0.29 | -0.13 | 0.87 | -0.54 | -0.67 | 3 | 34 | 5691 | 1.65 | -2.17 |
| 8 | -0.52 | -0.29 | 0.94 | -1.06 | -1.19 | 4 | 65 | 5874 | 1.59 | -4.82 |
| 9 | -0.82 | -0.48 | 1.18 | -1.40 | -1.47 | 4 | 107 | 6727 | 1.47 | -6.20 |
| Low (L) | -1.57 | -0.91 | 2.42 | -1.30 | -1.40 | 9 | 211 | 6849 | 1.15 | -5.32 |
| H-L | 3.05 | 1.68 | 3.11 | 1.86 | 4.70 | 7 | 10 | 92 | 1.14 | 8.31 |
| | | | | | NN3 | | | | | |
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.34 | 0.75 | 2.51 | 1.04 | 2.32 | 7 | 17 | 394 | 1.23 | 3.58 |
| 2 | 0.70 | 0.42 | 1.27 | 1.13 | 2.21 | 4 | 9 | 92 | 1.48 | 6.46 |
| 3 | 0.48 | 0.27 | 0.97 | 0.98 | 1.65 | 3 | 9 | 483 | 1.57 | 3.90 |
| 4 | 0.30 | 0.20 | 0.89 | 0.76 | 1.22 | 3 | 12 | 882 | 1.63 | 3.01 |
| 5 | 0.14 | 0.08 | 0.93 | 0.31 | 0.45 | 4 | 11 | 1219 | 1.66 | 1.28 |
| 6 | -0.03 | -0.10 | 0.92 | -0.38 | -0.47 | 5 | 32 | 4291 | 1.66 | -1.38 |
| 7 | -0.22 | -0.19 | 0.99 | -0.67 | -0.79 | 5 | 46 | 6329 | 1.65 | -2.65 |
| 8 | -0.44 | -0.34 | 0.95 | -1.25 | -1.36 | 5 | 80 | 6362 | 1.61 | -5.67 |
| 9 | -0.74 | -0.47 | 1.12 | -1.46 | -1.48 | 4 | 105 | 6634 | 1.49 | -6.57 |
| Low (L) | -1.38 | -0.81 | 2.05 | -1.37 | -1.51 | 7 | 181 | 6909 | 1.16 | -5.42 |
| H-L | 2.72 | 1.56 | 2.94 | 1.84 | 4.60 | 7 | 7 | 179 | 1.19 | 7.92 |

*Table 10*. Performance of equal-weighted decile portfolios sorted on out-of-sample return forecasts. Table 10 reports the average predicted monthly returns (Pred), the average realized monthly returns (Avg), their realized standard deviations (Std), the information ratios (IR), the Sortino ratios (SR), the most extreme negative monthly returns (Max 1M Loss), the maximum drawdowns (Max DD), the maximum drawdown durations (Max DD DUR), the average percentage changes in holdings (TO), and the *t*-statistics (*t*-stat) of equally-weighted decile portfolios sorted on out-of-sample return forecasts.

In reference to the H-L spread portfolios, the regularized linear methods failed to outperform our benchmark models. Moreover, the generalized linear model, which introduced nonlinearity via spline functions of each baseline predictor, also did not substantially surpass the linear specifications. However, tree-based methods and neural networks that allowed for nonlinear predictor interactions substantially outperformed both benchmark models. This suggests that allowing for complex nonlinear predictor interactions is critical.

Tree-based methods showed the best performance in terms of H-L returns (Avg), followed by neural networks. The *t*-statistics (*t*-stat), which incorporated a Newy-West correction with four lags to account for possible autocorrelation, were highest for neural networks and tree based methods, with a slightly lower *t*-statistic for boosted regression trees. Regarding the H-L

information ratio (IR), both tree-based methods and neural networks clearly outperformed all other methods by a similar margin. This indicated that standard deviations (Std), most extreme negative monthly returns (Max 1M Loss), and maximum drawdowns (Max DD) were lower for neural networks than tree-based methods, because the information ratio is a measure of the risk-adjusted returns. When examining the Sortino ratios (SR), which isolate downside volatility from total volatility by dividing excess return by the downside deviation, tree-based methods and neural networks showed comparable performance. Boosted regression trees showed high returns and low downward volatility, thus demonstrating an even higher Sortino ratio. The two-sided turnover (TO) was highest for Fama MacBeth regressions, but generally quite similar for all methods, ranging in values from 1.03 to 1.31.

Figure 7 displays the cumulative performance for the spread portfolios of all strategies over time.

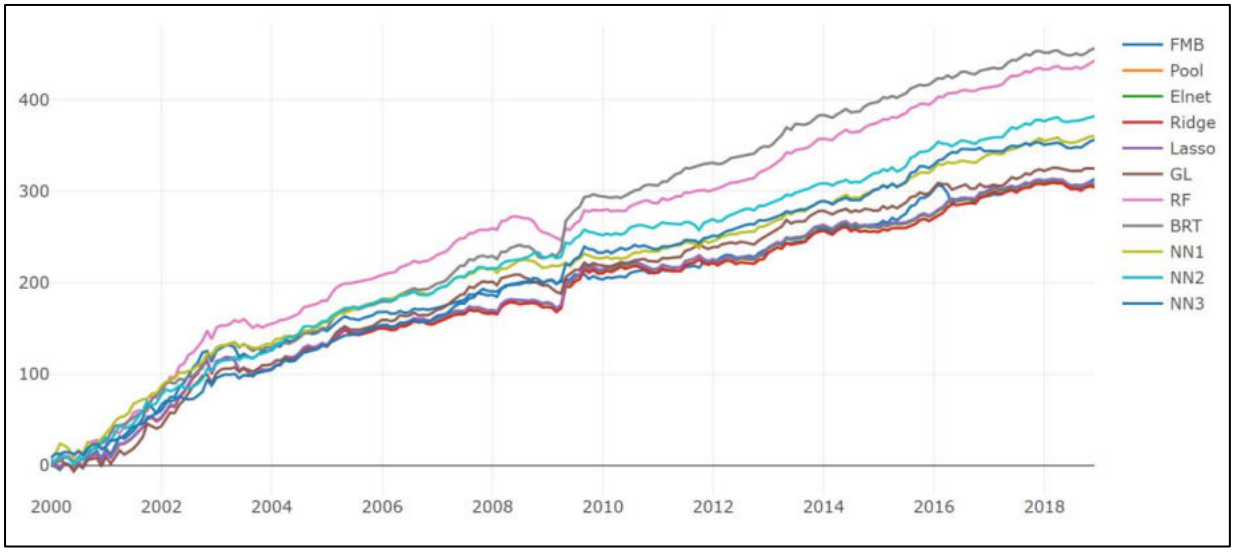



*Figure 7*. Cumulative returns of equally-weighted decile portfolios sorted on out-of-sample return forecasts. All portfolios are equal-weighted.

Performance was not driven by a specific sub-period and did not change significantly over time. Although different methods performed better and worse over different sub-periods, methods that incorporated nonlinear predictor interactions (tree-based methods and neural networks) were relatively stable in their ability to outperform linear models (Figure 7).

Specifically, tree-based methods provided the highest cumulative performance, followed by neural networks. However, as mentioned before, neural networks showed lower drawdowns and standard deviations than tree-based methods, at least when compared to the random forest. Because boosted regression trees produced a similarly low drawdown, the risk-return ratio for this model was somewhat pronounced compared to the other methods.

# 6 Robustness Checks

We conducted extensive robustness checks for all results documented in the previous section. In particular, we examined whether our results were robust against an alternative benchmark, the FTSE Developed Index. We also verified that our results were robust against various tuning parameters. Specifically, we (i) increased the standard deviation of the tuning parameters in the grid and (ii) randomly inserted tuning parameters into the grid.

As described in Section 3.1, our sample included small-, mid-, and large-cap stocks from the S&P Developed BMI (USD) with an average of 4177 stocks per month starting in October 1989 and ending in December 2018. To assess whether our results were overly influenced by small-cap stocks, we repeated our entire process with the FTSE Developed Index. This index is a widely recognized benchmark that includes large and mid-cap companies from globally developed markets. Following the procedure described in Section 3.1, we arrived at an average of 1,375 shares per month from October 1989 to December 2018. A summary statistic for monthly returns and the predictors is provided in Table H.1. Although overall predictive performance were somewhat weaker, models that allowed for nonlinear predictor interactions continued to outperform linear specifications. Table H.2 reports Fama-Macbeth regression results for out-of-sample cross-sectional return forecasts, and Table H.3 reports performance of equal-weighted decile portfolios sorted on out-of-sample return forecasts.[43] Finally, Figure H.1 shows the cumulative performance for the spread portfolios of all strategies.

We also found that our results are robust to different sets of tuning parameters. Methods allowing for nonlinear predictor interactions generally outperform linear settings – is not affected by changes in the grid. For the sake of brevity, we did not include this analysis in the study.

Finally, results for value-weighted portfolios are fairly similar to the results for equal-weighted portfolios and are available upon request.

[43] All other analyses that we provided for the S&P Developed BMI (USD) are also available on request for the FTSE Developed Index.

# 7 Conclusion

Here, we examined machine learning tools for their utility in effectively synthesizing and dissecting high-dimensional datasets into a return prediction model that yields more precise predictions compared to more traditional approaches. As a benchmark, we calculated monthly return forecasts based on traditional cross-sectional regressions in the spirit of Fama & MacBeth (1973). We also computed pooled time-series cross-sectional OLS regressions as a variation of this benchmark. We then compared this benchmark with a representative set of potentially well-suited methods from the machine learning repertoire, including penalized linear models, generalized linear models, boosted regression trees, random forests, and neural networks. Conventional and machine learning approaches shared the same set of 21 firm characteristics, therefore the differences in outcomes were directly related to how this data was modeled.

Using the empirical setting of forecasting stock returns in the cross-section, we find that nonlinear estimators can lead to a significantly improved predictive power compared to traditional OLS regressions. In accordance with Gu, Kelly, & Xiu (2018), we attribute this enhancement to their ability to: (1) model complex, nonlinear patterns that are difficult, if not impossible, to discern using traditional statistical techniques; and (2) their robustness to multi-colinear among predictors. We identify trees and neural networks as our most powerful methods and attribute their predictive gains to the allowance of nonlinear predictor interactions that are overlooked by other methods. However, penalized linear models, which reduce the complexity of the model through parameter shrinkage and variable selection, fail to improve the linear setting. This indicates that over-fitting noise, rather than extracting signal is not a major concern for a linear model with only 21 features. Moreover, the generalized linear model, which introduces nonlinearity via spline functions of each predictor, is also unable to substantially surpass linear specifications. This suggests that the inclusion of potentially complex interactions among predictors plays a crucial role in nonlinearities.

The benefits of machine learning very much justify its growing application in financial technologies, and for this reason it is an exciting time to deploy a disruptive technology that will change the way investment decisions are made on a broad scale. However, successful machine learning applications require considerable domain knowledge,[44] thus they are unlikely, and should not, to replace human experts in the short-term.

This research points to a number of important future directions that include:

- With machine learning algorithms, it is often best to forecast discrete variables to limit the influence of outliers. Accordingly, the target function could be adjusted. Rather than using a regression model to predict returns, one could use a classification model to forecast categories, which may be a more appropriate prediction objective as categories are typically less noisy.[45]
- Rather than giving equal weight to each $x_{i,t}$ (i.e., assuming 10-year-old information is as important as the latest information), or imposing random weighting schemes such as exponential weighting, one could consider models that are able to inherently learn such dependencies in the data.[46]
- Viewing our analytical problem through the lens of value-investing is an alternative method to reduce noise. More formally, a cross-sectional regression of the share price on multiple factors could be performed, following

$$Share\ Price_i = \underbrace{\alpha + \sum \beta_i * Factor_{i,j}}_{Fair\ Share\ Price_i} + Premium \tag{6.1}$$

  The premium itself represents market mispricing, which allows cheap and rich equities to be identified. The ranking of equities is then established with respect to the premium, (e.g., the top 10% in terms of highest positive premiums are selected each month). Rebalancing could follow a methodology that minimizes transaction costs, for example by incorporating minimum holding periods for equities, except where there is a short signal.

[44] In the context of stock selection, this may include questions such as: What are we attempting to predict? How should we standardize factors and returns? What factors are likely to provide valuable information? What training/validation/test windows are likely to be most effective? What algorithms are expected to perform most efficiently?

[45] More precisely, this could translate to sorting the return in quintile buckets and predicting bucket probability by minimizing cross-entropy.

[46] An LSTM recursive neural network is an example of such a model that can inherently discover these dependencies in the data.

# Glossary

**Activation Function**
In artificial neural networks, the activation function of a node defines the output of that node given an input signal. Common activation functions for neural networks are: logistic regression hypothesis (Sigmoid), hyperbolic tangent (tanh), and rectified linear unit (ReLU).

**Back-Propagation**
Backpropagation is an algorithm for supervised learning of artificial neural networks using gradient descent. Given an artificial neural network and an error function, the algorithm computes the gradient of the error function with respect to the neural network's weights. Common (regression) error functions for neural networks are: Mean Squared Error Loss (MSE), Mean Squared Logarithmic Error Loss (MSLE), and Mean Absolute Error Loss (MAE).

**Bagging / Bootstrap Aggregation**
Bootstrap aggregation (or bagging for short), is a simple yet powerful ensemble method. An ensemble method is a technique that combines the predictions of multiple machine learning algorithms to make more accurate predictions than any individual model.

**Bias**
Bias is a type of error that occurs due to erroneous assumptions in the learning algorithm.

**Bias Unit**
A bias unit is similar to the intercept in a linear regression. It is an additional parameter in the neural network that is used to adjust the output. In other words, the bias unit is a constant which helps the model fit best to the given data.

**Bias-Variance Trade-Off**
The bias-variance trade-off is the conflict of simultaneously trying to minimize the bias and variance error that prevents supervised learning algorithms from generalizing to unseen or out-of-sample data.

**Boosting**
Boosting is an ensemble method designed to transform weak learners into strong learners. The idea of boosting is to train weak learners sequentially.

**Branches**
Branches are the segments of a decision tree that connects the nodes. Each branch of a decision tree represents a possible decision.

**Cubic Spline**
A cubic spline refers to a function that is continuous, and has continuous first and second derivatives at the knots.

**Decision Tree**
Decision trees are a nonparametric, supervised learning technique for classification and regression. The goal is to construct a model that predicts the target variable by learning simple decision rules derived from the training data.

**Dropout**
Removes certain connections of the neuronal network at each iteration. Thus, the hidden units cannot depend on any particular feature too much.

**Early Stopping**
Stops training of the neural network at the point when performance on a validation dataset starts to degrade.

**Elastic Net Regularization**
Elastic-net regularization includes both Lasso ($l_1$) and Ridge ($l_2$) regularization methods. The idea of Elastic-net regularization is to degrade the in-sample performance of a model in order to improve its stability to unseen or out-of-sample data.

**End-to-End Learning**
End-to-end learning usually refers to omitting any hand-crafted intermediary algorithms. In other words, an end-to-end model directly learns all the features between the inputs (x) and the final outputs (y).

**Expanded**
Refers to the idea of successively expanding the estimation window.

**Feedforward Network**
Feed-forward neural networks are the simplest form of artificial neural networks. Information flows from the inputs (**x)**, through the intermediate computations, and finally to the output (**y)**. There are no feedback connections in which outputs of the model are fed back into itself.

**Forward Pass**
The forward pass in a neural network refers to the calculation process of the final outputs (y) based on the inputs (x). The process traverses all neurons from the first to the last layer.

**Forward-Propagation**
Forward-propagation computes the value of the output (y) based on the input (x). Input data is "forward propagated" through the network layer by layer to the final layer.

**Group Lasso**
The group lasso is an extension of the lasso to perform variable selection on (predefined) groups of variables in linear regression models.

**Hidden Layer**
A hidden (middle) layer in a neural network is a layer between input (x) and output (y) layers, where neurons receive a set of weighted inputs and generate an output by applying an activation function.

**Hidden Units**
Hidden (middle) units refer to the individual neuros or units of a hidden (middle) layer.

**Hyper-Parameter / Hyper-Parameter Tuning**
Hyper parameter optimization or hyper parameter tuning refers to the problem of selecting a set of optimal hyper parameters for a learning algorithm.

**Input Layer**
The input layer of a neural network consists of input neurons and feeds the initial data into the network for further processing by subsequent layers of neurons.

**Internal Nodes**
In a decision tree, internal nodes are the points along the tree where the predictor space is split.

**Knots**
Knots refer to the places where the pieces of a spline intersect.

**Lasso Regularization**
Lasso (least absolute shrinkage and selection operator) is a regression analysis method that performs variable selection in order to enhance the generalization capabilities of the model to unseen out-of-sample data.

**Leaves**
The leaves (terminal nodes) are the decisions or the final results of a decision tree.

**L1/L2 Regularization.**
Curbs high weight values so that the model is not dependent on a single feature.

**Mini-Batch Gradient Descent**
Mini-batch gradient descent computes the gradient of the cost function with regards to the weights for every mini-batch of *n* training examples.

**Natural Cubic Spline**
A natural cubic spline refers to a function that is continuous, has continuous first and second derivatives and is linear beyond the boundary knots.

**Neural Network**
An artificial neural network is a machine learning technique inspired by the way biological neural networks process information in the human brain.

**Neuron**
In a neural network, a neuron takes inputs, applies an activation function, and produces outputs.

**Optimum Model Complexity**
The optimum model complexity is the level of complexity at which the increase in bias is equivalent to the reduction in variance.

**Output Layer**
The output layer in an artificial neural network is the last layer of neurons that provides the final result.

**Random Forest**
A random forest consists of a large number of individual decision trees that operate as an ensemble
.

**Recursive Binary Splitting**
Recursive binary splitting refers to a top-down greedy approach that successively splits the predictor space into two new branches down a decision tree. The algorithm considers only the current split, and not any future splits that may lead to a better tree.

**Ridge Regularization**
Ridge is a regression analysis method that performs variable shrinkage in order to enhance the generalization capabilities of the model to unseen out-of-sample data.

**Rolled**
Refers to the idea of rolling forward the estimation window.

**Selection**
Refers to the LASSO algorithm which perform variable selection.

**Shrinkage**
Refers to the Ridge algorithm which perform variable shrinkage.

**Symmetry Breaking**
A neural network does not perform symmetry-breaking. In other words, if all weights are set to zero, then all neurons of all layers perform the same calculation, producing the same output, and thus making the neural network useless. Random initialization serves as a process of symmetry-breaking. More specifically, the weights are initialized very close to zero, but randomly. This helps in breaking symmetry and every neuron is no longer performing the same computation.

**Terminal Nodes**
The terminal nodes (leaves) are the decisions or the final results of a decision tree.

**Test Set**
A test set refers to a separate dataset which is neither used for estimation nor tuning to allow for an unbiased assessment of the final models.

**Total Error**
The total error consists of three components: $\text{bias}^2$, variance, and noise.

**Training Set**
A training set is a subset of the dataset used to build the models.

**Truncated Power Basis**
Truncated power basis functions can be used to construct splines.

**Validation Set**
A validation set is a subset of the dataset used to assess the performance of model built in the training phase.

**Variance**
Variance is a type of error that occurs due to a model's sensitivity to small fluctuations in the training set.

# References


Banz, R. W. (1981). The relationship between return and market value of common stocks. *Journal of Financial Economics, 9*(1), 3-18.

Basu, S. (1983). The relationship between earnings' yield, market value and return for NYSE common stocks: Further evidence. *Journal of Financial Economics, 12*(1), 129-156.

Bhandari, L. C. (1988). Debt/Equity Ratio and Expected Common Stock Returns: Empirical Evidence. *The Journal of Finance, 43*(2), 507-528.

Blitz, D., Huij, J., & Martens, M. (2011). Residual Momentum. *Journal of Empirical Finance, 18*(3), 506-521.

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5-32.

Byström, H. (2006). Merton unraveled: A flexible way of modeling default risk. *The Journal of Alternative Investments, 8*(4), 39-47.

Chinco, A., Clark-Joseph, A. D., & Ye, M. (2019). Sparse Signals in the Cross-Section of Returns. *The Journal of Finance, 74*(1), 449-492.

Cochrane, J. H. (2011). Presidential Address: Discount Rates. *The Journal of Finance, 66*(4), 1047-1108.

Fama, E. F., & French, K. R. (1992). The Cross-Section of Expected Stock Returns. *The Journal of Finance, 47*(2), 427-465.

Fama, E. F., & French, K. R. (1993). Common risk factors in the returns on stocks and bonds. *Journal of Financial Economics, 33*(1), 3-56.

Fama, E. F., & MacBeth, J. D. (1973). Risk, Return, and Equilibrium: Empirical Tests. *Journal of Political Economy, 81*(3), 607-636.

Fama, E., & French, K. (2015). A five-factor asset pricing mode. *Journal of Financial Economics, 116*(1), 1-22.

Feng, G., Polson, N. G., & Xu, J. (2018). Deep Learning Factor Alpha. *SSRN Electronic Journal*. Retrieved from https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3243683

Fortmann-Roe, S. (2012). *Understanding the bias-variance tradeoff.* Retrieved from http://scott.fortmann-roe.com/docs/BiasVariance.html

Freund, Y. (1995). Boosting a weak learning algorithm by majority. *Information and Computation, 121*(2), 256-285.

Freyberger, J., Neuhierl, A., & Weber, M. (2018). Dissecting characteristics nonparametrically. *SSRN Electronic Journal*. Retrieved from https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2820700

Friedman, J. (2001). Greedy function approximation: a gradient boosting machine. *Annals of statistics, 29*(5), 1189-1232.

Friedman, J., Hastie, T., & Tibshirani, R. (2000). Additive logistic regression: A statistical. *The Annals of Statistics, 28*(2), 337-374.

Giglio, S., & Xiu, D. (2017). Inference on Risk Premia in the Presence of Omitted Factors. *SSRN Electronic Journal*. Retrieved from https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2988748

Green, J., Hand, J. R., & Zhang, X. F. (2017). The Characteristics that Provide Independent Information about Average U.S. Monthly Stock Returns. *The Review of Financial Studies, 30*(12), 4389-4436.

Gu, S., Kelly, B. T., & Xiu, D. (2018). Empirical Asset Pricing via Machine Learning. *SSRN Electronic Journal*. Retrieved from https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3159577

Han, Y., He, A., Rapach, D., & Zhou, G. (2018). What Firm Characteristics Drive US Stock Returns? *SSRN Electronic Journal*. Retrieved from https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3185335

Hanna, J. D., & Ready, M. (2005). Profitable predictability in the cross section of stock returns. *Journal of Financial Economics, 78*(3), 463-505.
Harvey, C. R., & Liu, Y. (2018). Lucky Factors. *SSRN Electronic Journal*. Retrieved from https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2528780
Harvey, C. R., Liu, Y., & Zhu, H. (2016). … and the Cross-Section of Expected Returns. *The Review of Financial Studies, 29*(1), 5-68.
Hastie, T., & Zou, H. (2005). Regularization and Variable Selection via the Elastic Net. *Journal of the Royal Statistical Society, 67*(2), 301-320.
Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction.* New York: Springer.
Haugen, R. A., & Baker, N. L. (1996). Commonality in the determinants of expected stock returns. *Journal of Financial Economics, 41*(3), 401-439.
Hoerl, A. E., & Kennard, R. W. (2000). Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics, 42*(1), 80-86.
Hou, K., Xue, C., & Zhang, L. (2017). Replicating Anomalies. *SSRN Electronic Journal*. Retrieved from https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2961979
Jacobs, H., & Mueller, S. (Forthcoming). Anomalies Across the Globe: Once Public, No Longer Existent? *Journal of Financial Economics*.
James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning.* New York: Springer.
Kelly, B. T., Pruitt, S., & Su, Y. (2017). Some Characteristics Are Risk Exposures, and the Rest Are Irrelevant. *SSRN Electronic Journal*. Retrieved from https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3032013
Kingma, D., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations.* San Diego.
Kozak, S., Nagel, S., & Santosh, S. (2017). Shrinking the Cross Section. *SSRN Electronic Journal*. Retrieved from https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2945663
Lewellen, J. (2015). The Cross-section of Expected Stock Returns. *Critical Finance Review, 4*(1), 1-44.
Lintner, J. (1965). The Valuation of Risk Assets and The Selection of Risky Investments in Stock Portfolios and Capital Budgets. *The Review of Economics and Statistics, 47*(1), 13-37.
Marcos, L. D. (2018). *Advances in Financial Machine Learning.* New Jersey: John Willey & Sons.
McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *The bulletin of mathematical biophysics, 5*(4), 115-133.
McLean, R. D., & Pontiff, J. (2016). Does Academic Research Destroy Stock Return Predictability? *The Journal of Finance, 71*(1), 5-32.
Mincer, J., & Zarnowitz, V. (1969). The Evaluation of Economic Forecasts. *National Bureau of Economic Research*, 3-46.
Moritz, B., & Zimmermann, T. (2016). Tree-Based Conditional Portfolio Sorts: The Relation between Past and Future Stock Returns. *SSRN Electronic Journal*. Retrieved from https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2740751
Mossin, J. (1966). Equilibrium in a Capital Asset Market. *Econometrica, 34*(4), 768-783.
Müller, A. C., & Guido, S. (2016). *Introduction to Machine Learning with Python: A Guide for Data Scientists.* O'Reilly Media.
Newey, W. K., & West, K. D. (1987). A Simple, Positive Semi-Definite, Heteroskedasticity and Autocorrelation Consistent Covariance Matrix. *Econometrica, 55*(3), 703-708.
Rasekhschaffe, K., & Jones, R. (2019). Machine Learning for Stock Selection. *Financial Analysts Journal, 75*(3), 70-88.

Rosenberg, B., Reid, K., & Lanstein, R. (1985). Persuasive evidence of market inefficiency. *The Journal of Portfolio Management, 11*(3), 9-16.
Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review, 65*(6), 386-408.
Samuel, A. L. (1959). Some Studies in Machine Learning Using the Game of Checkers. *IBM Journal of Research and Development, 3*(3), 210-229.
Schapire, R. E. (1990). The strength of weak learnability. *Machine Learning, 5*(2), 197-227.
Sharpe, W. F. (1964). Capital Asset Prices: A Theory of Market Equilibrium under the Conditions of Risk. *The Journal of Finance, 19*(3), 425-442.
Simonian, J., Wu, C., Itano, D., & Narayanam, V. (2019). A Machine Learning Approach to Risk Factors. *The Journal of Financial Data Science, 1*(1), 32-44.
Stattman, D. (1980). Book Values and Stock Returns. *MBA: A Journal of Selected Papers, 4*, 25-45.
Subrahmanyam, A. (2010). The Cross-Section of Expected Stock Returns: What Have We Learnt from the Past Twenty-Five Years of Research? *European Financial Management, 16*(1), 27 - 42.
Tibshirani, R. (1996). Regression Shrinkage and Selection via the Lasso. *Journal of the Royal Statistical Society, 58*(1), 267-288.
Titman, S., & Jegadeesh, N. (1993). Returns to Buying Winners and Selling Losers: Implications for Stock Market Efficiency. *The Journal of Finance, 48*(1), 65-91.

# Appendix

## A Figures

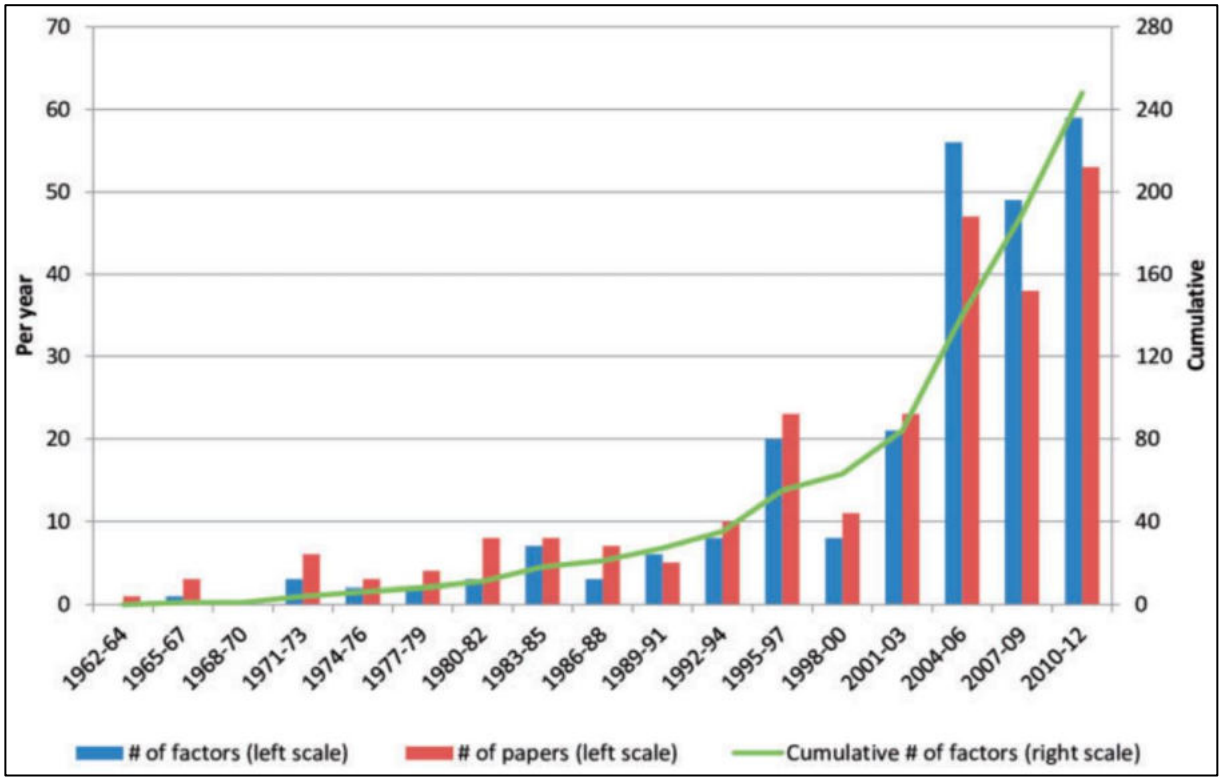



*Figure A.1*. Factors discovered from 1962 to 2010. Source: Figure 2 of Harvey, Liu, and Zhu *(2016)*.

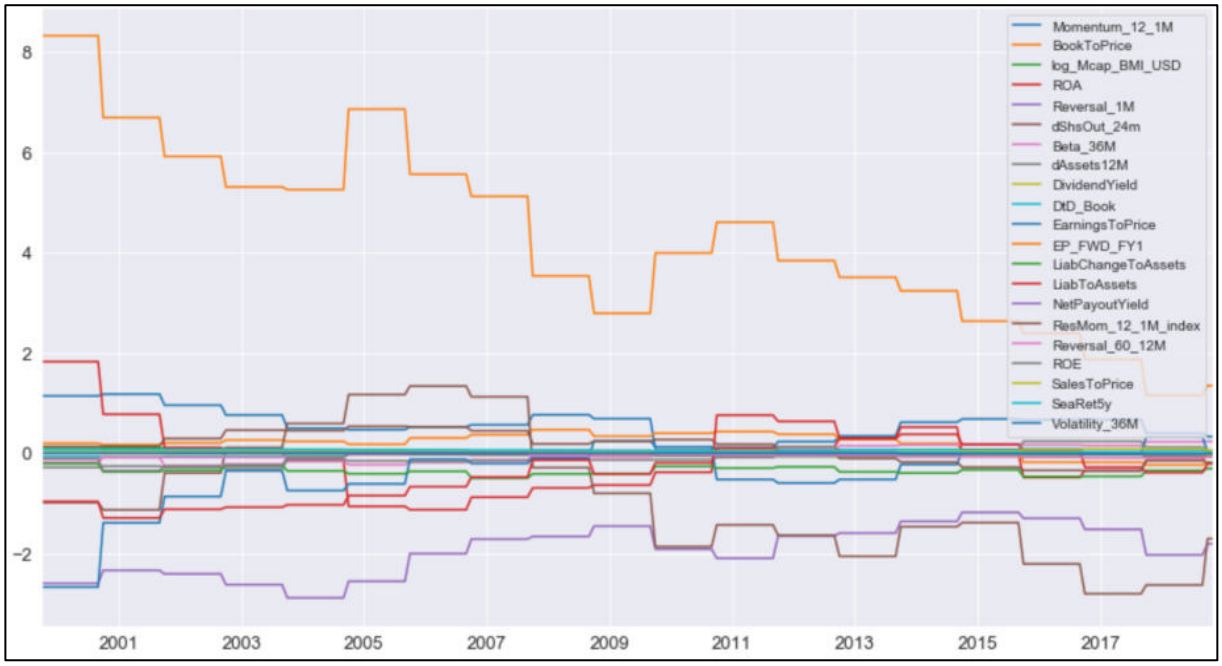



*Figure A.2(i).* 10-year Fama-MacBeth rolling slope estimates, 1999 to 2018.

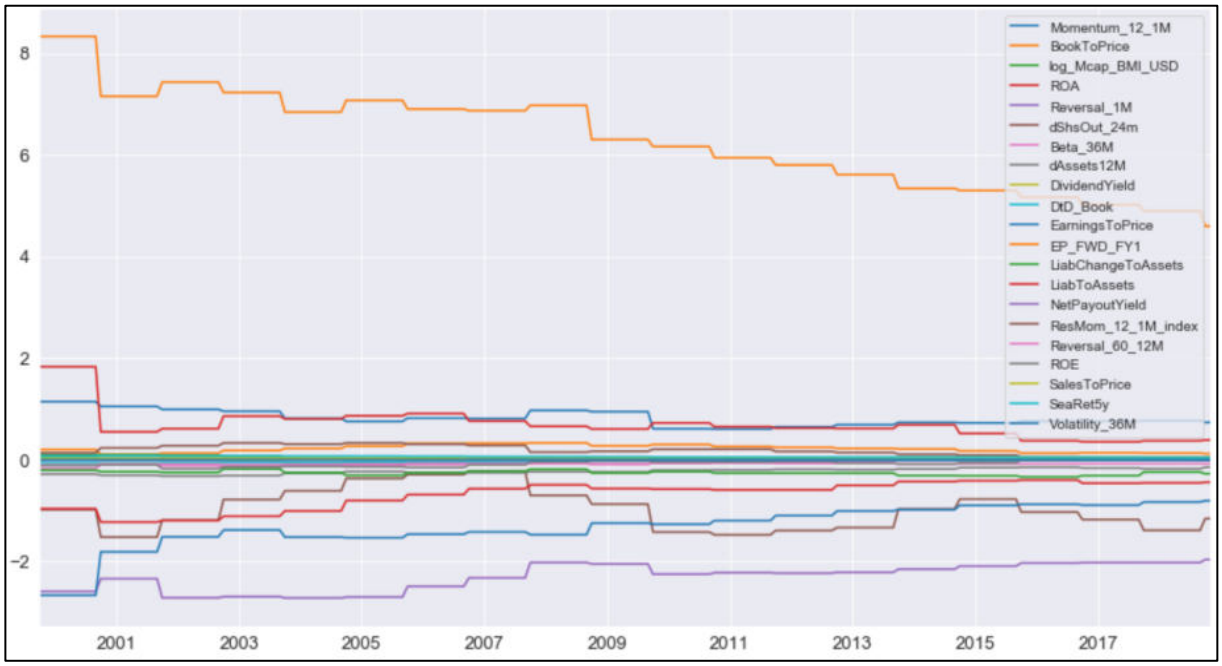



*Figure A.2(ii).* 10-year Fama-Macbeth expanding slope estimates, 1999 to 2018.

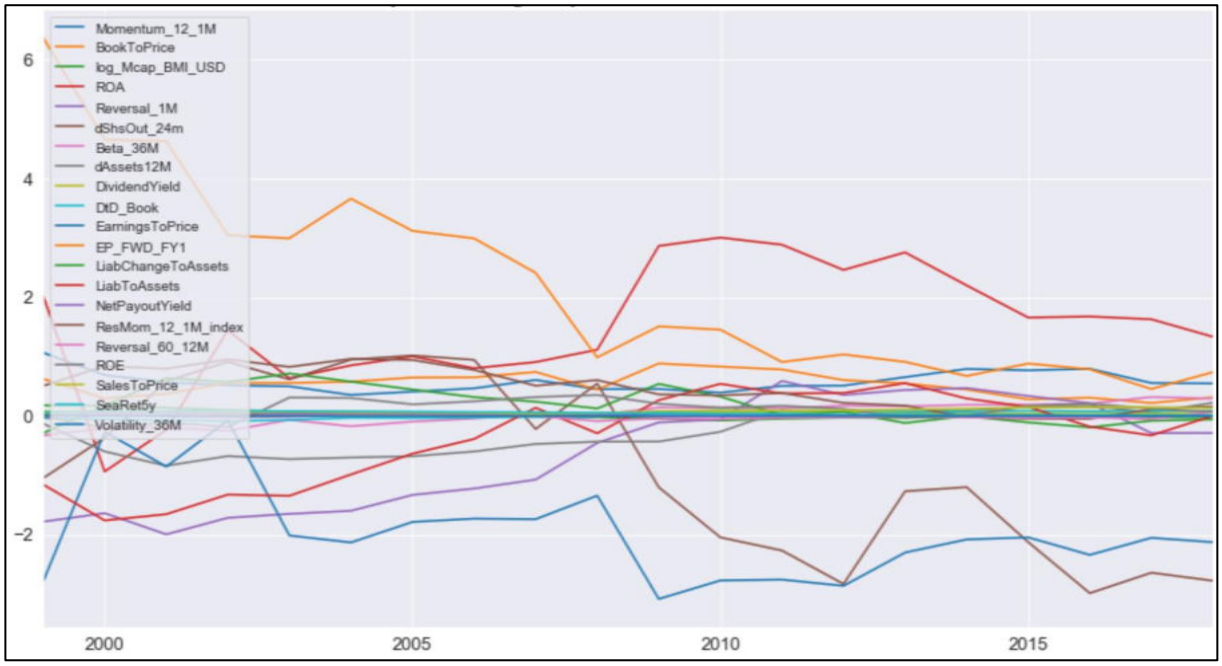



*Figure A.3(i).* 10-year rolling pooled OLS slope estimates, 1999 to 2018.

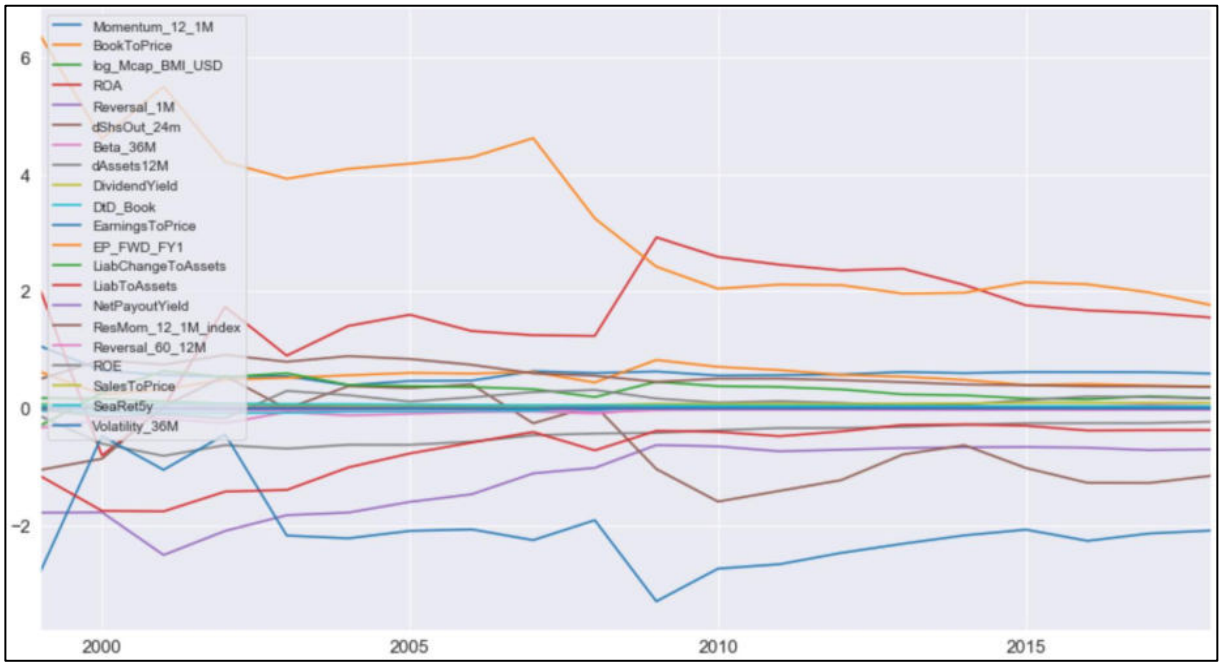



*Figure A.3(ii).* 10-year expanding pooled OLS slope estimates, 1999 to 2018.

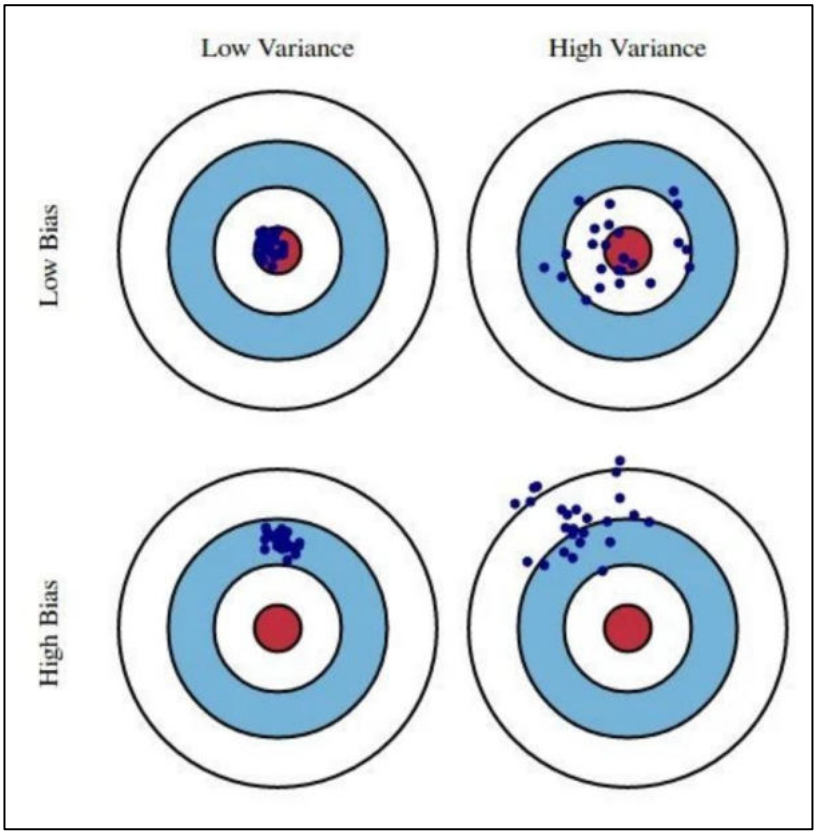



*Figure A.4(i).* Bias-variance trade-off. Source: Fortmann-Roe *(2012)*. The center of the target (bull's-eye) represents the true population parameter that the model is attempting to estimate, θ. The points demonstrate the values of our estimates from four different estimators—low bias and variance, high bias and variance, and their combinations.

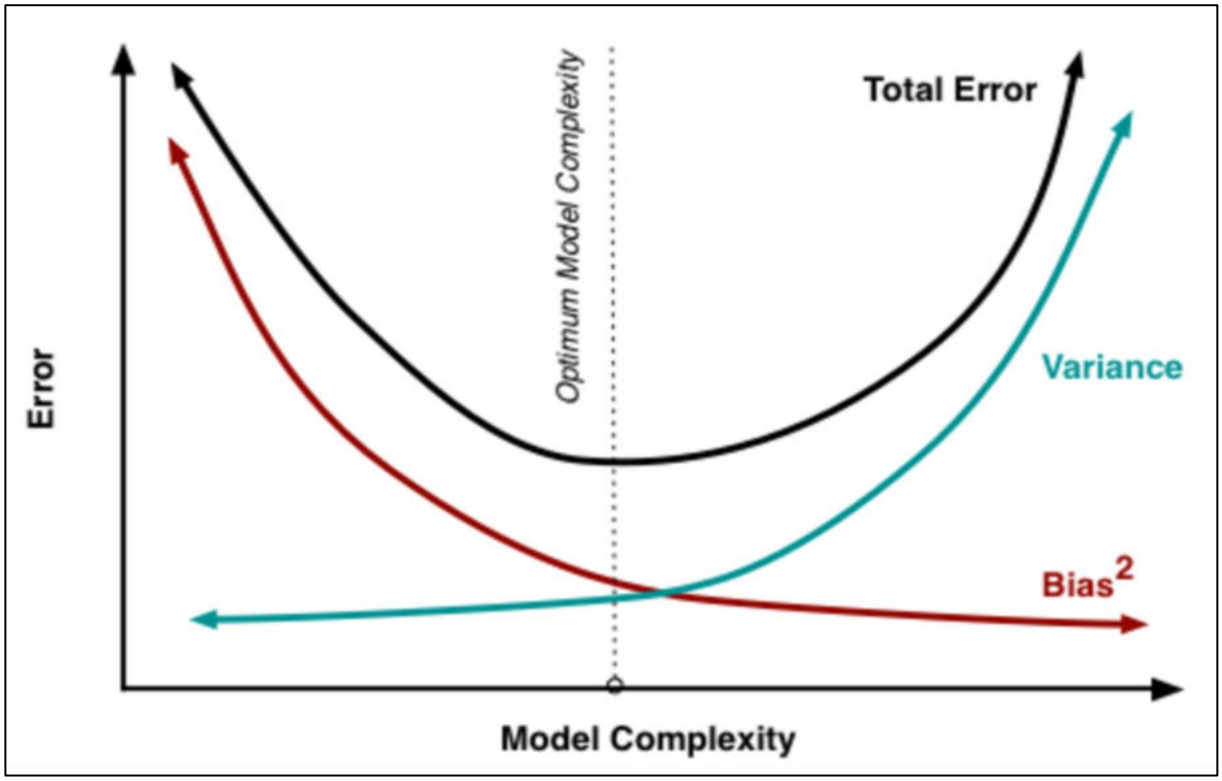



*Figure A.4(ii)*. Bias-variance trade-off (continued). Source: Fortmann-Roe *(2012)*.

**Note:** As the complexity of our model increases, which depends on the number of predictors in the case of linear regression, *variance* becomes the primary concern, whereas *bias* steadily decreases. Understanding bias and variance is crucial to understanding the behavior of prediction models, but in general the *total error* is the primary concern, not the specific decomposition. The *optimum model complexity* for any model is the level of complexity at which the increase in bias is equivalent to the reduction in variance (i.e., where the total error is smallest):

$$\frac{\partial Bias}{\partial Complexity} = -\frac{\partial Variance}{\partial Complexity} \tag{A.1}$$

If our model complexity exceeds this level, we run the risk of over-fitting, whereas we are under-fitting if our model complexity falls below this level.

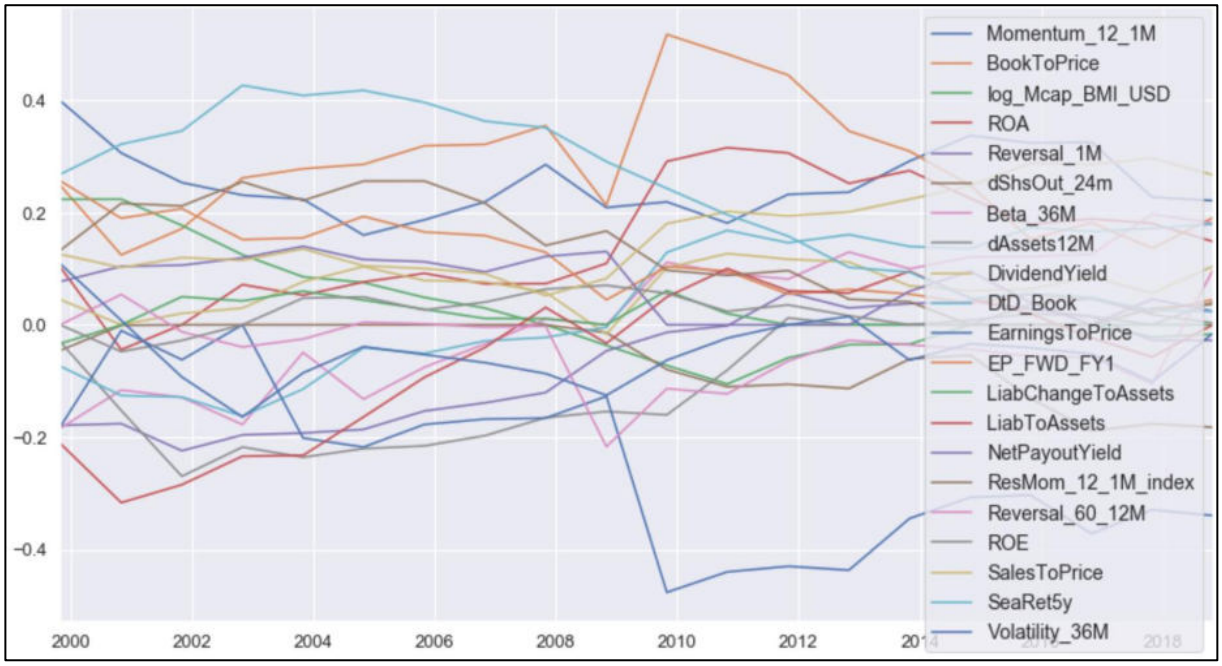



*Figure A.5(i).* 10-year rolling pooled OLS slope estimates incorporating an elastic-net penalty, 1999 to 2018.

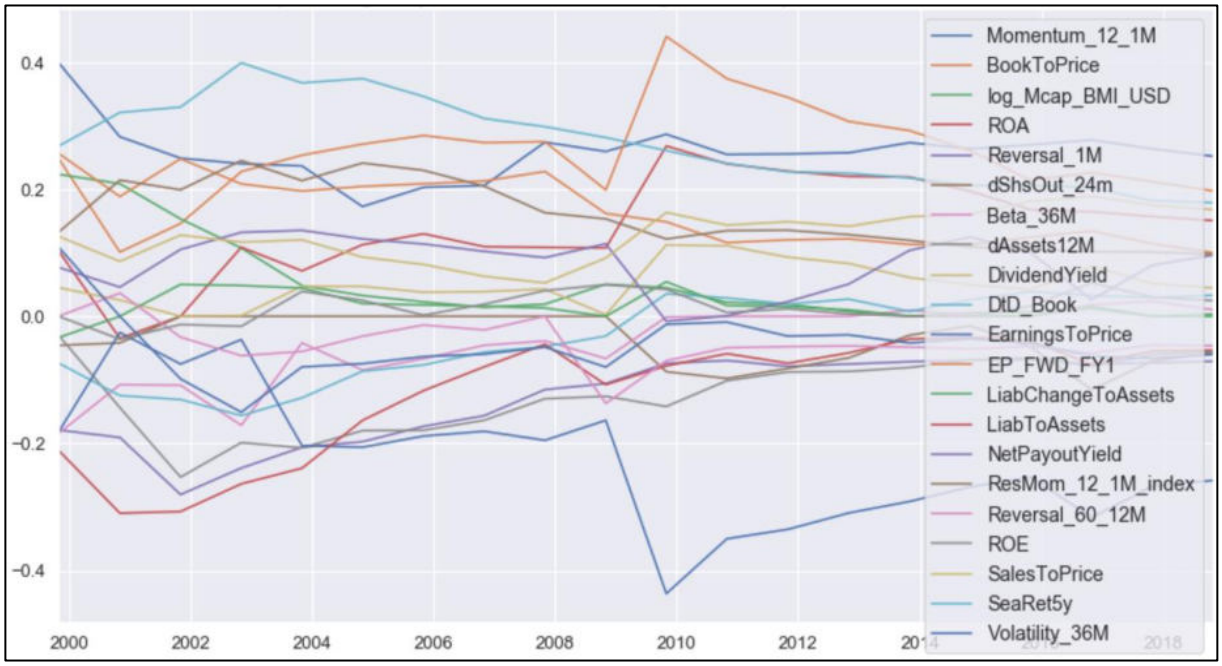



*Figure A.5(ii).* 10-year expanding pooled OLS slope estimates incorporating an elastic-net penalty, 1999 to 2018.

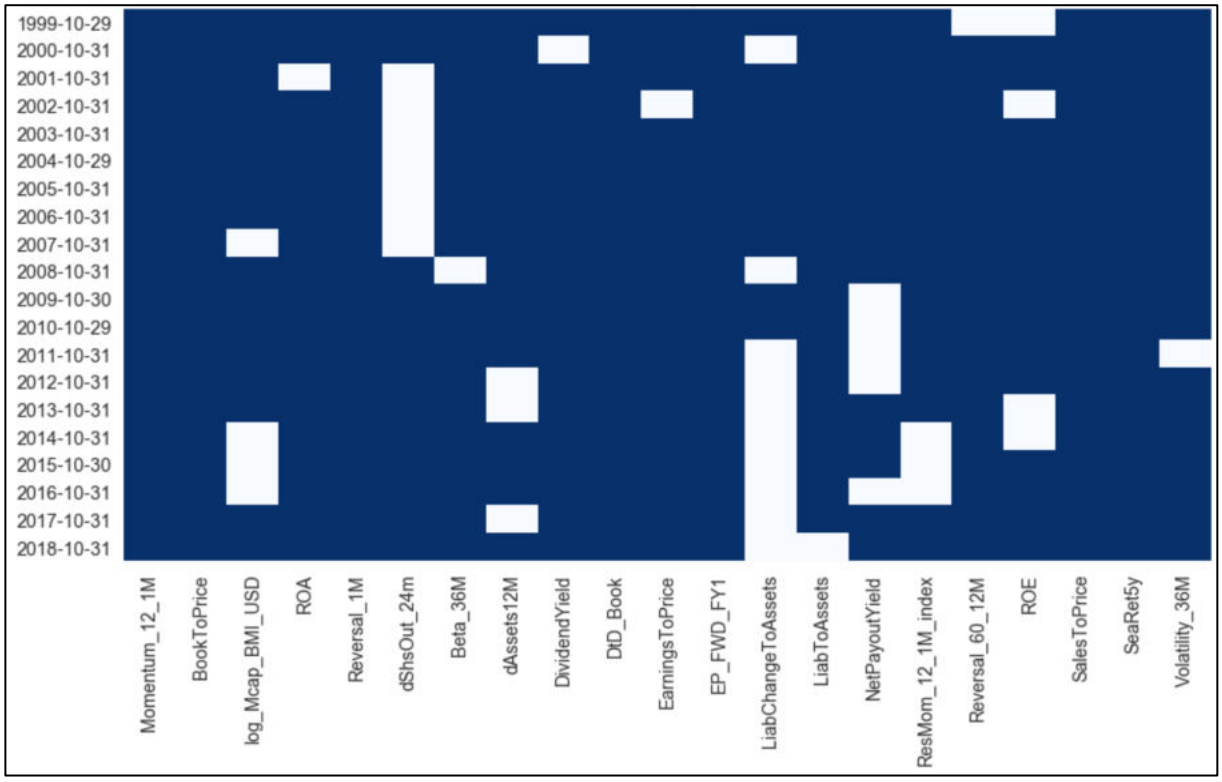



*Figure A.6(i)*. Selected features (dark blue) using elastic-net rolling perspective, 1999 to 2018.

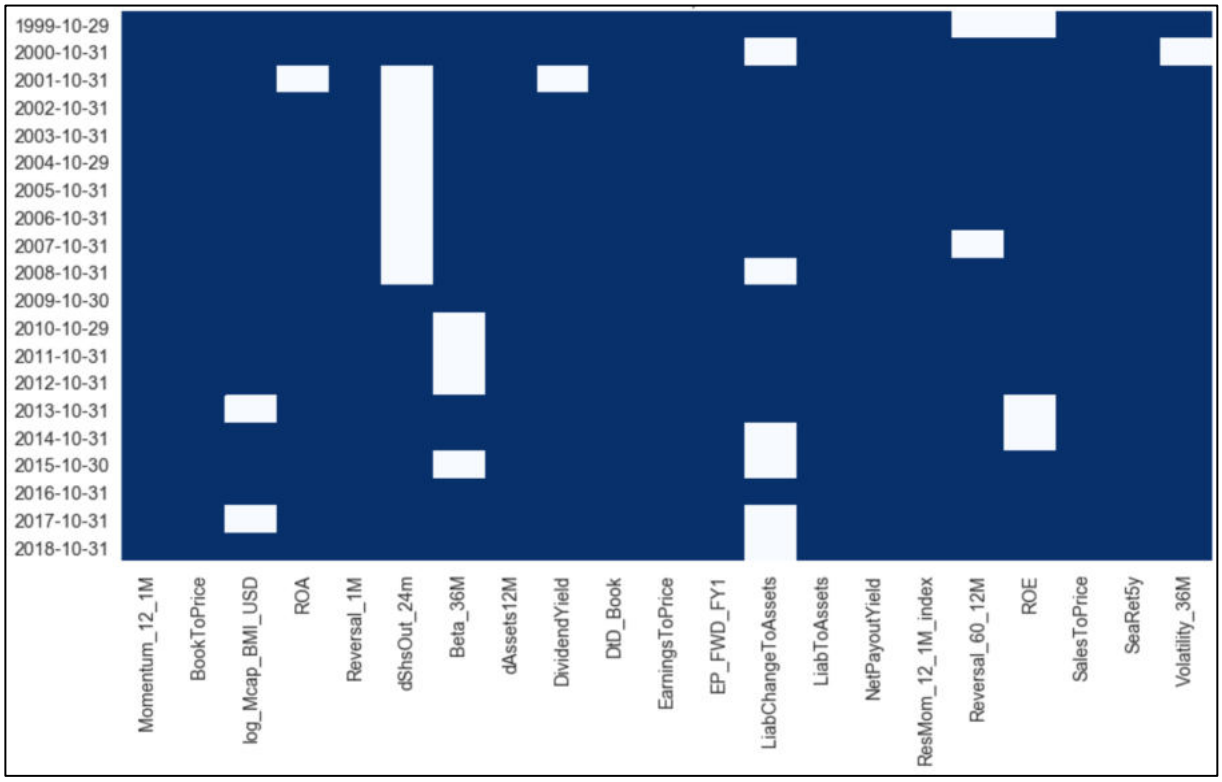



*Figure A.6(ii).* Selected features (dark blue) using elastic-net expanding perspective, 1999 to 2018.

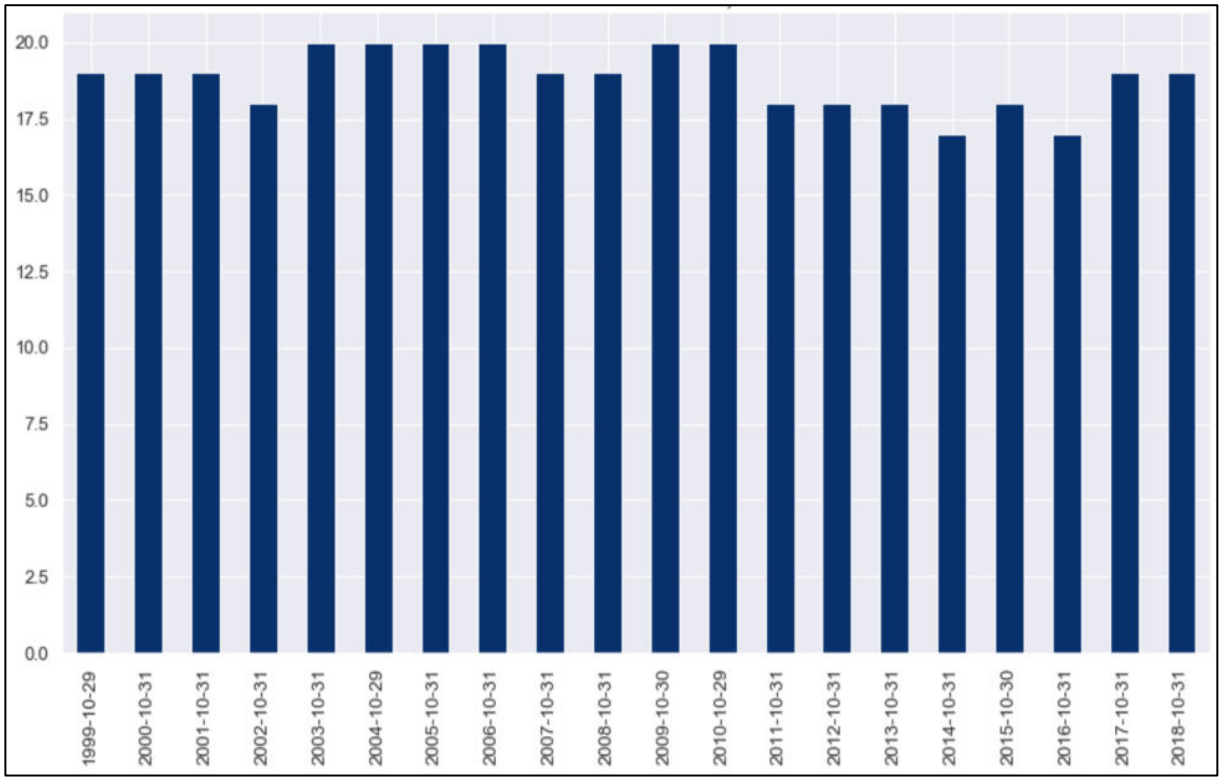



*Figure A.7(i):* Number of elastic-net features (rolling perspective), 1999 to 2018.

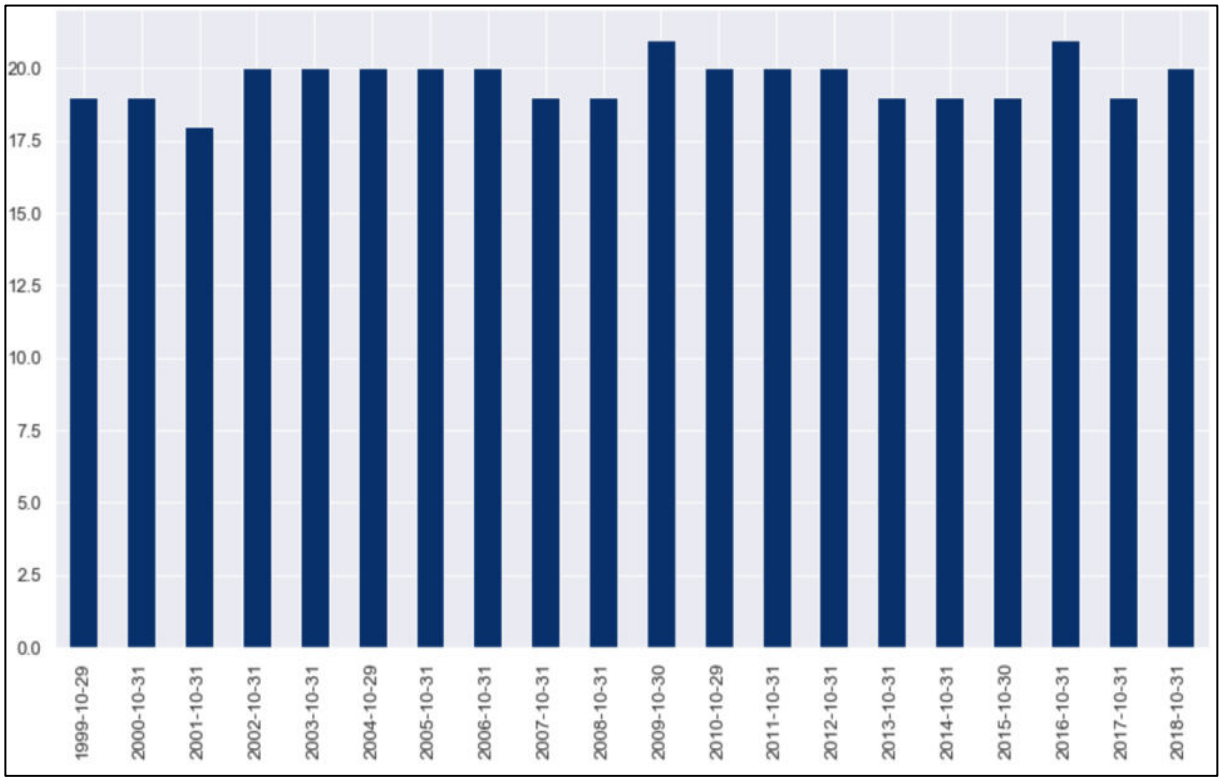



*Figure A.7(ii):* Number of elastic-net features (expanding perspective), 1999 to 2018.




*Figure A.8(i):* Average elastic-net slopes (rolling perspective), 1999 to 2018.

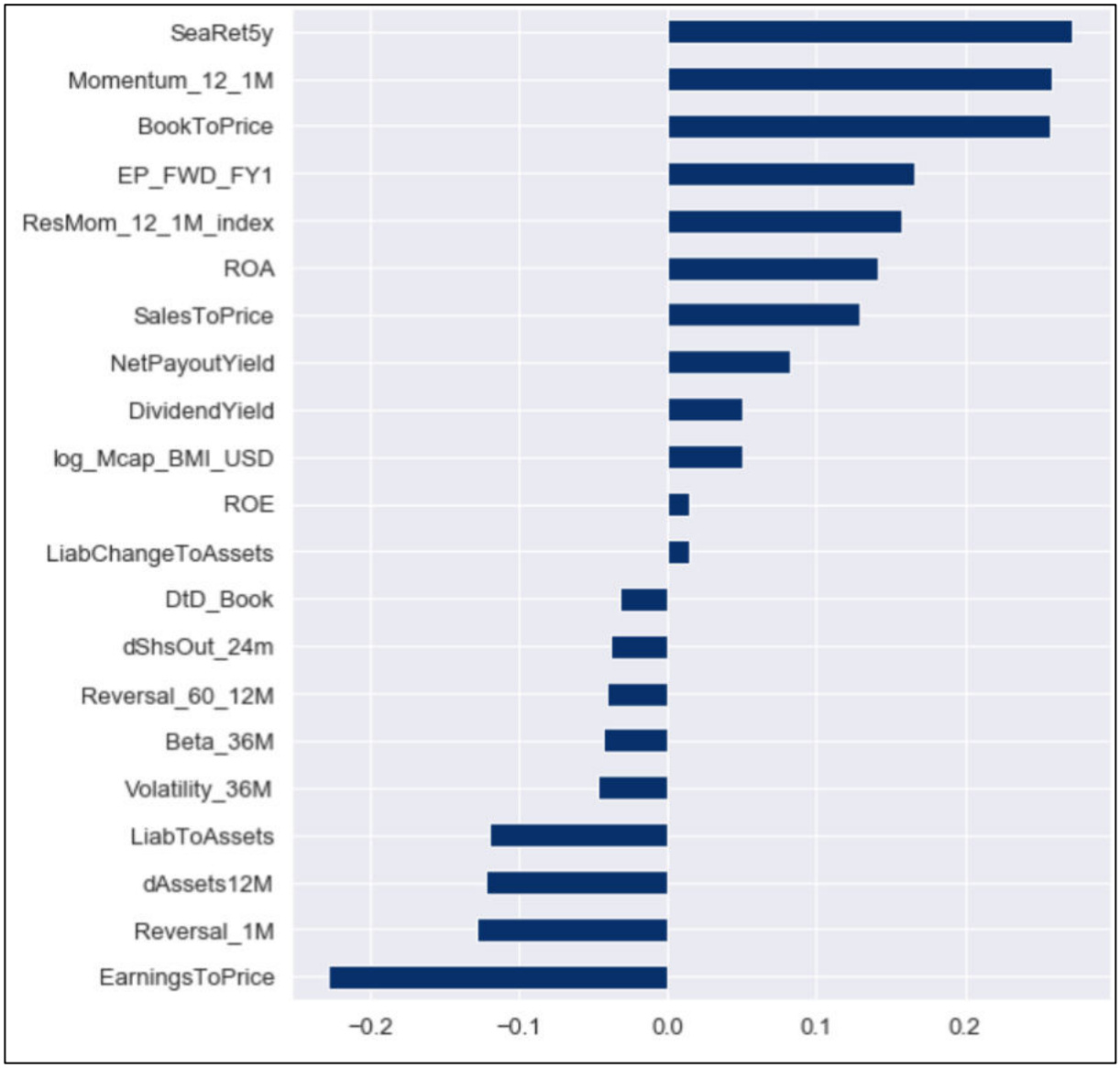



*Figure A.8(ii):* Average elastic-net slopes (expanding perspective), 1999 to 2018.

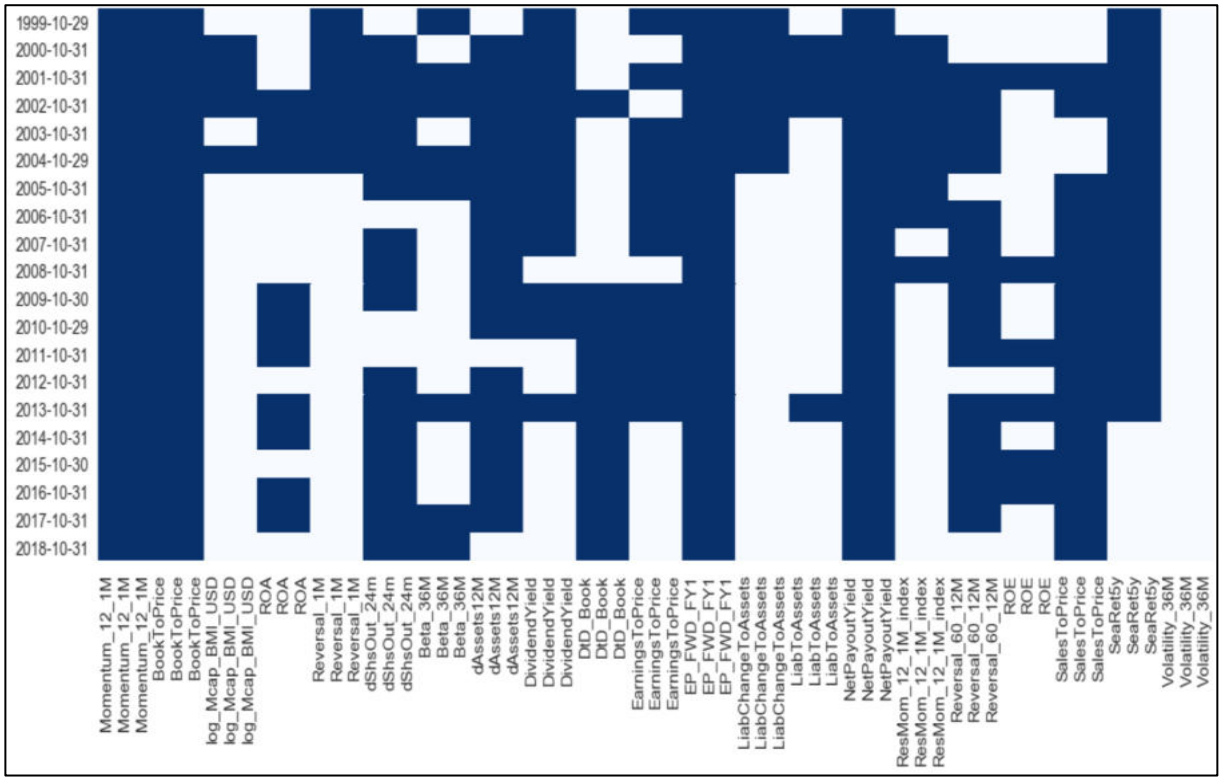



*Figure A.9(i)*. Selected features (dark blue) using group lasso rolling perspective, 1999 to 2018.

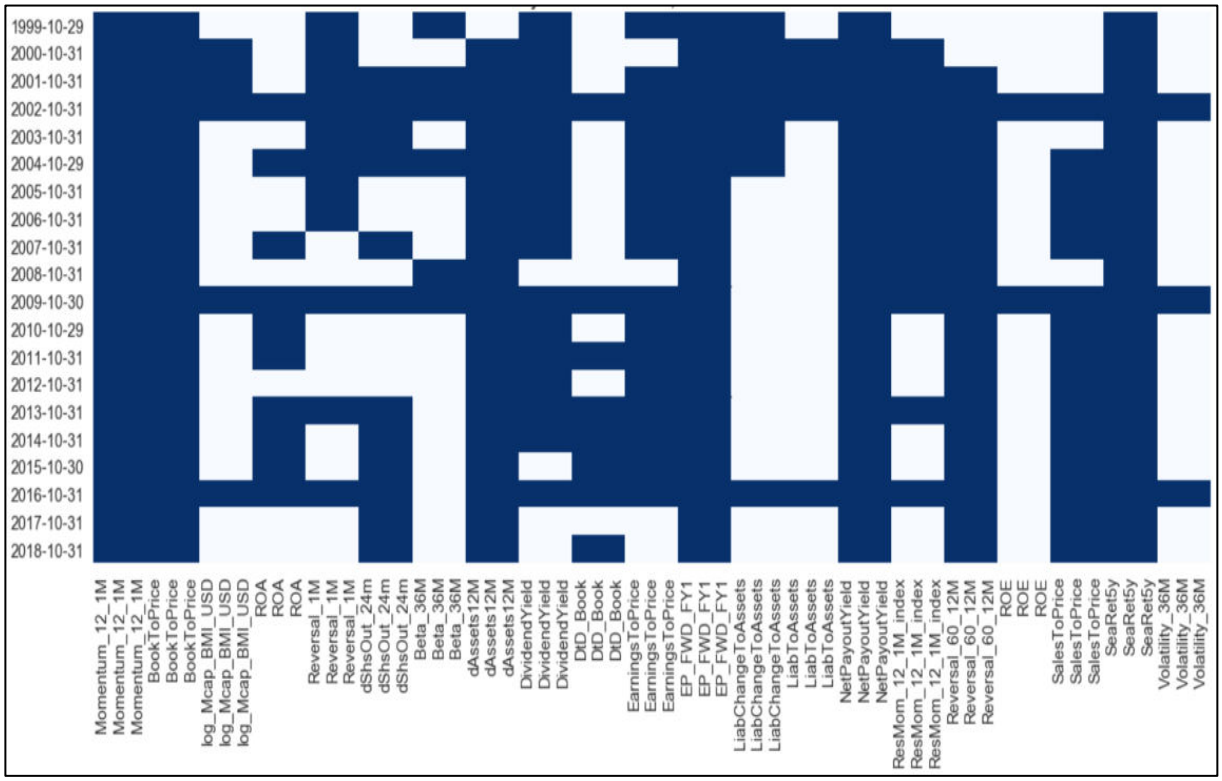



*Figure A.9(ii).* Selected features (dark blue) using group lasso expanding perspective, 1999 to 2018.

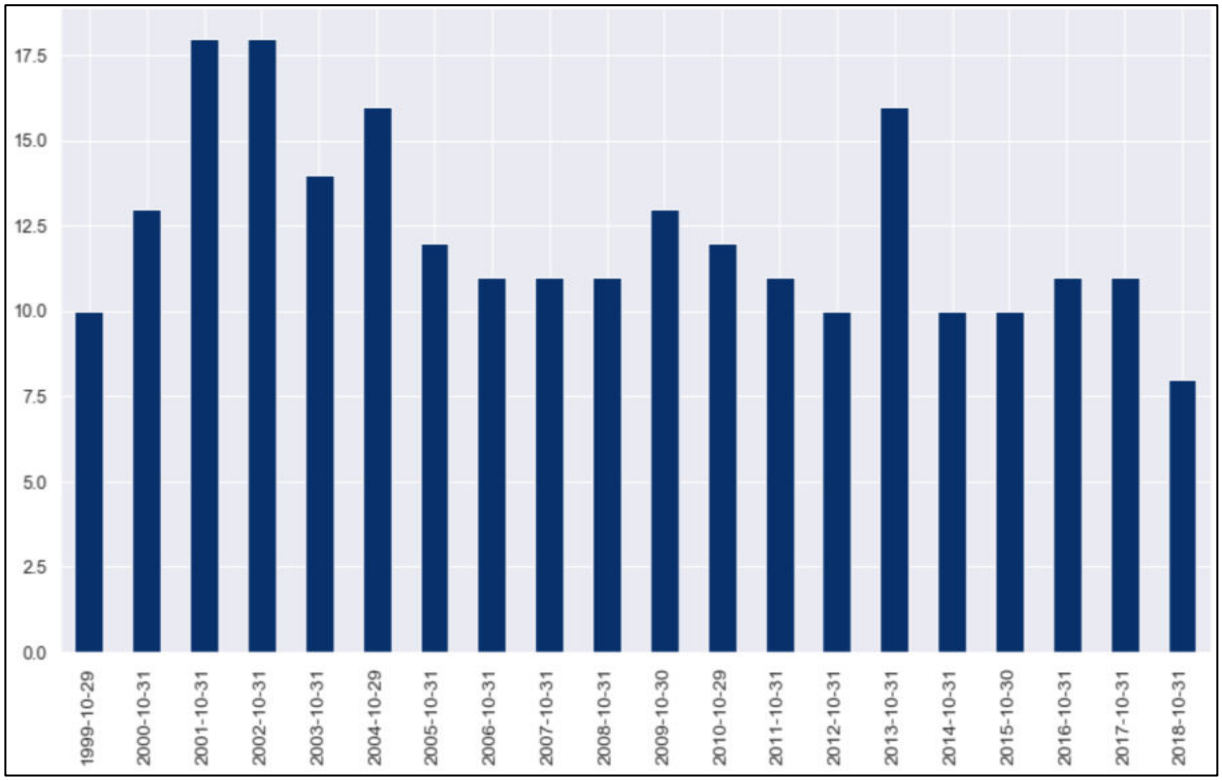



*Figure A.10(i).* Number of group lasso features (rolling perspective), 1999 to 2018.

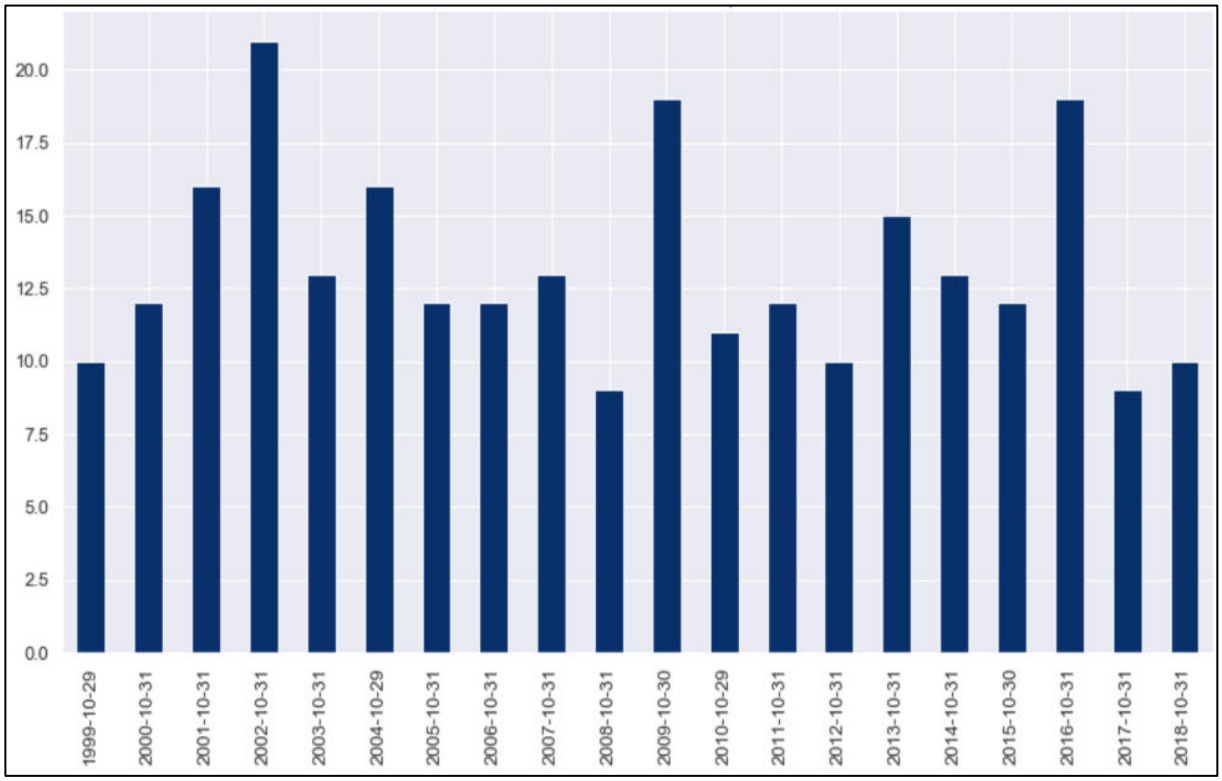



*Figure A.10(ii).* Number of group lasso features (expanding perspective), 1999 to 2018.

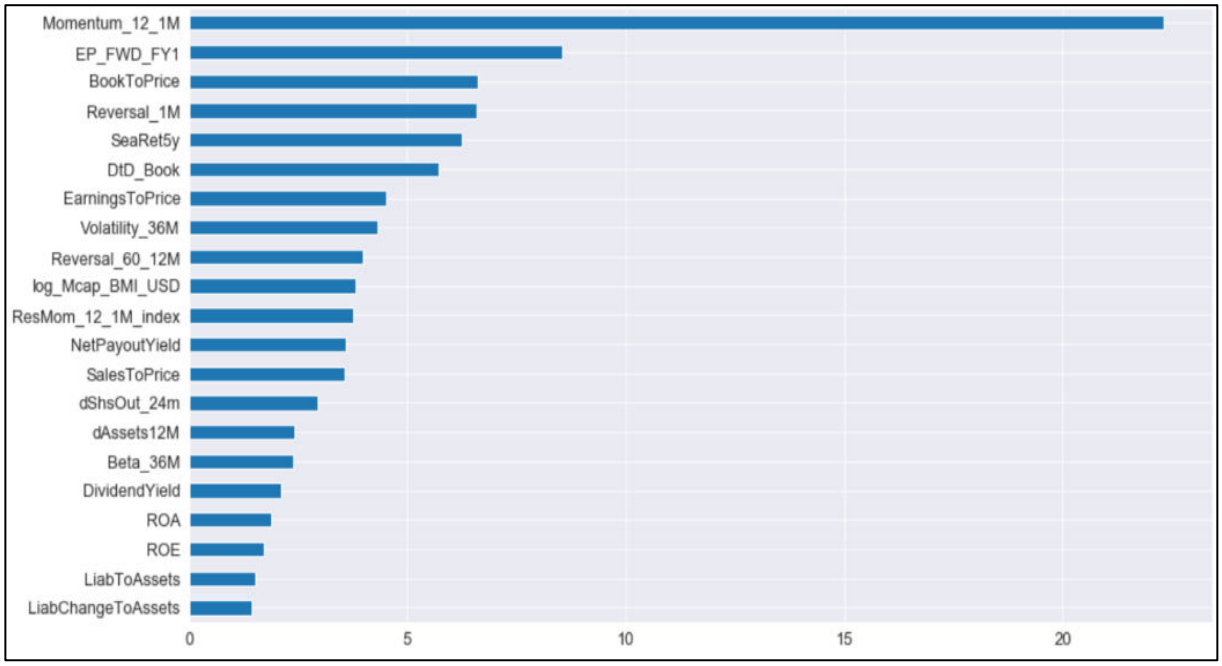



*Figure A.11(i).* Feature importance for random forests (rolling perspective), 1999 to 2018.

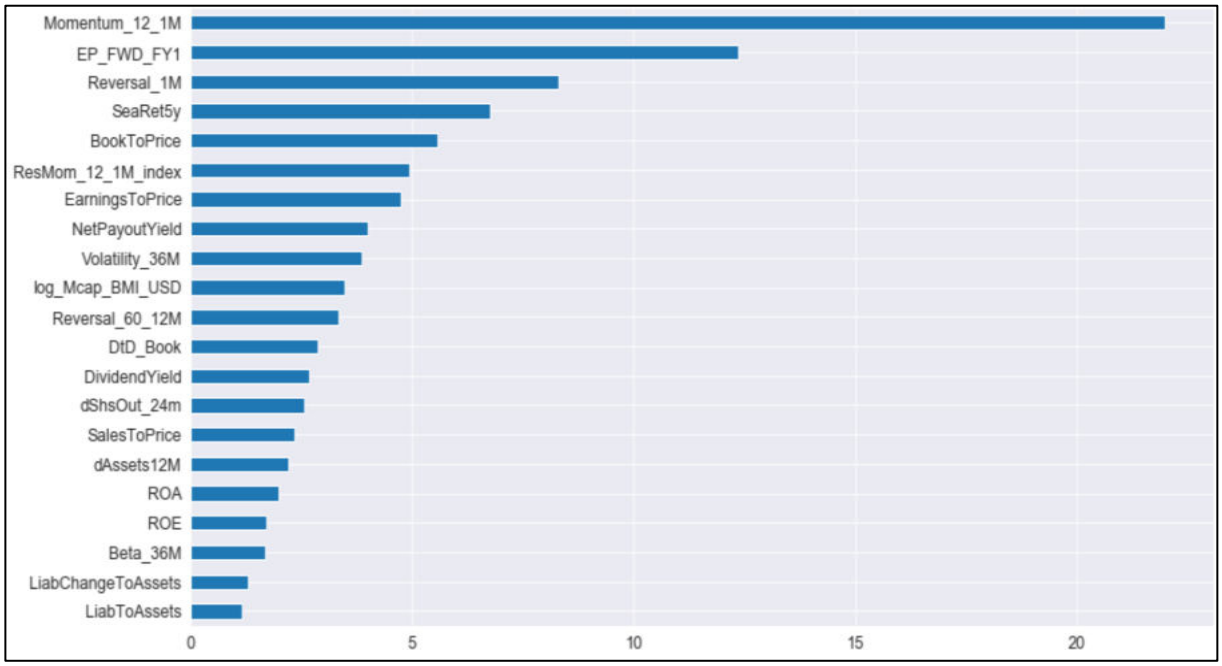



*Figure A.11(ii).* Feature importance for random forests (expanding perspective), 1999 to 2018.

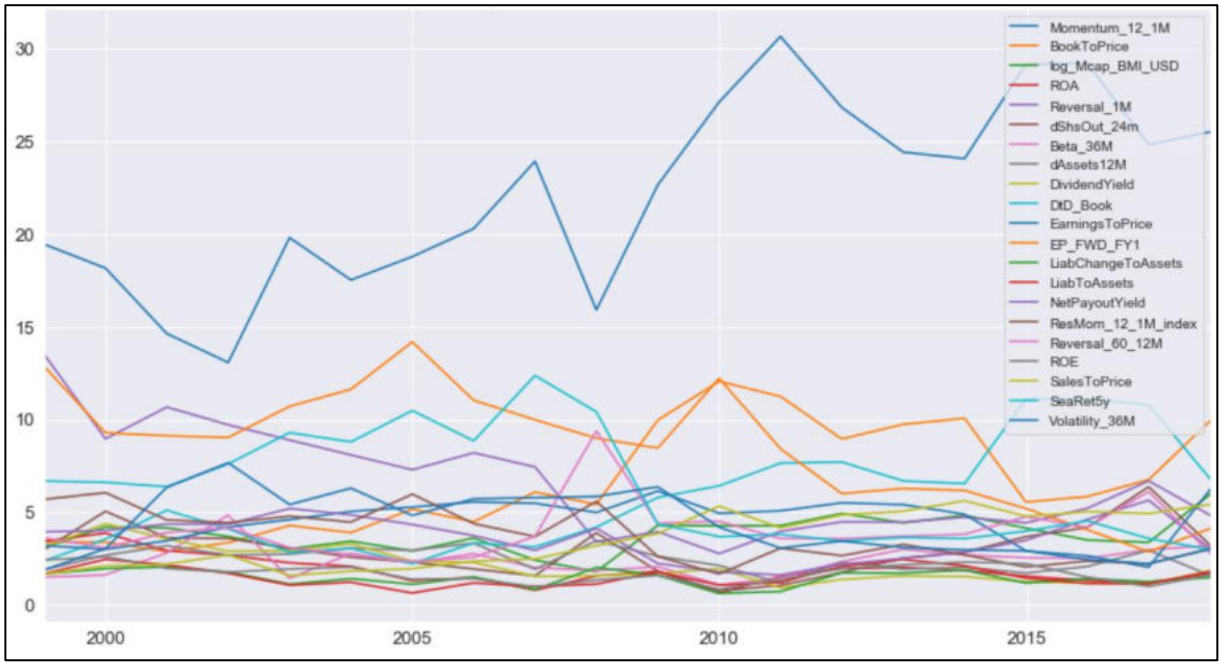



*Figure A.12(i).* Feature importance over time for random forests (rolling perspective), 1999 to 2018.

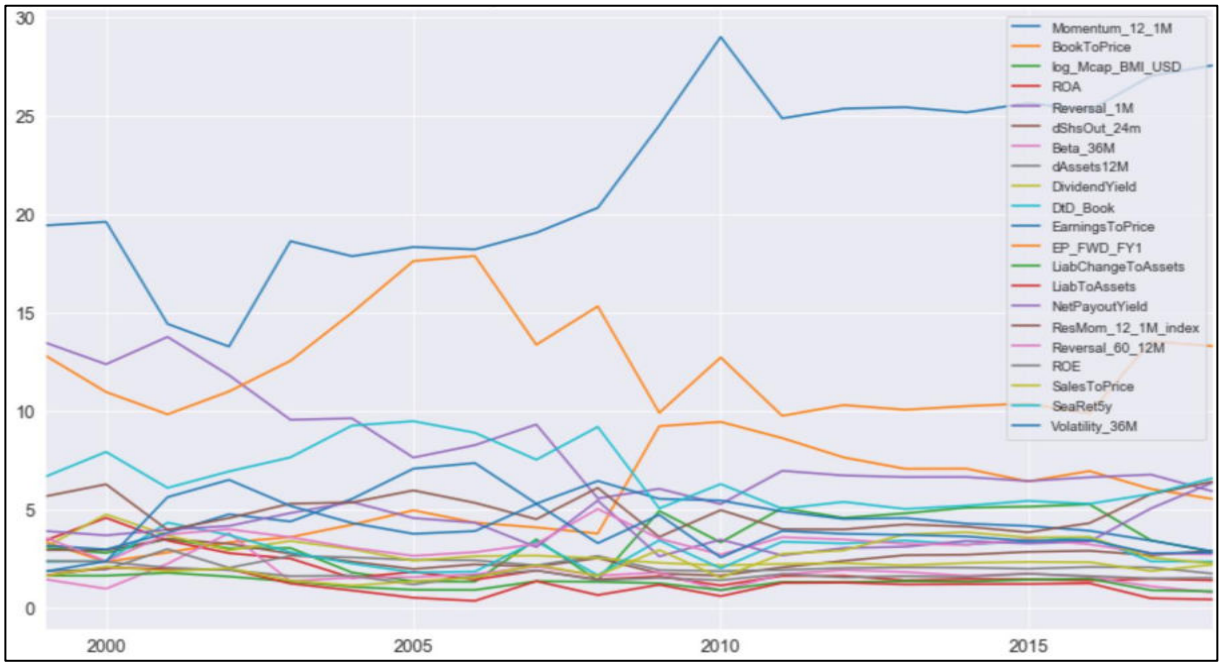



*Figure A.12(ii).* Feature importance over time for random forests (expanding perspective), 1999 to 2018.

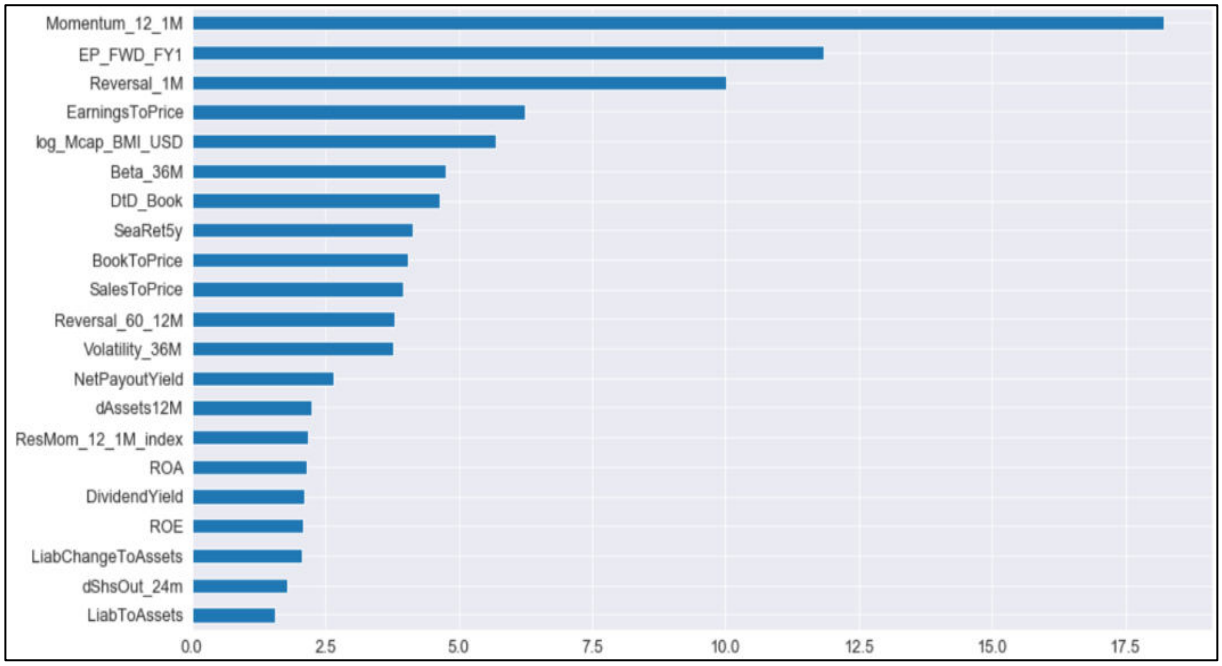



*Figure A.13(i).* Feature importance for boosted regression trees (rolling perspective), 1999 to 2018.

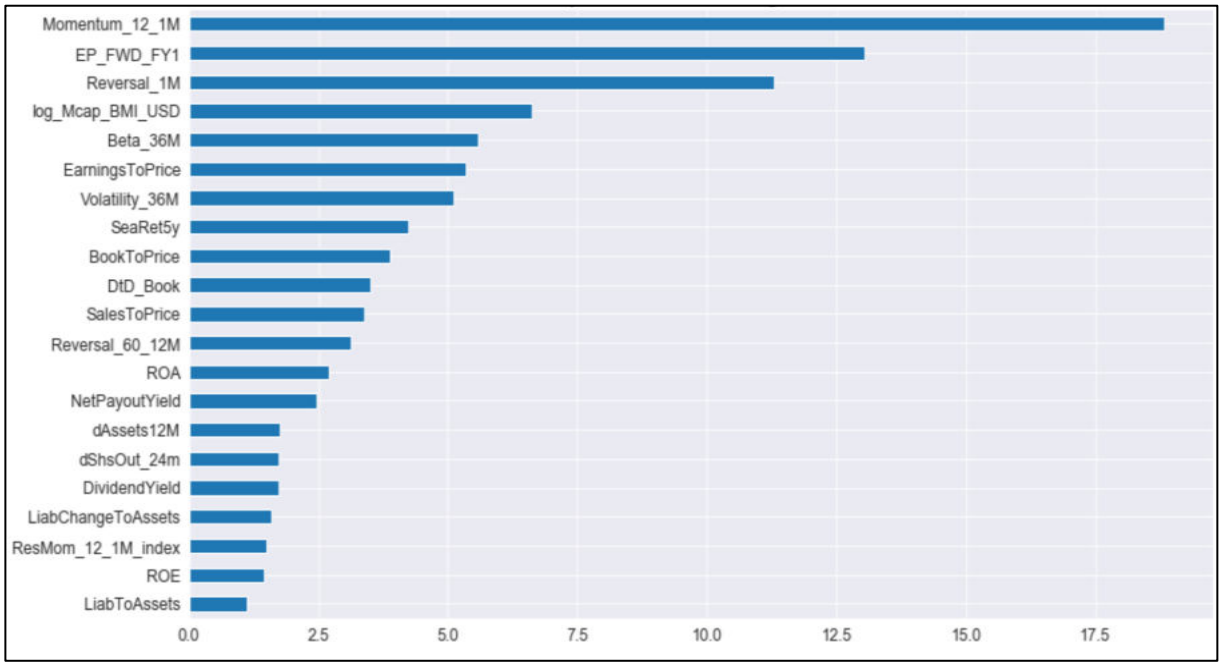



*Figure A.13(ii).* Feature importance for boosted regression trees (expanding perspective), 1999 to 2018.

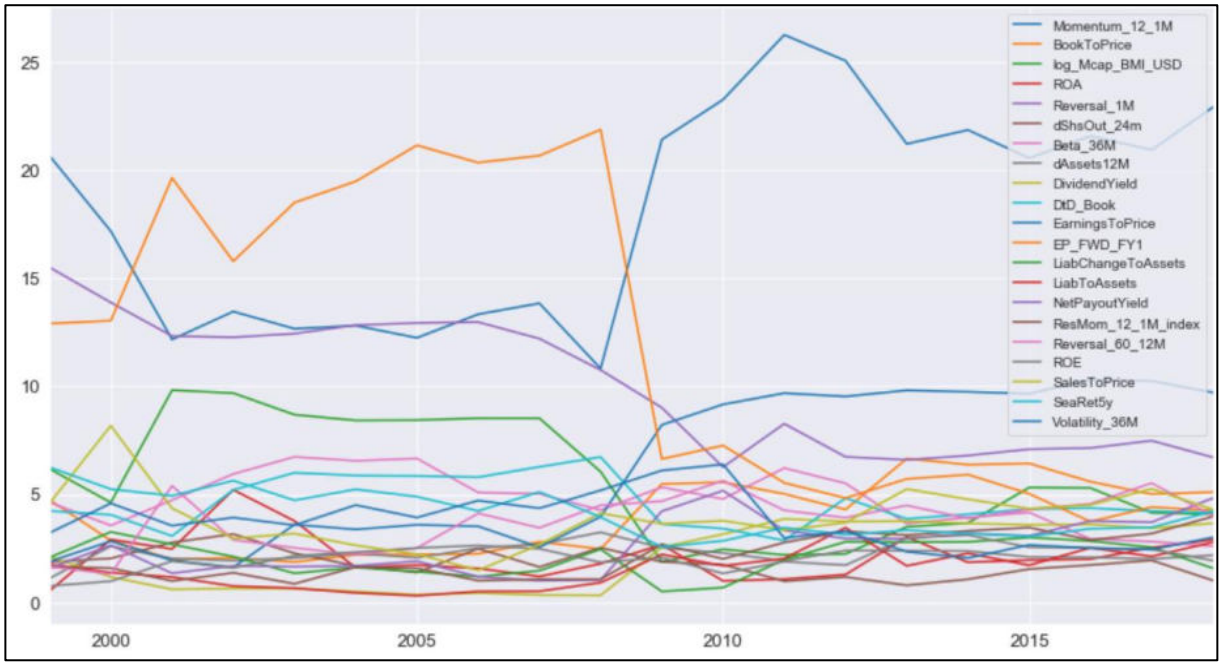



*Figure A.14(i).* Feature importance over time for boosted regression trees (rolling perspective), 1999 to 2018.

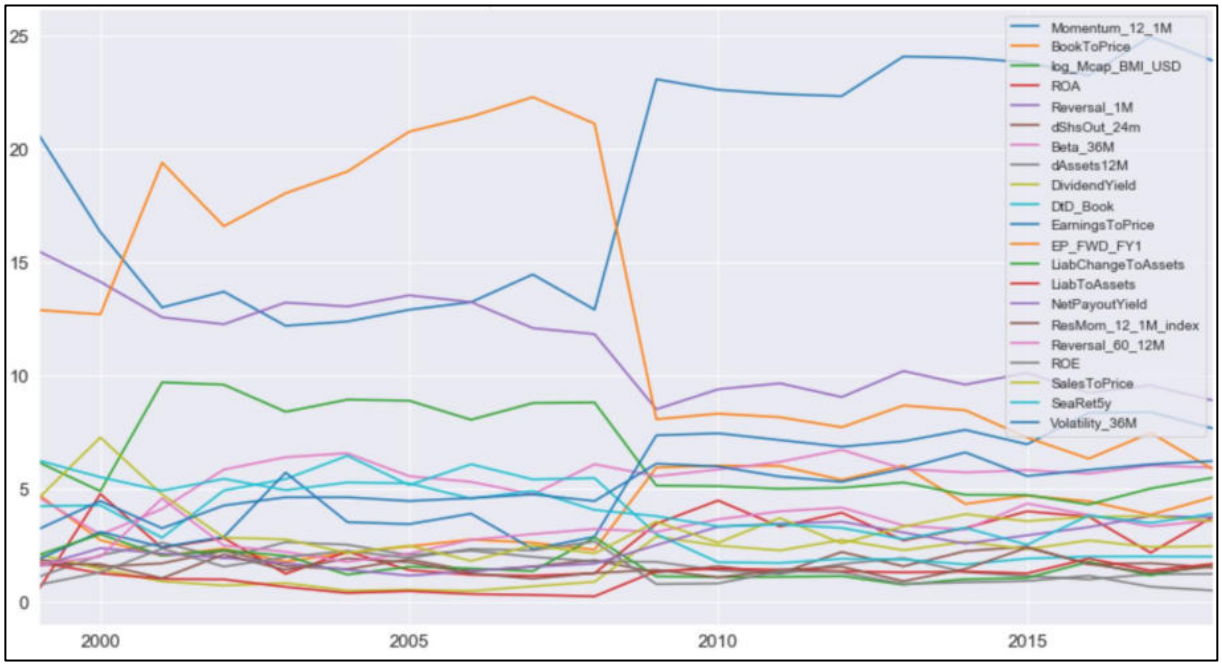



*Figure A.14(ii).* Feature importance over time for boosted regression trees (expanding perspective), 1999 to 2018.

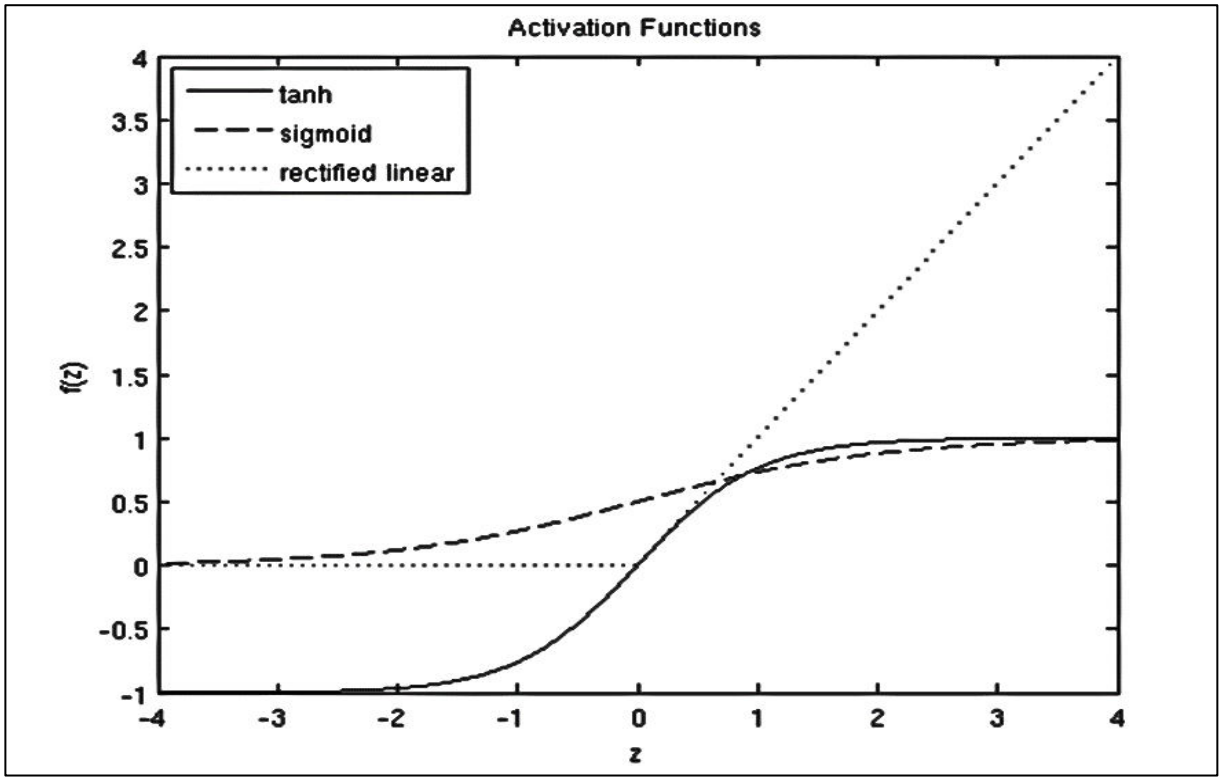



*Figure A.15.* Sigmoid, tanh, and rectified linear functions. Figure A.15 illustrates the three most common activation functions for neural networks: logistic regression hypothesis (Sigmoid), hyperbolic tangent (tanh), and rectified linear unit (ReLU).

# B Tables

| Characteristic | Definition |
|---|---|
| *Momentum_12_1M* | Stock return from month –12 to month –2 (excl. last month) |
| *BookToPrice* | Book value per share in month –1 divided by share price in month –1<br>$$B/P\ ratio = \frac{book\ value\ per\ share}{share\ price} \quad \text{(B.1)}$$ |
| *log_Mcap_BMI_USD* | Log market value of equity in month –1 |
| *ROA* | Income before extraordinary items (NI) in the prior fiscal year divided by total assets (TA) in month –1<br>$$ROA = \frac{NI}{TA} \quad \text{(B.2)}$$ |
| *Reversal_1M* | Stock return in month –1 (short term reversal) |
| *dShsOut_24m* | Change in outstanding shares from month –24 to month –1 |
| *Beta_36M* | Market beta from month –36 to month –1<br>$$\beta_i = \frac{Cov(R_i,\ R_m)}{Var_m} \quad \text{(B.3)}$$ |
| *dAssets12M* | Change in total assets from month –13 to month –1 divided by total assets in moth –1 |
| *DividendYield* | Annual dividend in the prior fiscal year divided by share price in month –1<br>$$Dividend\ Yield = \frac{annual\ dividend}{share\ price} \quad \text{(B.4)}$$ |
| *DtD_Book* | $$DD_t = \frac{\log\left(\frac{V_t}{X_t}\right) + (\mu - 0.5\sigma^2)(T-t)}{\sigma\sqrt{T-t}} \quad \text{(B.5)}$$<br>*Where* $V_t =$ Value of the firm's assets at time t<br>$X_t =$ Strike price of the option<br>$\mu =$ Drift rate of the firm's assets<br>$\sigma =$ Volatility of the assets<br>$T =$ Time to maturity of the option<br>The drift rate $\mu$ and volatility $\sigma$ are not observable. Thus, we follow the procedure of Byström (2006) to estimate them. Equation (B5) then simplifies to: |

| Characteristic | Definition |
|---|---|
| | $$DD_t = \frac{\log\left(\frac{V_t}{X_t}\right)}{\sigma_E\left(1-\frac{V_t}{X_t}\right)} \quad \text{(B.6)}$$ where $\sigma_E$ is the volatility of the firm´s equity. We proxy the value of the firm with the sum of book value of total liabilities and market value of equity. The strike is set at the book value of total liabilities and $\sigma_E$ is assumed to be the past 1- year daily equity return volatility. |
| *EarningsToPrice* | Annual earnings per share in the prior fiscal year divided by share price in month –1 $$E/P\ ratio = \frac{earnings\ per\ share}{share\ price} \quad \text{(B.7)}$$ |
| *EP_FWD_FY1* | I/B/E/S mean FY1 consensus forecast from month –1 divided by share price in month –1 $$FWD\ E/P\ ratio = \frac{forcasted\ earnings\ per\ share}{share\ price} \quad \text{(B.8)}$$ |
| *LiabChangeToAssets* | Δ in liabilities from month –13 to –1 divided by total assets in month –1 $$\Delta L/TA = \frac{\Delta\ liablities}{total\ assets} \quad \text{(B.9)}$$ |
| *LiabToAssets* | Total liabilities in month –1 divided by total assets in month –1 $$L/A\ ratio = \frac{total\ debts}{total\ assets} \quad \text{(B.10)}$$ |
| *NetPayoutYield* | Annual dividends plus net share issuance in the prior fiscal year divided by price in month –1 |
| ResMom_12_1M_index | Residual stock returns from month –12 to month –2 (excl. last month)[47] |
| *Reversal_60_12M* | Stock return from month –60 to month –12 (long-term reversal) |
| *ROE* | Income before extraordinary items (NI) in the prior fiscal year divided by book equity (BE) in month –1 $$ROE = \frac{NI}{BE} \quad \text{(B.11)}$$ |

[47] For details see Blitz, Huij, & Martens (2011).

| Characteristic | Definition |
|---|---|
| *SalesToPrice* | Annual sales per stock in the prior fiscal year divided by share price $$S/P\ ratio = \frac{sales\ per\ share}{share\ price} \quad \text{(B.12)}$$ |
| SeaRet5y | Average return of month +1 of the last 5 years (seasonality) |
| *Volatility_36M* | Standard deviation from month –36 to month –1 |

*Table B.1.* Characteristics and definitions of our predictor set

**Note**: There are two important details about these characteristics. First, our characteristics were quite persistent because they either represented level values that change slowly (such as BookToPrice) or flow values measured over at least one year (such as DividendYield). Second, many of the characteristics were correlated because they were mechanically related (such as ROA and ROE) or because they captured related aspects of the enterprise (such as Beta_36M and Volatility_36M).

| | Avg | Std | N |
|---|---|---|---|
| *Momentum_12_1M* | 0.12 | 0.39 | 4177 |
| *BookToPrice* | 0.64 | 0.48 | 4177 |
| *log_Mcap_BMI_USD* | 6.64 | 1.49 | 4177 |
| *ROA* | 0.03 | 0.09 | 4177 |
| *Reversal_1M* | 0.01 | 0.10 | 4177 |
| *dShsOut_24m* | 0.03 | 0.09 | 4177 |
| *Beta_36M* | 1.05 | 0.71 | 4177 |
| *dAssets12M* | 1.13 | 0.29 | 4177 |
| *DividendYield* | 1.92 | 2.15 | 4177 |
| *DtD_Book* | 4.68 | 2.03 | 4177 |
| *EarningsToPrice* | 0.03 | 0.12 | 4177 |
| *EP_FWD_FY1* | 0.05 | 0.06 | 4177 |
| *LiabChangeToAssets* | 0.04 | 0.12 | 4177 |
| *LiabToAssets* | 0.53 | 0.20 | 4177 |
| *NetPayoutYield* | -0.54 | 7.44 | 4177 |
| *ResMom_12_1M_index* | -0.03 | 0.27 | 4177 |
| *Reversal_60_12M* | 0.94 | 1.72 | 4177 |
| *ROE* | 0.07 | 0.25 | 4177 |
| *SalesToPrice* | 1.58 | 1.89 | 4177 |
| *SeaRet5y* | 1.47 | 4.95 | 4177 |
| *Volatility_36M* | 37.70 | 16.83 | 4177 |
| *TotalReturnUSD (%)* | 0.84 | 10.92 | 4177 |

*Table B.2.* Descriptive statistics for monthly returns, 1989 to 2018.The numbers represent time-series averages of the cross-sectional mean (Avg), standard deviation (Std), and sample size (N) for each variable.

| | Slope | *t*-stat | N |
|---|---|---|---|
| *Momentum_12_1M* | 0.74 | 3.02 | 4177 |
| *BookToPrice* | 0.12 | 1.04 | 4177 |
| *log_Mcap_BMI_USD* | -0.00 | -0.04 | 4177 |
| *ROA* | 0.31 | 0.43 | 4177 |
| *Reversal_1M* | -1.99 | -3.92 | 4177 |
| *dShsOut_24m* | -1.18 | -1.38 | 4177 |
| *Beta_36M* | 0.01 | 0.13 | 4177 |
| *dAssets12M* | -0.14 | -1.40 | 4177 |
| *DividendYield* | -0.01 | -0.52 | 4177 |
| *DtD_Book* | 0.01 | 0.22 | 4177 |
| *EarningsToPrice* | -0.72 | -1.57 | 4177 |
| *EP_FWD_FY1* | 4.63 | 3.61 | 4177 |
| *LiabChangeToAssets* | -0.29 | -1.53 | 4177 |
| *LiabToAssets* | -0.44 | -1.70 | 4177 |
| *NetPayoutYield* | 0.01 | 0.48 | 4177 |
| *ResMom_12_1M_index* | 0.03 | 0.17 | 4177 |
| *Reversal_60_12M* | -0.08 | -2.90 | 4177 |
| *ROE* | 0.01 | 0.04 | 4177 |
| *SalesToPrice* | 0.04 | 2.14 | 4177 |
| *SeaRet5y* | 0.04 | 6.17 | 4177 |
| *Volatility_36M* | -0.00 | -1.16 | 4177 |

*Table B.3:* Fama-MacBeth cross-sectional regressions, 1989 to 2018. Monthly returns (in %) are regressed on lagged firm characteristics, and the numbers represent time-series averages of the slope estimates (Slope) and sample size (N) for each variable. The *t*-statistics are based on the time-series variability of the slopes, incorporating a Newey-West correction with four legs.

Perform a feedforward pass, computing the activations for layers $l, \ldots, L$

1. For each output unit $u$ in layer $L$ (the output layer), set

$$\delta_u^{(L)} = \frac{\partial}{\partial z_u^L} \frac{1}{2} \|r - g(x; \theta, b)\|^2 = -(r_u - a_u^L) \cdot f'(z_u^L) \tag{B.11}$$

2. For $l = L-1, L-2, L-3, \ldots, 2$

   For each node $u$ in layer $l$, set

$$\delta_{u^{(l)}}^{(L)} = \left( \sum_{u^{(l+1)}}^{U^{(l+1)}} \theta_{u^{(l+1)}, u^{(l)}}^{(l)} \, \delta_{u^{(l+1)}}^{(l+1)} \right) \cdot f'(z_{u^{(l)}}^{l}) \tag{B.12}$$

3. Compute the desired partial derivatives, which are given as:

$$\begin{aligned} \frac{\partial}{\partial \theta_{u^{(l)}, u^{l+1}}^{(l)}} \mathcal{L}(\theta, b; x, r) &= a_{u^{(l+1)}}^{l} \delta_{u^{(l)}}^{(l+1)} \\ \frac{\partial}{\partial b^{(l)}} \mathcal{L}(\theta, b; x, r) &= \delta_u^{(l+1)}. \end{aligned} \tag{B.13}$$

*Table B.4.* Back-propagation algorithm details.

# C Notation

Our starting point is a regression Equation of the following form:

$$r_{i,t} = x'_{i,t}\theta + \epsilon_{i,t}, \qquad i\ stocks = 1, \dots, N_t; t\ months = 1, \dots, T. \tag{C.1}$$

Here $x'_{i,t}$ denotes a *row* vector of regressor values and $\theta$ a *column* vector of coefficients:

$$x'_{i,t} \equiv \left[x_{i,t,1} \dots x_{i,t,M}\right], \qquad \theta \equiv [\theta_1 \dots \theta_M]'. \tag{C.2}$$

In Panel matrix form:

$$r_i = X_i\theta + \epsilon_i, \qquad stocks\ \mathrm{i} = 1, \dots, \mathrm{N}. \tag{C.3}$$

The vectors and matrices $r_i$, $X_i$, $\epsilon_i$ are individual-specific:

**Fama MacBeth**

$$r_i \equiv \underbrace{\left[r_{i,\mathrm{t}}\right]}_{(t \times 1)}, \qquad \mathrm{X}_i \equiv \underbrace{\left[x_{i,t,1} \quad \cdots \quad x_{i,t,M}\right]}_{(t \times M)} \equiv \left[\mathrm{x'}_{i,\mathrm{t}}\right], \qquad \epsilon_i \equiv \underbrace{\left[\epsilon_{i,\mathrm{t}}\right]}_{(t \times 1)} \tag{C.4}$$

In super-compact panel matrix notation:

$$\mathrm{R} \equiv \underbrace{\begin{bmatrix} r_1 \\ \vdots \\ r_{\mathrm{N}} \end{bmatrix}}_{(N \times 1)}, \qquad \mathrm{X} \equiv \underbrace{\begin{bmatrix} \mathrm{X}_1 \\ \vdots \\ \mathrm{X}_{\mathrm{N}} \end{bmatrix}}_{(N \times M)}, \qquad \epsilon \equiv \underbrace{\begin{bmatrix} \epsilon_1 \\ \vdots \\ \epsilon_{\mathrm{N}} \end{bmatrix}}_{(N \times 1)} \tag{C.5}$$

**Pooled OLS**

$$r_i \equiv \underbrace{\begin{bmatrix} r_{i,1} \\ \vdots \\ r_{i,\mathrm{T}} \end{bmatrix}}_{(T \times 1)}, \qquad \mathrm{X}_i \equiv \underbrace{\begin{bmatrix} x_{i,1,1} & \cdots & x_{i,1,M} \\ \vdots & & \vdots \\ x_{i,T,1} & \cdots & x_{i,T,M} \end{bmatrix}}_{(T \times M)} \equiv \begin{bmatrix} \mathrm{x'}_{i,1} \\ \vdots \\ \mathrm{x'}_{i,\mathrm{T}} \end{bmatrix}, \qquad \epsilon_i \equiv \underbrace{\begin{bmatrix} \epsilon_{i,1} \\ \vdots \\ \epsilon_{i,\mathrm{T}} \end{bmatrix}}_{(T \times 1)} \tag{C.6}$$

In super-compact panel matrix notation:

$$\mathrm{R} \equiv \underbrace{\begin{bmatrix} r_1 \\ \vdots \\ r_{\mathrm{N}} \end{bmatrix}}_{(NT \times 1)}, \qquad \mathrm{X} \equiv \underbrace{\begin{bmatrix} \mathrm{X}_1 \\ \vdots \\ \mathrm{X}_{\mathrm{N}} \end{bmatrix}}_{(NT \times M)}, \qquad \epsilon \equiv \underbrace{\begin{bmatrix} \epsilon_1 \\ \vdots \\ \epsilon_{\mathrm{N}} \end{bmatrix}}_{(NT \times 1)} \tag{C.7}$$

In this notation our **Fama MacBeth** and **Pooled OLS** regression models translate into the recognizable expression:

$$R = X\theta + \epsilon, \tag{C.8}$$

and:

$$\hat{\theta} = (X'X)^{-1}(X'R), \tag{C.9}$$

follows

# D Standardization

Before performing any preprocessing the splitting of the data set should be completed. Any process that extracts knowledge from the dataset should only be applied to the training portion of the data set, i.e., any validation should be the "outermost loop" in our processing. The Pipeline class in python allows "gluing" together multiple processing steps into a single scikit-learn estimator. The most common use-case of the pipeline class is in chaining preprocessing steps together with a supervised model like a regressor. For each split in the validation set approach, the Scaler is fit with only the Sub training splits, and therefore does not lose any information of the validation split into the parameter search, as illustrated for the first iteration of our splitting approach below:[48]

[48] See chapter "Algorithm Chains and Pipelines" in Müller & Guido (2016) for a review.

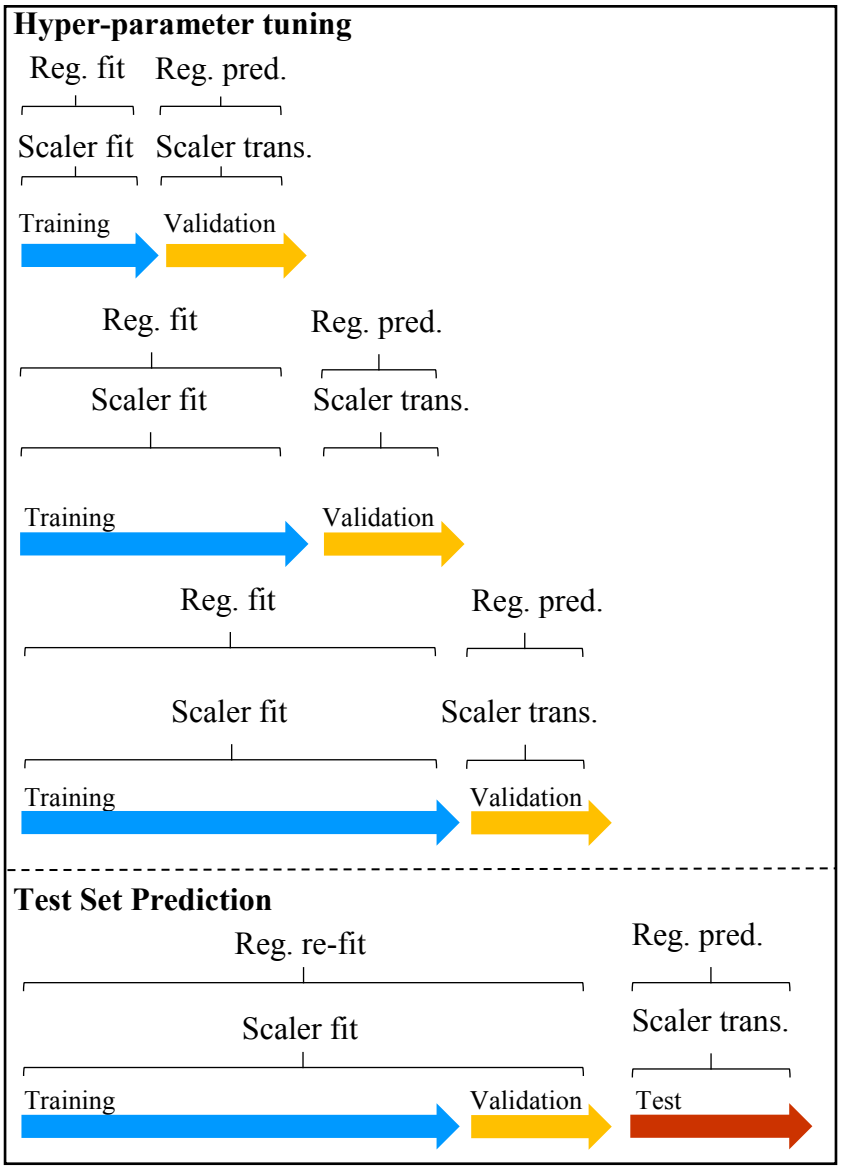



*Figure D.1*. Scaling process for the first iteration of our splitting approach.

**Note**: The standardization includes the following steps:
Step 0: The data is split into TRAINING, VALIDATION and TEST data
Step 1: the scaler is fitted on the TRAINING data
Step 2: the scaler transforms TRAINING data
Step 3: the models are fitted/trained using the transformed TRAINING data
Step 4: the scaler is used to transform the VALIDATION data
Step 5: the trained models predict using the transformed VALIDATION data
Step 6: the scaler is fitted on the TRAINING and VALIDATION data
Step 7: the scaler transforms TRAINING and VALIDATION data
Step 8: the model is fitted/trained using the transformed TRAINING and VALIDATION data and the best found parameters during Validation set approach (Steps 1-5)
Step 9: the scaler transforms TEST data
Step 10: the trained model predicts using the transformed TEST data

# E Results for an expanding scheme

## Predictive Slope

| Method | Coefficient | *t*-stat | S.E. |
|---|---|---|---|
| Fama & MacBeth | 0.52 | 7.24 | 0.07 |
| Pooled OLS | 0.49 | 5.58 | 0.09 |
| Elastic-net | 0.52 | 5.57 | 0.09 |
| Lasso | 0.52 | 5.54 | 0.09 |
| Ridge | 0.49 | 5.58 | 0.09 |
| GLM | 0.71 | 6.29 | 0.11 |
| Random forest | 0.81 | 7.61 | 0.11 |
| Boosted Regression Tree | 0.65 | 7.57 | 0.09 |
| NN1 | 0.44 | 8.60 | 0.05 |
| NN2 | 0.56 | 8.81 | 0.06 |
| NN3 | 0.49 | 8.70 | 0.06 |

*Table E.1.* Fama-MacBeth regression results from current firm characteristics and 10-year expanding perspective coefficients. All numbers equal time-series averages of monthly cross-sectional parameters. Predictive slopes come from out-of-sample Fama-MacBeth regressions of 1-month returns on the return forecasts; *t*-statistics are based on the time-series variability of the predictive slopes, incorporating a Newey-West correction with 4 lags. Estimates of expected 1-month returns derived from all models described in section 4 have significant out-of-sample predictive power for actual 1-month returns.

## Portfolios

| Fama & MacBeth | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.21 | 0.66 | 2.15 | 1.07 | 1.85 | 11 | 15 | 244 | 1.47 | 4.39 |
| 2 | 0.59 | 0.31 | 1.05 | 1.03 | 1.79 | 5 | 12 | 273 | 1.65 | 4.04 |
| 3 | 0.35 | 0.26 | 0.95 | 0.96 | 1.55 | 6 | 11 | 365 | 1.70 | 3.53 |
| 4 | 0.18 | 0.11 | 1.00 | 0.38 | 0.53 | 6 | 16 | 306 | 1.71 | 1.41 |
| 5 | 0.03 | 0.08 | 0.93 | 0.31 | 0.47 | 5 | 15 | 914 | 1.71 | 1.13 |
| 6 | -0.11 | -0.02 | 0.98 | -0.09 | -0.12 | 3 | 19 | 4141 | 1.71 | -0.38 |
| 7 | -0.26 | -0.12 | 0.91 | -0.45 | -0.58 | 4 | 32 | 6089 | 1.71 | -2.09 |
| 8 | -0.45 | -0.29 | 0.87 | -1.16 | -1.28 | 4 | 69 | 6606 | 1.71 | -4.90 |
| 9 | -0.72 | -0.39 | 1.09 | -1.24 | -1.30 | 4 | 93 | 6454 | 1.63 | -4.99 |
| Low (L) | -1.54 | -0.80 | 3.11 | -0.89 | -1.10 | 12 | 193 | 6849 | 1.37 | -3.62 |
| H-L | 2.74 | 1.47 | 3.50 | 1.45 | 2.85 | 13 | 15 | 151 | 1.40 | 6.42 |
| Pooled OLS | | | | | | | | | | |
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.36 | 0.56 | 2.52 | 0.77 | 1.49 | 10 | 18 | 275 | 1.31 | 2.76 |
| 2 | 0.71 | 0.27 | 1.00 | 0.93 | 1.62 | 3 | 11 | 518 | 1.55 | 3.34 |
| 3 | 0.47 | 0.26 | 0.97 | 0.93 | 1.56 | 4 | 11 | 334 | 1.64 | 3.39 |
| 4 | 0.30 | 0.12 | 0.94 | 0.43 | 0.60 | 4 | 16 | 398 | 1.66 | 1.62 |
| 5 | 0.15 | 0.05 | 0.96 | 0.19 | 0.27 | 5 | 14 | 488 | 1.67 | 0.79 |
| 6 | 0.01 | 0 | 0.89 | -0.01 | -0.02 | 4 | 14 | 3104 | 1.67 | -0.06 |
| 7 | -0.14 | -0.12 | 0.81 | -0.53 | -0.65 | 4 | 32 | 6726 | 1.67 | -2.17 |
| 8 | -0.31 | -0.23 | 0.80 | -1.01 | -1.14 | 3 | 57 | 6606 | 1.64 | -3.90 |
| 9 | -0.56 | -0.36 | 1.03 | -1.23 | -1.33 | 4 | 90 | 6543 | 1.56 | -5.01 |
| Low (L) | -1.31 | -0.73 | 2.79 | -0.90 | -1.10 | 10 | 175 | 6849 | 1.23 | -3.74 |
| H-L | 2.66 | 1.29 | 3.55 | 1.26 | 2.52 | 14 | 22 | 151 | 1.26 | 4.99 |
| Elastic-net | | | | | | | | | | |
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.30 | 0.57 | 2.54 | 0.78 | 1.49 | 10 | 19 | 275 | 1.31 | 2.79 |
| 2 | 0.68 | 0.26 | 1.02 | 0.89 | 1.49 | 3 | 12 | 518 | 1.54 | 3.41 |
| 3 | 0.45 | 0.28 | 0.95 | 1.00 | 1.71 | 4 | 10 | 426 | 1.64 | 3.62 |
| 4 | 0.28 | 0.14 | 0.95 | 0.51 | 0.72 | 4 | 15 | 398 | 1.67 | 1.95 |
| 5 | 0.14 | 0.05 | 0.93 | 0.20 | 0.27 | 4 | 13 | 672 | 1.68 | 0.76 |
| 6 | 0.01 | -0.03 | 0.95 | -0.12 | -0.16 | 4 | 14 | 4169 | 1.68 | -0.54 |
| 7 | -0.14 | -0.10 | 0.85 | -0.41 | -0.53 | 3 | 31 | 4508 | 1.66 | -1.63 |
| 8 | -0.31 | -0.25 | 0.84 | -1.02 | -1.15 | 3 | 58 | 6606 | 1.64 | -3.86 |
| 9 | -0.55 | -0.38 | 1.05 | -1.24 | -1.34 | 4 | 93 | 6454 | 1.56 | -5.06 |
| Low (L) | -1.25 | -0.74 | 2.81 | -0.91 | -1.11 | 10 | 177 | 6849 | 1.24 | -3.75 |
| H-L | 2.56 | 1.30 | 3.58 | 1.26 | 2.53 | 14 | 22 | 151 | 1.26 | 4.96 |

| Lasso | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.29 | 0.57 | 2.54 | 0.78 | 1.50 | 10 | 19 | 275 | 1.31 | 2.80 |
| 2 | 0.67 | 0.26 | 1.03 | 0.89 | 1.50 | 3 | 12 | 518 | 1.55 | 3.36 |
| 3 | 0.45 | 0.26 | 0.97 | 0.92 | 1.53 | 4 | 11 | 210 | 1.64 | 3.40 |
| 4 | 0.28 | 0.15 | 0.94 | 0.54 | 0.75 | 5 | 15 | 398 | 1.67 | 1.99 |
| 5 | 0.14 | 0.06 | 0.93 | 0.22 | 0.31 | 4 | 12 | 397 | 1.67 | 0.86 |
| 6 | 0 | -0.02 | 0.94 | -0.08 | -0.11 | 4 | 12 | 4320 | 1.67 | -0.38 |
| 7 | -0.14 | -0.11 | 0.85 | -0.44 | -0.56 | 3 | 32 | 6058 | 1.66 | -1.73 |
| 8 | -0.31 | -0.24 | 0.82 | -1.03 | -1.18 | 3 | 58 | 6422 | 1.64 | -3.96 |
| 9 | -0.55 | -0.38 | 1.05 | -1.26 | -1.35 | 4 | 94 | 6454 | 1.56 | -5.02 |
| Low (L) | -1.24 | -0.73 | 2.81 | -0.90 | -1.11 | 10 | 177 | 6849 | 1.24 | -3.73 |
| H-L | 2.53 | 1.30 | 3.59 | 1.26 | 2.52 | 14 | 22 | 151 | 1.27 | 4.95 |
| Ridge | | | | | | | | | | |
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.36 | 0.56 | 2.52 | 0.77 | 1.49 | 10 | 18 | 275 | 1.31 | 2.76 |
| 2 | 0.71 | 0.27 | 1.00 | 0.93 | 1.62 | 3 | 11 | 518 | 1.55 | 3.34 |
| 3 | 0.47 | 0.26 | 0.97 | 0.93 | 1.56 | 4 | 11 | 334 | 1.64 | 3.39 |
| 4 | 0.30 | 0.12 | 0.94 | 0.43 | 0.60 | 4 | 16 | 398 | 1.66 | 1.62 |
| 5 | 0.15 | 0.05 | 0.96 | 0.19 | 0.27 | 5 | 14 | 488 | 1.67 | 0.79 |
| 6 | 0.01 | 0 | 0.89 | -0.01 | -0.02 | 4 | 14 | 3104 | 1.67 | -0.06 |
| 7 | -0.14 | -0.12 | 0.81 | -0.53 | -0.65 | 4 | 32 | 6726 | 1.67 | -2.17 |
| 8 | -0.31 | -0.23 | 0.80 | -1.01 | -1.14 | 3 | 57 | 6606 | 1.64 | -3.90 |
| 9 | -0.56 | -0.36 | 1.03 | -1.23 | -1.33 | 4 | 90 | 6543 | 1.56 | -5.01 |
| Low (L) | -1.31 | -0.73 | 2.79 | -0.90 | -1.10 | 10 | 175 | 6849 | 1.23 | -3.74 |
| H-L | 2.66 | 1.29 | 3.55 | 1.26 | 2.52 | 14 | 22 | 151 | 1.26 | 4.99 |
| GLM | | | | | | | | | | |
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.11 | 0.65 | 2.52 | 0.90 | 1.59 | 11 | 22 | 337 | 1.13 | 3.29 |
| 2 | 0.61 | 0.42 | 1.16 | 1.26 | 2.62 | 3 | 7 | 151 | 1.49 | 5.55 |
| 3 | 0.39 | 0.27 | 0.86 | 1.09 | 2.14 | 2 | 9 | 578 | 1.59 | 3.97 |
| 4 | 0.23 | 0.15 | 0.87 | 0.61 | 0.94 | 3 | 10 | 306 | 1.64 | 2.36 |
| 5 | 0.10 | 0.12 | 0.87 | 0.47 | 0.76 | 2 | 10 | 2557 | 1.65 | 1.77 |
| 6 | -0.03 | -0.07 | 0.88 | -0.29 | -0.38 | 3 | 24 | 5783 | 1.65 | -1.10 |
| 7 | -0.16 | -0.18 | 0.90 | -0.71 | -0.83 | 5 | 45 | 6606 | 1.63 | -2.89 |
| 8 | -0.30 | -0.28 | 0.92 | -1.05 | -1.19 | 4 | 66 | 6574 | 1.60 | -4.00 |
| 9 | -0.49 | -0.50 | 1.09 | -1.60 | -1.59 | 4 | 118 | 6482 | 1.48 | -6.08 |
| Low (L) | -0.97 | -0.77 | 2.52 | -1.06 | -1.20 | 9 | 181 | 6849 | 1.19 | -4.33 |
| H-L | 2.08 | 1.42 | 3.65 | 1.35 | 2.86 | 10 | 21 | 274 | 1.16 | 5.73 |

| Random Forest | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.04 | 0.86 | 2.24 | 1.33 | 2.62 | 9 | 20 | 182 | 1.24 | 5.41 |
| 2 | 0.65 | 0.44 | 1.16 | 1.33 | 2.20 | 6 | 11 | 333 | 1.47 | 4.78 |
| 3 | 0.47 | 0.28 | 0.81 | 1.19 | 2.22 | 3 | 8 | 333 | 1.59 | 4.77 |
| 4 | 0.31 | 0.23 | 0.73 | 1.10 | 2.10 | 3 | 3 | 122 | 1.66 | 4.23 |
| 5 | 0.17 | 0.08 | 0.75 | 0.38 | 0.62 | 3 | 8 | 516 | 1.66 | 1.44 |
| 6 | 0.02 | -0.04 | 0.75 | -0.17 | -0.22 | 3 | 21 | 2130 | 1.66 | -0.64 |
| 7 | -0.13 | -0.19 | 0.85 | -0.79 | -0.94 | 4 | 46 | 6271 | 1.66 | -3.19 |
| 8 | -0.30 | -0.35 | 0.93 | -1.30 | -1.40 | 3 | 79 | 6846 | 1.61 | -5.08 |
| 9 | -0.53 | -0.60 | 0.98 | -2.11 | -1.92 | 4 | 136 | 6634 | 1.45 | -7.95 |
| Low (L) | -1.04 | -0.91 | 2.28 | -1.38 | -1.50 | 11 | 207 | 6757 | 0.98 | -5.27 |
| H-L | 2.08 | 1.77 | 3.44 | 1.78 | 3.97 | 11 | 28 | 305 | 1.10 | 6.97 |
| Boosted Regression Trees | | | | | | | | | | |
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 2.27 | 1.03 | 3.40 | 1.05 | 2.89 | 8 | 20 | 182 | 1.15 | 3.72 |
| 2 | 0.63 | 0.46 | 1.26 | 1.26 | 2.21 | 6 | 10 | 151 | 1.45 | 5.40 |
| 3 | 0.39 | 0.21 | 1.10 | 0.67 | 1.00 | 6 | 22 | 700 | 1.56 | 2.37 |
| 4 | 0.23 | 0.13 | 1.03 | 0.44 | 0.65 | 6 | 16 | 2677 | 1.61 | 1.62 |
| 5 | 0.09 | 0.09 | 1.05 | 0.31 | 0.42 | 6 | 16 | 763 | 1.63 | 1.10 |
| 6 | -0.04 | -0.05 | 0.96 | -0.18 | -0.24 | 5 | 23 | 4291 | 1.64 | -0.78 |
| 7 | -0.18 | -0.23 | 1.00 | -0.79 | -0.91 | 6 | 52 | 6362 | 1.60 | -3.10 |
| 8 | -0.35 | -0.37 | 0.91 | -1.42 | -1.46 | 5 | 86 | 6726 | 1.57 | -5.47 |
| 9 | -0.57 | -0.55 | 1.15 | -1.66 | -1.64 | 7 | 125 | 6452 | 1.42 | -6.22 |
| Low (L) | -1.20 | -0.91 | 2.40 | -1.31 | -1.42 | 10 | 210 | 6604 | 1.07 | -5.29 |
| H-L | 3.47 | 1.94 | 3.68 | 1.83 | 5.67 | 8 | 21 | 151 | 1.09 | 6.38 |
| NN1 | | | | | | | | | | |
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.71 | 0.65 | 2.52 | 0.89 | 1.70 | 7 | 22 | 242 | 1.22 | 3.27 |
| 2 | 0.86 | 0.38 | 0.98 | 1.36 | 2.70 | 2 | 5 | 185 | 1.49 | 5.93 |
| 3 | 0.55 | 0.23 | 0.81 | 0.97 | 1.65 | 4 | 7 | 483 | 1.58 | 4.00 |
| 4 | 0.33 | 0.13 | 0.89 | 0.52 | 0.73 | 5 | 11 | 395 | 1.62 | 2.14 |
| 5 | 0.14 | 0.05 | 0.93 | 0.18 | 0.25 | 6 | 15 | 882 | 1.65 | 0.67 |
| 6 | -0.04 | -0.04 | 0.92 | -0.15 | -0.20 | 4 | 24 | 4169 | 1.67 | -0.58 |
| 7 | -0.23 | -0.12 | 0.92 | -0.45 | -0.55 | 4 | 32 | 6545 | 1.67 | -1.88 |
| 8 | -0.45 | -0.31 | 0.84 | -1.28 | -1.35 | 3 | 70 | 6909 | 1.64 | -5.41 |
| 9 | -0.75 | -0.38 | 0.96 | -1.37 | -1.41 | 4 | 88 | 6544 | 1.55 | -5.93 |
| Low (L) | -1.52 | -0.78 | 2.05 | -1.33 | -1.51 | 8 | 192 | 6849 | 1.29 | -5.14 |
| H-L | 3.23 | 1.43 | 2.70 | 1.84 | 4.36 | 6 | 17 | 181 | 1.24 | 7.33 |

| | NN2 | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.54 | 0.84 | 2.71 | 1.08 | 2.57 | 7 | 20 | 394 | 1.17 | 3.70 |
| 2 | 0.77 | 0.43 | 0.96 | 1.55 | 3.34 | 2 | 4 | 151 | 1.48 | 7.02 |
| 3 | 0.49 | 0.31 | 0.84 | 1.27 | 2.43 | 2 | 7 | 121 | 1.55 | 4.85 |
| 4 | 0.28 | 0.19 | 0.86 | 0.77 | 1.19 | 3 | 8 | 275 | 1.63 | 2.81 |
| 5 | 0.09 | 0.04 | 0.86 | 0.15 | 0.19 | 5 | 17 | 973 | 1.64 | 0.52 |
| 6 | -0.08 | -0.06 | 0.97 | -0.20 | -0.26 | 6 | 24 | 3014 | 1.66 | -0.73 |
| 7 | -0.26 | -0.14 | 0.96 | -0.51 | -0.61 | 6 | 38 | 3289 | 1.66 | -1.98 |
| 8 | -0.47 | -0.33 | 0.92 | -1.25 | -1.29 | 5 | 77 | 6634 | 1.63 | -5.04 |
| 9 | -0.74 | -0.53 | 1.06 | -1.72 | -1.65 | 4 | 122 | 6574 | 1.53 | -7.86 |
| Low (L) | -1.48 | -0.95 | 2.24 | -1.46 | -1.56 | 9 | 221 | 6849 | 1.26 | -5.80 |
| H-L | 3.02 | 1.79 | 3.13 | 1.98 | 6.17 | 6 | 11 | 242 | 1.20 | 8.10 |
| | NN3 | | | | | | | | | |
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.65 | 0.77 | 2.62 | 1.01 | 2.12 | 8 | 17 | 182 | 1.20 | 3.65 |
| 2 | 0.87 | 0.36 | 1.01 | 1.23 | 2.06 | 6 | 6 | 62 | 1.49 | 5.97 |
| 3 | 0.60 | 0.24 | 0.79 | 1.03 | 1.87 | 2 | 9 | 422 | 1.57 | 3.86 |
| 4 | 0.39 | 0.18 | 0.84 | 0.72 | 1.18 | 3 | 8 | 182 | 1.64 | 3.00 |
| 5 | 0.21 | 0.06 | 0.91 | 0.23 | 0.33 | 4 | 14 | 518 | 1.65 | 0.89 |
| 6 | 0.02 | -0.04 | 0.97 | -0.15 | -0.19 | 6 | 26 | 4809 | 1.68 | -0.60 |
| 7 | -0.17 | -0.16 | 0.92 | -0.61 | -0.73 | 4 | 36 | 3926 | 1.67 | -2.38 |
| 8 | -0.40 | -0.31 | 0.91 | -1.17 | -1.26 | 3 | 72 | 5995 | 1.65 | -5.31 |
| 9 | -0.71 | -0.41 | 0.98 | -1.45 | -1.48 | 4 | 96 | 6362 | 1.57 | -6.12 |
| Low (L) | -1.49 | -0.86 | 2.14 | -1.39 | -1.50 | 9 | 194 | 6909 | 1.31 | -5.33 |
| H-L | 3.13 | 1.63 | 3.06 | 1.85 | 4.85 | 8 | 10 | 242 | 1.25 | 7.55 |

*Table E.2.* Performance of equal-weighted decile portfolios sorted on out-of-sample return forecasts.

**Note**: Table E.2 reports the average predicted monthly returns (Pred), the average realized monthly returns (Avg), their realized standard deviations (Std), the information ratios (IR), the sortino ratios (SR), the most extreme negative monthly returns (Max 1M Loss), the maximum drawdowns (Max DD), the maximum drawdown durations (Max DD DUR), the average percentage changes in holdings (TO), and the *t*-statistics (*t*-stat) of equally-weighted decile portfolios sorted on out-of-sample return forecasts.

# F Lasso

## Tables

| | Rolling perspective | Expanding perspective |
|---|---|---|
| *Avg* | 0.05 | 0.06 |
| *Std* | 0.70 | 0.71 |
| *p10* | -0.72 | -0.73 |
| *p90* | 0.81 | 0.84 |

*Table F.1.* Cross-sectional stock forecasts, 1999 to 2018. Table F.1 reports the distribution (average, standard deviation, 10th and 90th percentiles) of 1-month forecasts derived from current firm characteristics and 10-year rolling or expanding penalized slope estimates. All numbers equal time-series averages of monthly cross-sectional parameters. Estimates of expected 1-month returns derived from penalized pooled time-series cross-sectional OLS regressions vary substantially across stocks.

## Figures

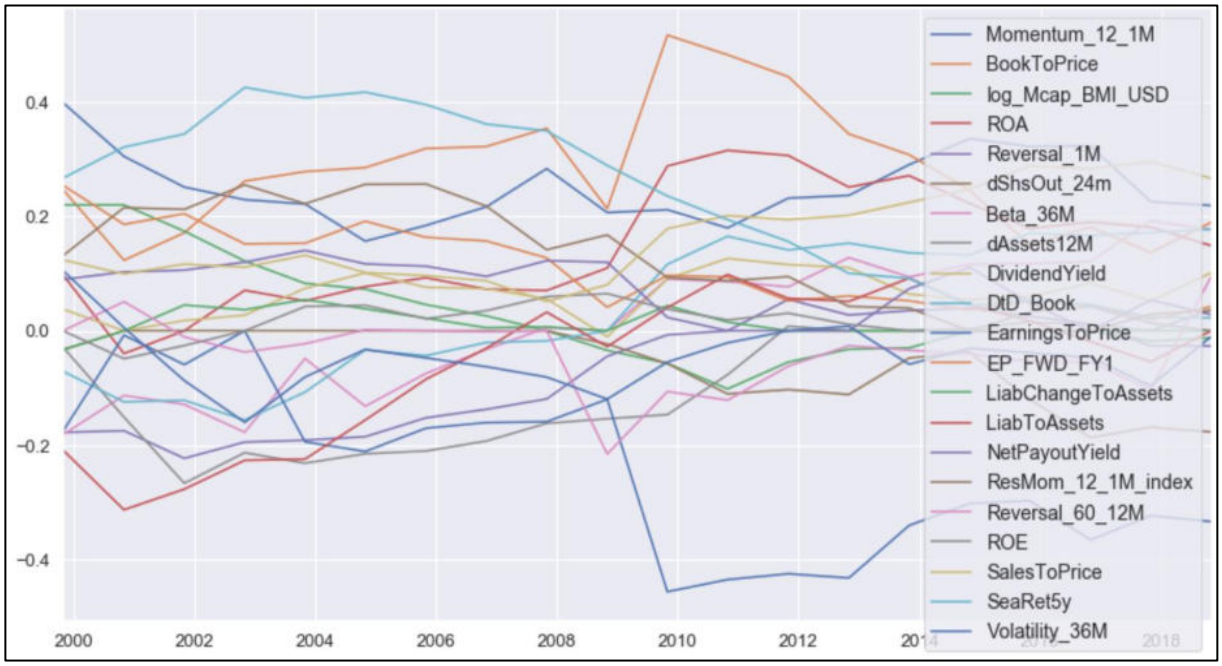



*Figure F.1(i):* 10-year rolling slope estimates, 1999 to 2018.

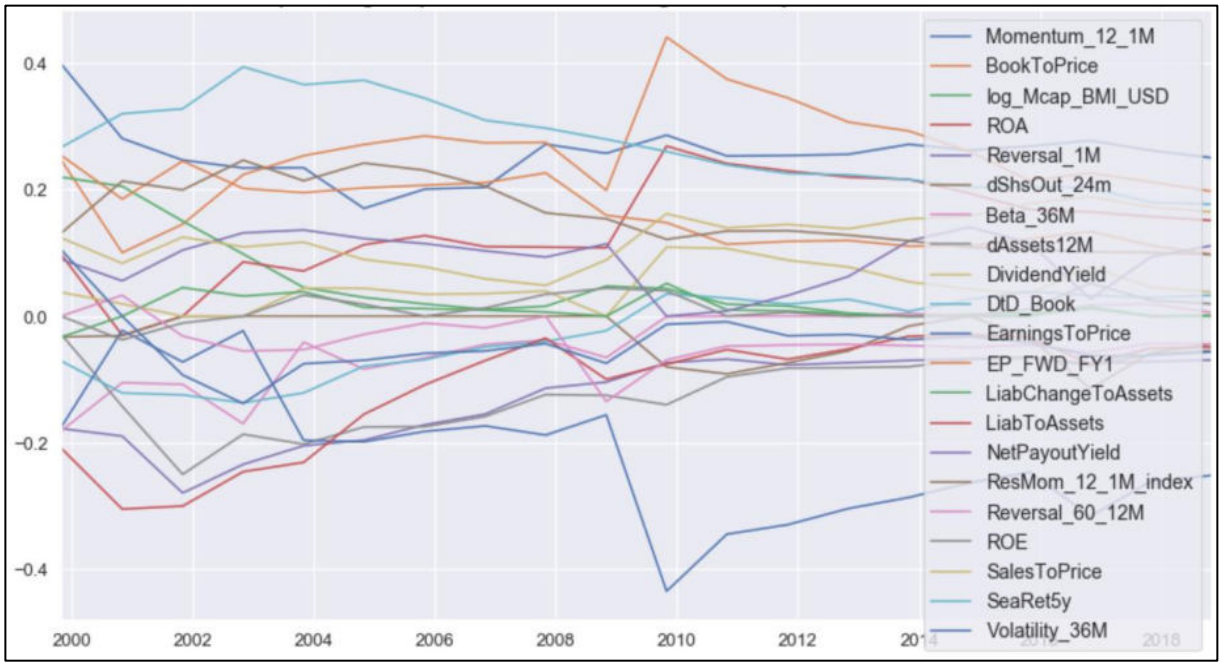



*Figure F.1(ii):* 10-year expanding slope estimates, 1999 to 2018.

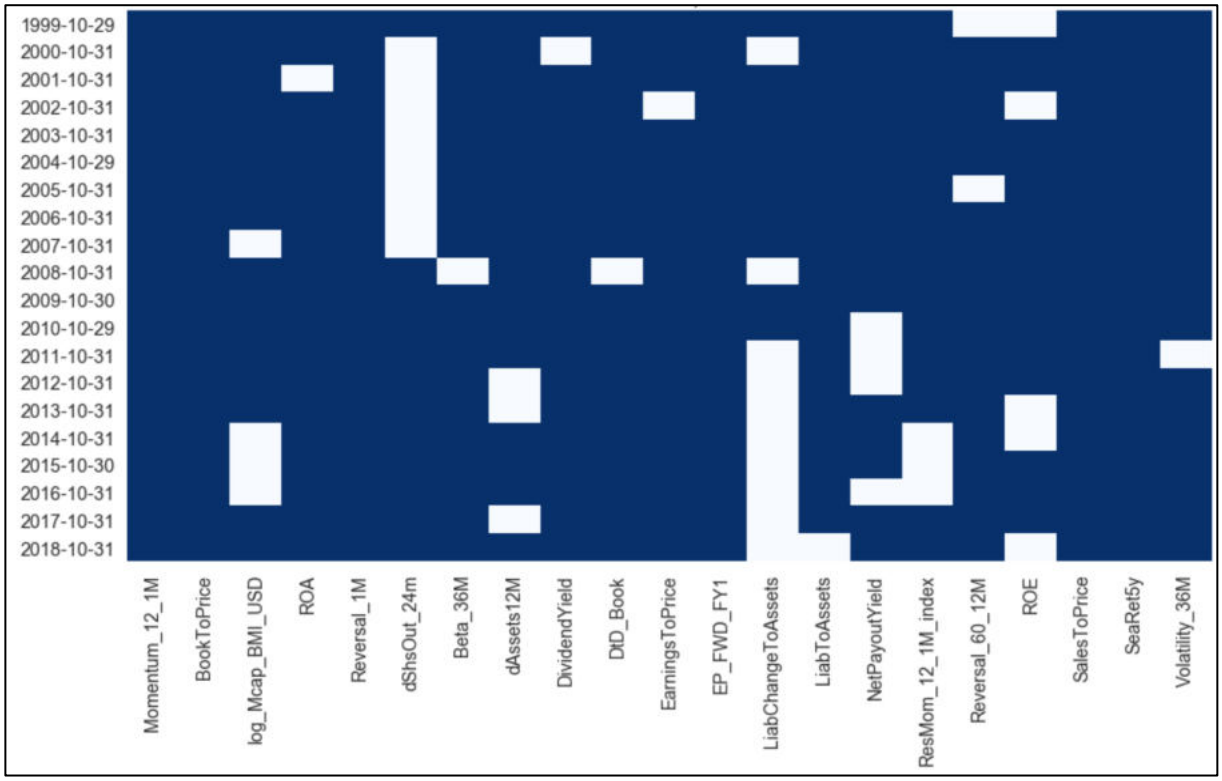



*Figure F.2(i).* Selected features (dark blue) using lasso rolling perspective, 1999 to 2018.

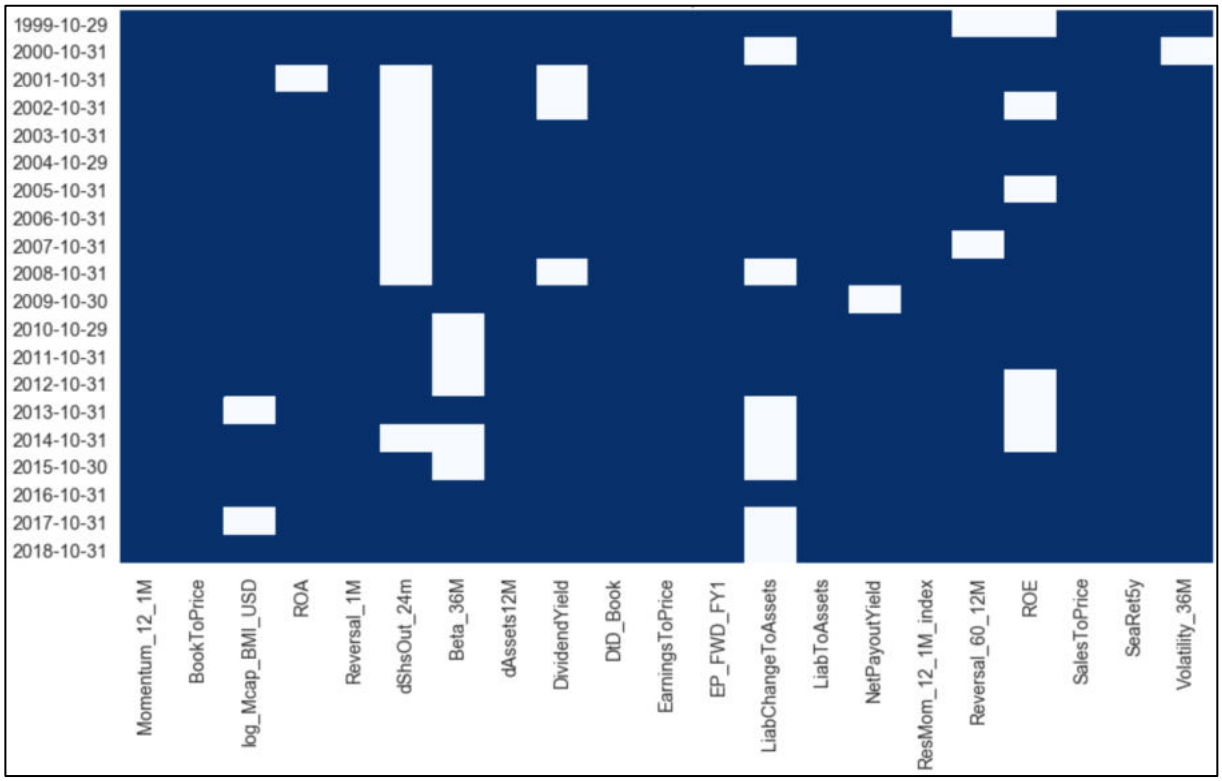



*Figure F.2(ii)*. Selected features (dark blue) using lasso expanding perspective, 1999 to 2018.

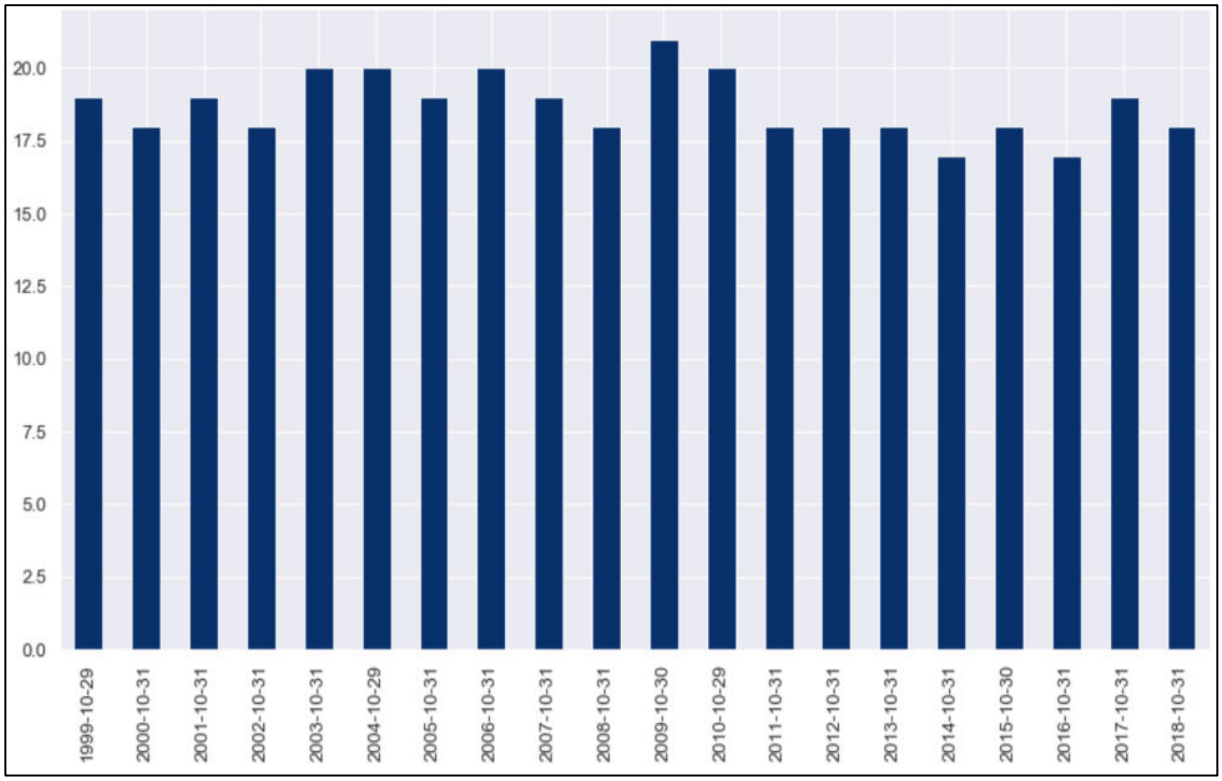



*Figure F.3(i).* Number of features (rolling perspective), 1999 to 2018.

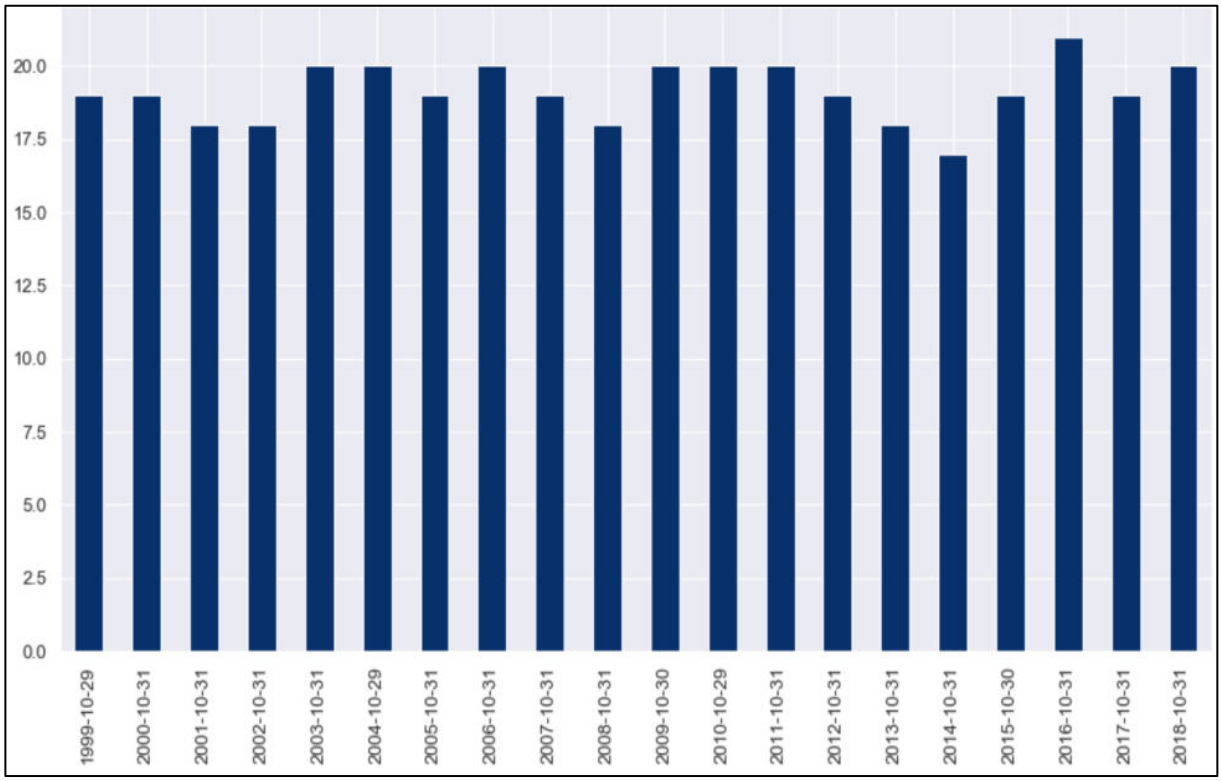



*Figure F.3(ii).* Number of features (expanding perspective), 1999 to 2018.




*Figure F.4(i)*. Average slopes (rolling perspective), 1999 to 2018.

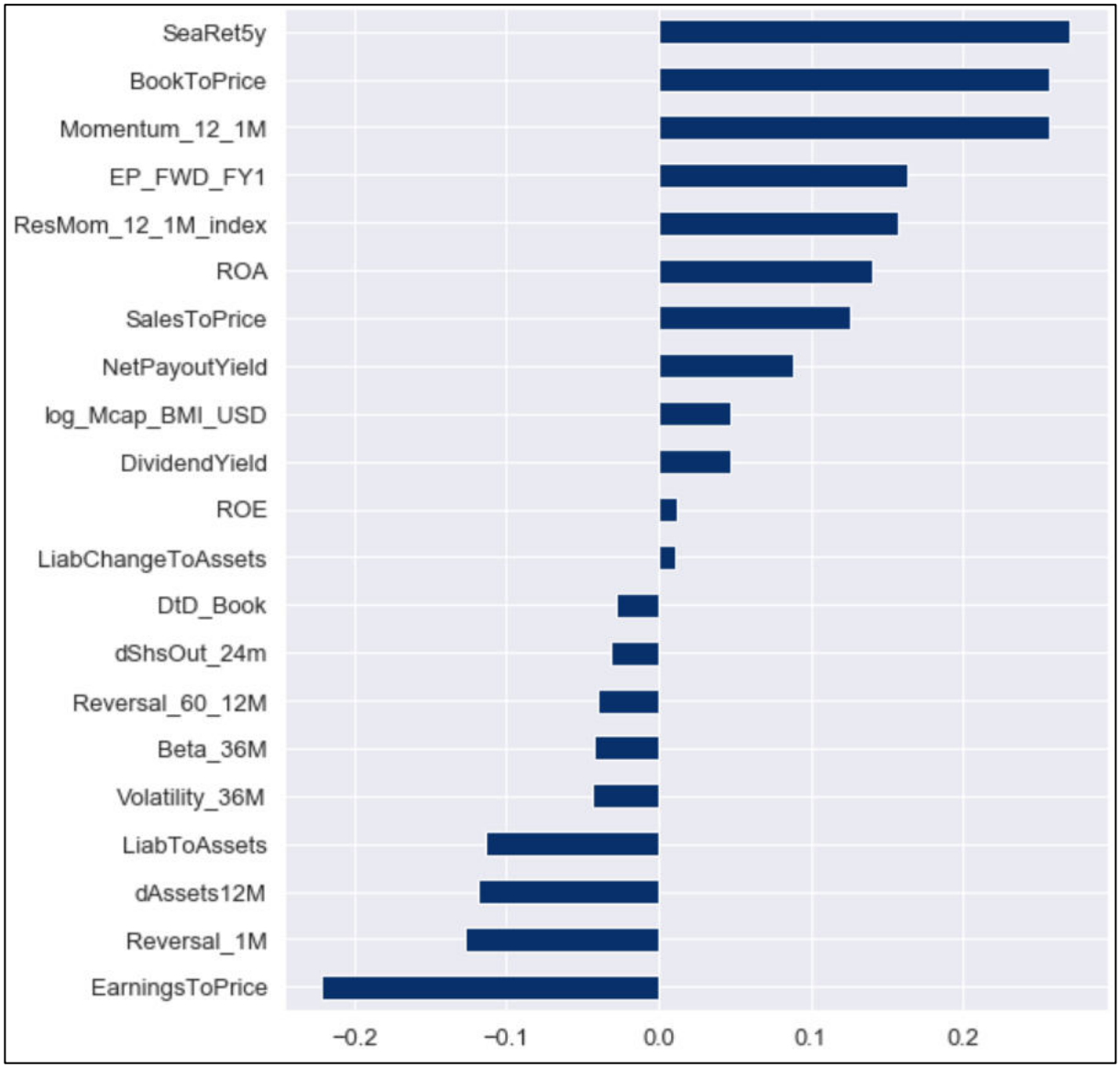



*Figure F.4(ii)*. Average slopes (expanding perspective), 1999 to 2018.

# G Ridge

## Tables

| | Rolling perspective | Expanding perspective |
|---|---|---|
| *Avg* | 0.06 | 0.07 |
| *Std* | 0.74 | 0.75 |
| *p10* | -0.75 | -0.76 |
| *p90* | 0.85 | 0.88 |

*Table G.1.* Cross-sectional stock forecasts, 1999 to 2018. Table G.1 reports the distribution (average, standard deviation, $10^{th}$ and $90^{th}$ percentiles) of 1-month forecasts derived from current firm characteristics and 10-year rolling (rolling perspective) or expanding (expanding perspective) penalized slope estimates. All numbers equal time-series averages of monthly cross-sectional parameters. Estimates of expected 1-month returns derived from penalized pooled time-series cross-sectional OLS regressions vary substantially across stocks.

## Figures

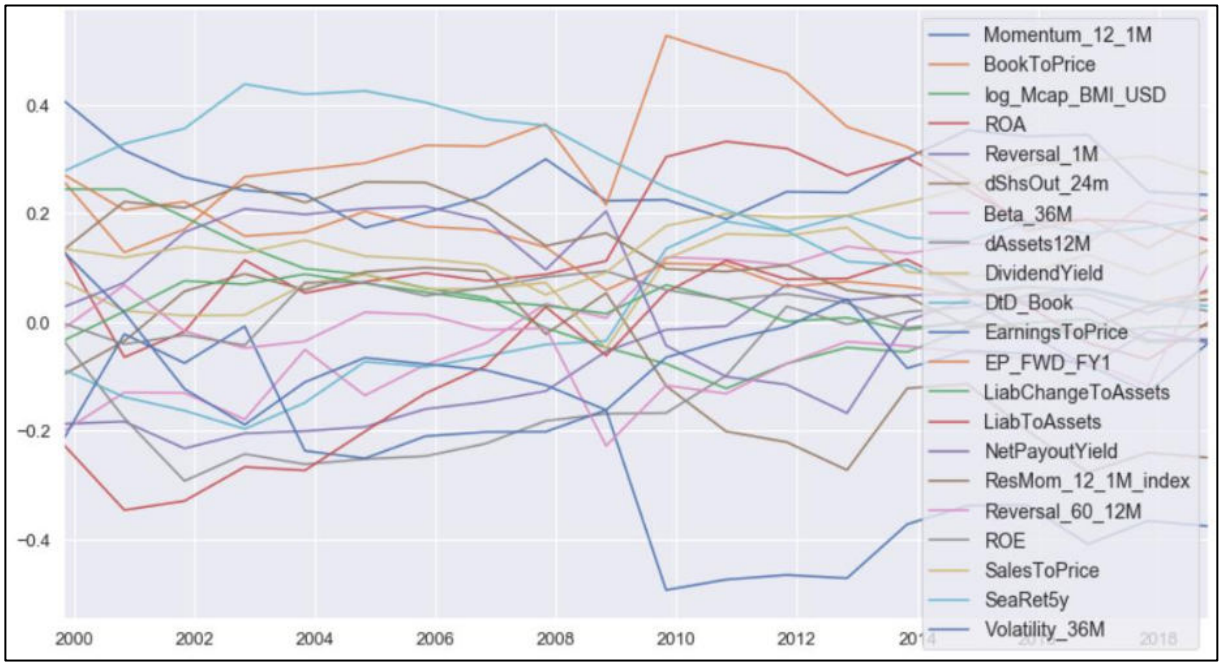



*Figure G.1(i).* 10-year rolling slope estimates, 1999 to 2018.

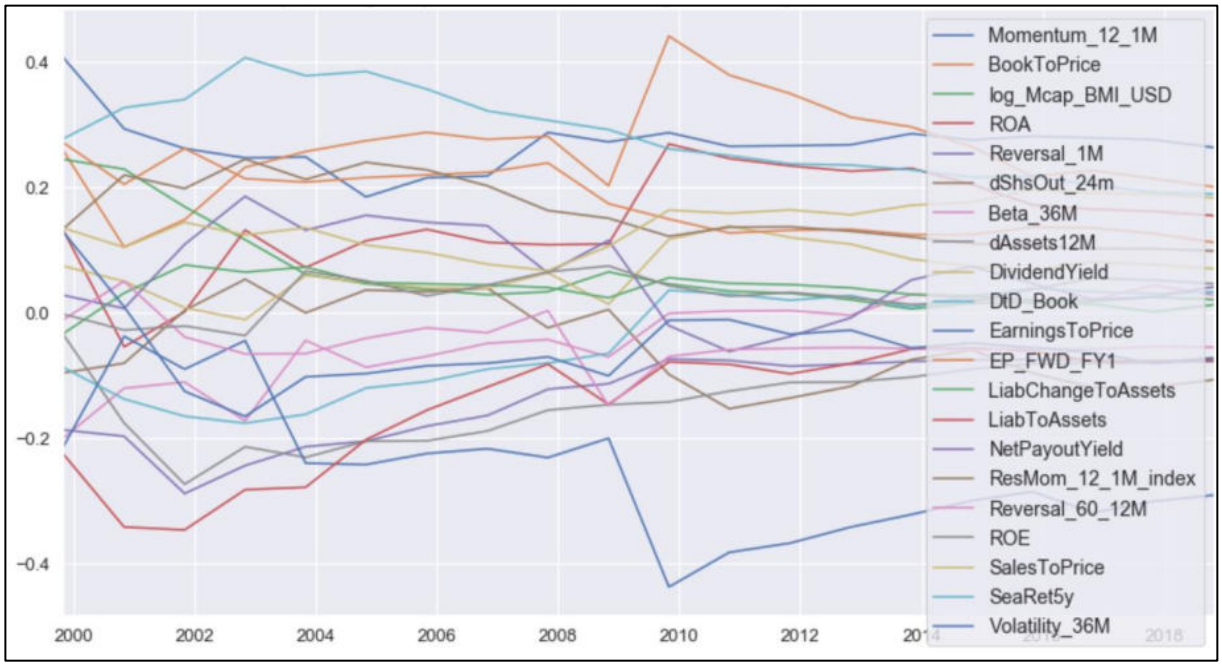



*Figure G.1(ii).* 10-year expanding slope estimates, 1999 to 2018.

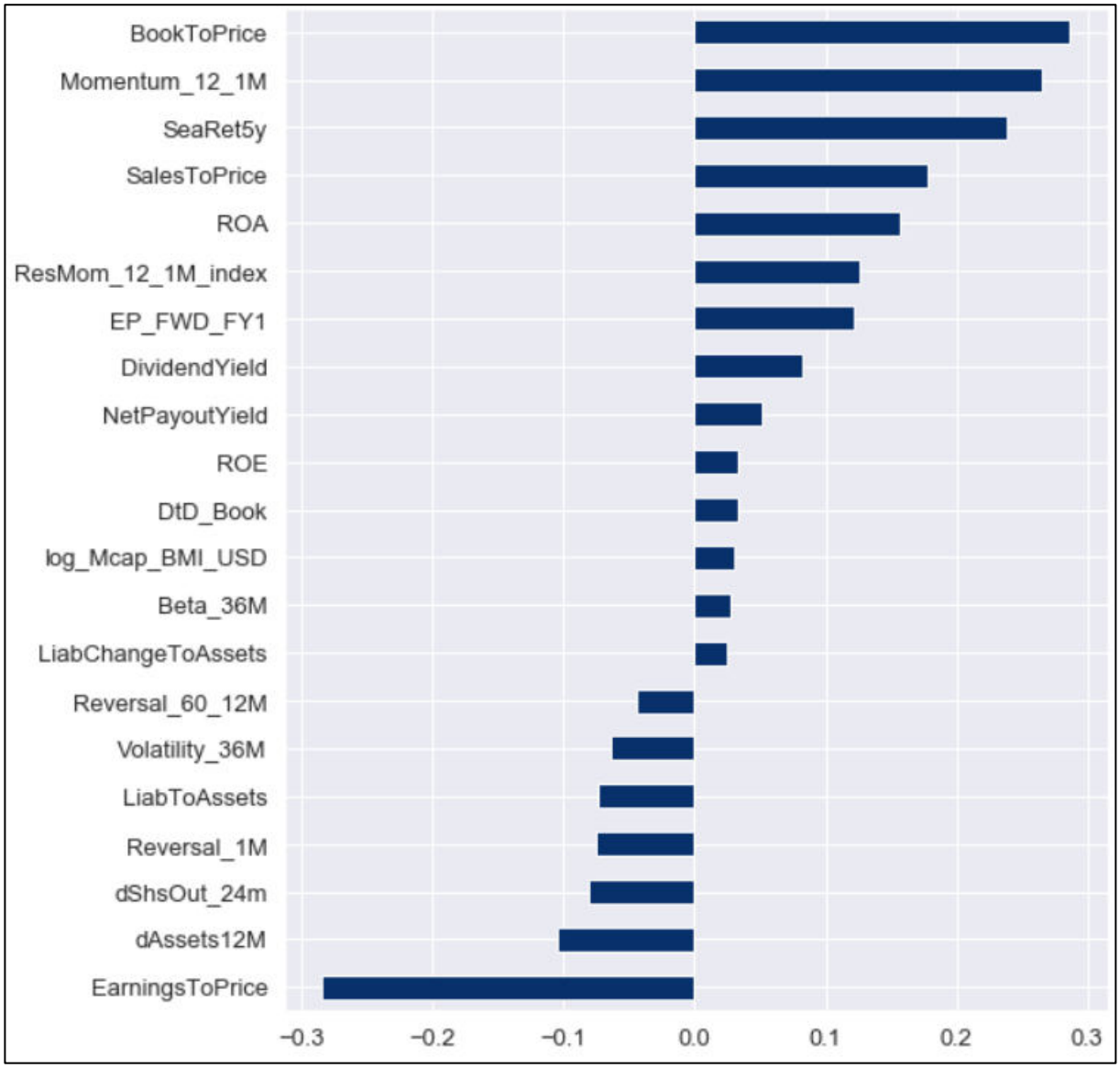



*Figure G.2(i).* Average slopes (rolling perspective), 1999 to 2018.

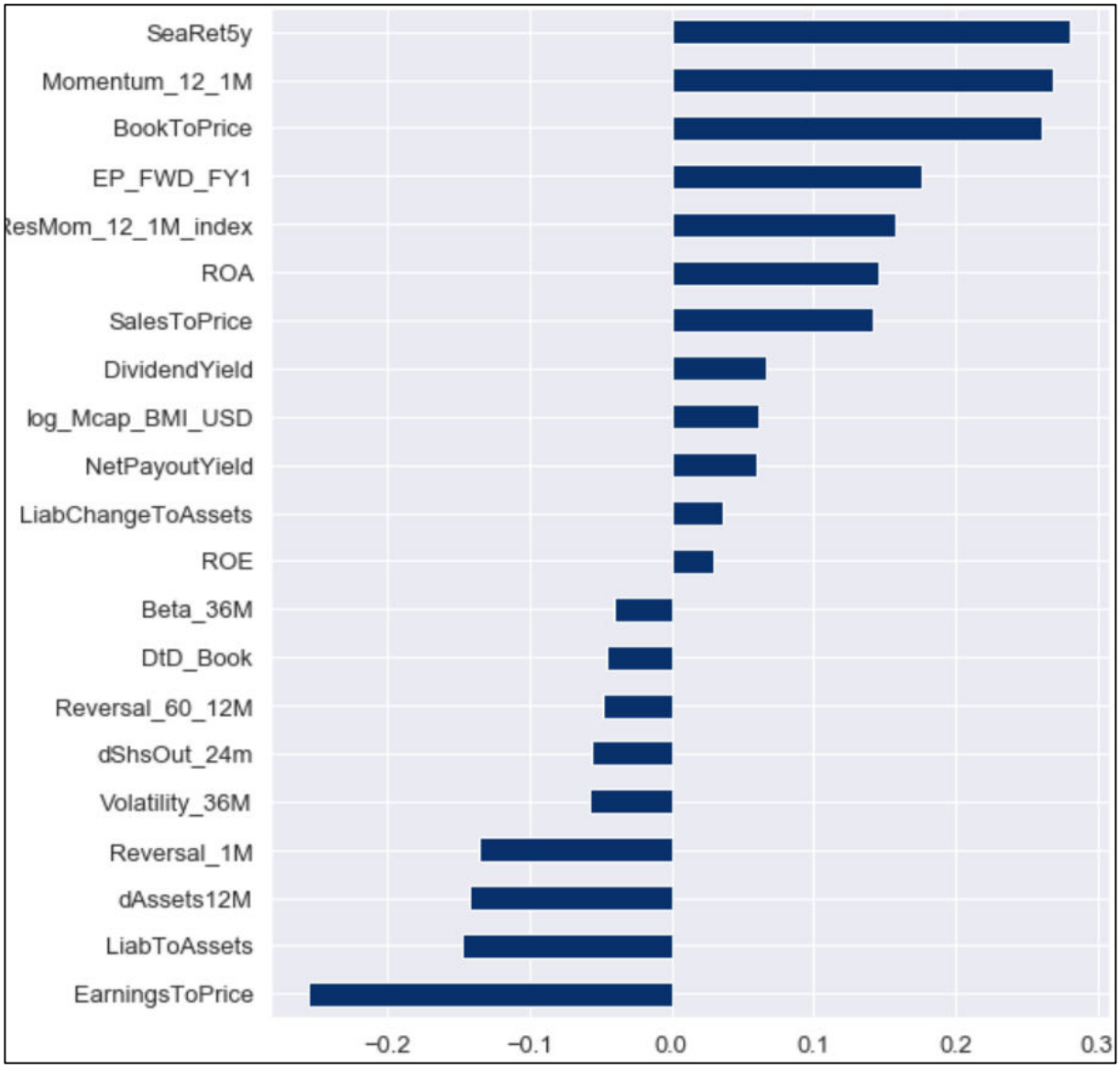



*Figure G.2(ii).* Average slopes (expanding perspective), 1999 to 2018.

# H Robustness Checks

## Tables

| | Avg | Std | N |
|---|---|---|---|
| *Momentum_12_1M* | 0.10 | 0.29 | 1375 |
| *BookToPrice* | 0.59 | 0.43 | 1375 |
| *log_Mcap_BMI_USD* | 8.25 | 1.29 | 1375 |
| *ROA* | 0.05 | 0.06 | 1375 |
| *Reversal_1M* | 0.01 | 0.08 | 1375 |
| *dShsOut_24m* | 0.02 | 0.06 | 1375 |
| *Beta_36M* | 0.97 | 0.61 | 1375 |
| *dAssets12M* | 1.09 | 0.20 | 1375 |
| *DividendYield* | 2.25 | 2.05 | 1375 |
| *DtD_Book* | 5.08 | 1.91 | 1375 |
| *EarningsToPrice* | 0.04 | 0.08 | 1375 |
| *EP_FWD_FY1* | 0.06 | 0.04 | 1375 |
| *LiabChangeToAssets* | 0.03 | 0.01 | 1375 |
| *LiabToAssets* | 0.58 | 0.19 | 1375 |
| *NetPayoutYield* | 1.10 | 5.66 | 1375 |
| *ResMom_12_1M_index* | -0.02 | 0.26 | 1375 |
| *Reversal_60_12M* | 0.74 | 1.22 | 1375 |
| *ROE* | 0.12 | 0.18 | 1375 |
| *SalesToPrice* | 1.43 | 1.57 | 1375 |
| *SeaRet5y* | 1.19 | 3.86 | 1375 |
| *Volatility_36M* | 30.87 | 11.1 | 1375 |
| *TotalReturnUSD (%)* | 0.81 | 8.68 | 1375 |

*Table H.1.* Descriptive statistics for monthly returns, 1989 to 2018. The numbers represent time-series averages of the cross-sectional mean (Avg), standard deviation (Std), and sample size (N) for each variable.

## Predictive Slope

| Method | Coefficient | *t*-stat | S.E. |
|---|---|---|---|
| Fama & MacBeth | 0.42 | 3.78 | 0.11 |
| Pooled OLS | 0.42 | 4.06 | 0.10 |
| Elastic-net | 0.43 | 3.83 | 0.11 |
| Lasso | 0.43 | 3.79 | 0.11 |
| Ridge | 0.42 | 4.06 | 0.10 |
| GLM | 0.85 | 4.43 | 0.19 |
| Random forest | 0.73 | 5.44 | 0.13 |
| Boosted Regression Tree | 0.62 | 4.63 | 0.13 |
| NN1 | 0.56 | 5.79 | 0.10 |
| NN2 | 0.49 | 5.37 | 0.09 |
| NN3 | 0.53 | 4.99 | 0.11 |

*Table H.2.* Fama-MacBeth regression results from current firm characteristics and 10-year expanding perspective coefficients. All numbers equal time-series averages of monthly cross-sectional parameters. Predictive slopes come from out-of-sample Fama-MacBeth regressions of 1-month returns on the return forecasts; *t*-statistics are based on the time-series variability of the predictive slopes, incorporating a Newey-West correction with 4 lags. Estimates of expected 1-month returns derived from all models described in section 4 have significant out-of-sample predictive power for actual 1-month returns.

## Portfolios

| Fama & MacBeth | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 0.89 | 0.37 | 2.21 | 0.57 | 0.86 | 10 | 28 | 427 | 1.47 | 2.16 |
| 2 | 0.41 | 0.32 | 1.37 | 0.82 | 1.50 | 4 | 20 | 1096 | 1.66 | 2.62 |
| 3 | 0.21 | 0.21 | 1.01 | 0.70 | 1.16 | 5 | 16 | 1372 | 1.71 | 2.81 |
| 4 | 0.06 | 0.16 | 1.03 | 0.55 | 0.84 | 6 | 13 | 762 | 1.75 | 2.21 |
| 5 | -0.07 | 0.05 | 1.00 | 0.16 | 0.22 | 4 | 16 | 1586 | 1.76 | 0.61 |
| 6 | -0.20 | 0.03 | 0.84 | 0.13 | 0.18 | 3 | 8 | 853 | 1.76 | 0.61 |
| 7 | -0.33 | -0.11 | 0.99 | -0.39 | -0.51 | 4 | 35 | 6179 | 1.76 | -1.64 |
| 8 | -0.49 | -0.20 | 0.97 | -0.70 | -0.82 | 5 | 48 | 6241 | 1.73 | -2.77 |
| 9 | -0.70 | -0.30 | 1.21 | -0.86 | -0.98 | 4 | 73 | 3104 | 1.67 | -3.10 |
| Low (L) | -1.19 | -0.52 | 2.55 | -0.71 | -0.88 | 12 | 128 | 5873 | 1.40 | -3.04 |
| H-L | 2.08 | 0.89 | 3.68 | 0.84 | 1.49 | 12 | 25 | 365 | 1.40 | 3.65 |
| Pooled OLS | | | | | | | | | | |
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.20 | 0.45 | 2.37 | 0.66 | 1.08 | 7 | 23 | 182 | 1.00 | 2.66 |
| 2 | 0.61 | 0.28 | 1.18 | 0.83 | 1.56 | 3 | 12 | 580 | 1.42 | 2.78 |
| 3 | 0.39 | 0.19 | 1.05 | 0.64 | 1.02 | 5 | 15 | 944 | 1.54 | 2.26 |
| 4 | 0.24 | 0.03 | 1.08 | 0.09 | 0.13 | 6 | 17 | 1125 | 1.61 | 0.39 |
| 5 | 0.10 | 0.06 | 0.95 | 0.21 | 0.31 | 4 | 10 | 641 | 1.64 | 0.83 |
| 6 | -0.02 | 0.08 | 0.94 | 0.30 | 0.43 | 4 | 16 | 792 | 1.66 | 1.22 |
| 7 | -0.15 | -0.09 | 0.88 | -0.34 | -0.43 | 3 | 30 | 5082 | 1.65 | -1.47 |
| 8 | -0.29 | -0.17 | 0.94 | -0.61 | -0.76 | 3 | 43 | 3013 | 1.62 | -2.44 |
| 9 | -0.48 | -0.37 | 1.17 | -1.09 | -1.24 | 4 | 87 | 6514 | 1.51 | -4.08 |
| Low (L) | -0.96 | -0.46 | 2.49 | -0.65 | -0.80 | 10 | 112 | 6574 | 1.03 | -2.55 |
| H-L | 2.16 | 0.91 | 3.54 | 0.89 | 1.64 | 11 | 20 | 214 | 1.00 | 3.75 |
| Elastic-net | | | | | | | | | | |
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.13 | 0.42 | 2.37 | 0.62 | 1.01 | 7 | 23 | 427 | 1.01 | 2.51 |
| 2 | 0.58 | 0.30 | 1.18 | 0.88 | 1.61 | 4 | 12 | 580 | 1.45 | 3.04 |
| 3 | 0.37 | 0.16 | 1.03 | 0.55 | 0.90 | 4 | 14 | 1369 | 1.56 | 2.12 |
| 4 | 0.22 | 0.01 | 1.07 | 0.04 | 0.06 | 6 | 19 | 1097 | 1.62 | 0.17 |
| 5 | 0.10 | 0.12 | 0.95 | 0.44 | 0.67 | 4 | 9 | 245 | 1.64 | 1.72 |
| 6 | -0.02 | 0.05 | 0.98 | 0.18 | 0.27 | 4 | 21 | 3014 | 1.66 | 0.74 |
| 7 | -0.14 | -0.09 | 0.89 | -0.34 | -0.43 | 3 | 25 | 2493 | 1.65 | -1.38 |
| 8 | -0.27 | -0.18 | 0.96 | -0.65 | -0.80 | 4 | 48 | 2982 | 1.61 | -2.68 |
| 9 | -0.46 | -0.33 | 1.12 | -1.02 | -1.17 | 4 | 78 | 6634 | 1.52 | -3.81 |
| Low (L) | -0.91 | -0.47 | 2.44 | -0.67 | -0.83 | 10 | 113 | 6604 | 1.05 | -2.65 |
| H-L | 2.04 | 0.89 | 3.51 | 0.88 | 1.62 | 11 | 23 | 731 | 1.02 | 3.64 |

| Lasso | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.12 | 0.42 | 2.37 | 0.62 | 1.01 | 7 | 23 | 427 | 1.02 | 2.48 |
| 2 | 0.57 | 0.30 | 1.20 | 0.86 | 1.58 | 4 | 11 | 580 | 1.43 | 2.98 |
| 3 | 0.36 | 0.15 | 1.05 | 0.49 | 0.79 | 4 | 14 | 1369 | 1.56 | 1.93 |
| 4 | 0.22 | 0.02 | 1.06 | 0.07 | 0.09 | 6 | 19 | 1004 | 1.62 | 0.26 |
| 5 | 0.10 | 0.13 | 0.98 | 0.47 | 0.71 | 4 | 10 | 732 | 1.64 | 1.88 |
| 6 | -0.02 | 0.04 | 0.97 | 0.15 | 0.22 | 4 | 20 | 2678 | 1.66 | 0.62 |
| 7 | -0.14 | -0.08 | 0.90 | -0.29 | -0.37 | 3 | 24 | 2434 | 1.65 | -1.16 |
| 8 | -0.27 | -0.18 | 0.98 | -0.65 | -0.81 | 3 | 51 | 2982 | 1.61 | -2.65 |
| 9 | -0.45 | -0.31 | 1.12 | -0.96 | -1.11 | 4 | 74 | 6634 | 1.51 | -3.53 |
| Low (L) | -0.90 | -0.49 | 2.45 | -0.69 | -0.85 | 10 | 117 | 6604 | 1.06 | -2.75 |
| H-L | 2.02 | 0.91 | 3.49 | 0.91 | 1.67 | 11 | 22 | 731 | 1.03 | 3.74 |
| Ridge | | | | | | | | | | |
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.20 | 0.45 | 2.37 | 0.66 | 1.08 | 7 | 23 | 182 | 1.00 | 2.66 |
| 2 | 0.61 | 0.28 | 1.18 | 0.83 | 1.56 | 3 | 12 | 580 | 1.42 | 2.78 |
| 3 | 0.39 | 0.19 | 1.05 | 0.64 | 1.02 | 5 | 15 | 944 | 1.54 | 2.26 |
| 4 | 0.24 | 0.03 | 1.08 | 0.09 | 0.13 | 6 | 17 | 1125 | 1.61 | 0.39 |
| 5 | 0.10 | 0.06 | 0.95 | 0.21 | 0.31 | 4 | 10 | 641 | 1.64 | 0.83 |
| 6 | -0.02 | 0.08 | 0.93 | 0.30 | 0.43 | 4 | 16 | 792 | 1.66 | 1.22 |
| 7 | -0.15 | -0.09 | 0.88 | -0.34 | -0.43 | 3 | 30 | 5082 | 1.65 | -1.47 |
| 8 | -0.29 | -0.17 | 0.94 | -0.61 | -0.76 | 3 | 43 | 3013 | 1.62 | -2.44 |
| 9 | -0.48 | -0.37 | 1.17 | -1.09 | -1.23 | 4 | 87 | 6514 | 1.51 | -4.08 |
| Low (L) | -0.96 | -0.46 | 2.49 | -0.65 | -0.80 | 10 | 112 | 6574 | 1.03 | -2.55 |
| H-L | 2.16 | 0.91 | 3.54 | 0.89 | 1.65 | 11 | 20 | 214 | 1.00 | 3.75 |
| GLM | | | | | | | | | | |
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 0.73 | 0.49 | 2.20 | 0.78 | 1.27 | 9 | 22 | 485 | 0.99 | 2.89 |
| 2 | 0.43 | 0.29 | 1.28 | 0.78 | 1.38 | 4 | 10 | 364 | 1.43 | 3.36 |
| 3 | 0.28 | 0.28 | 1.20 | 0.80 | 1.58 | 3 | 7 | 609 | 1.58 | 3.49 |
| 4 | 0.17 | 0.13 | 0.94 | 0.48 | 0.76 | 3 | 11 | 245 | 1.62 | 2.03 |
| 5 | 0.08 | 0.14 | 0.94 | 0.50 | 0.75 | 3 | 9 | 1278 | 1.65 | 2.15 |
| 6 | -0.01 | 0.05 | 0.78 | 0.23 | 0.34 | 3 | 16 | 1280 | 1.63 | 0.87 |
| 7 | -0.10 | -0.06 | 1.04 | -0.19 | -0.26 | 3 | 30 | 3774 | 1.62 | -0.68 |
| 8 | -0.20 | -0.23 | 1.02 | -0.77 | -0.89 | 4 | 53 | 6603 | 1.57 | -3.40 |
| 9 | -0.32 | -0.43 | 1.25 | -1.21 | -1.30 | 6 | 100 | 6514 | 1.45 | -4.92 |
| Low (L) | -0.59 | -0.65 | 2.21 | -1.02 | -1.13 | 8 | 147 | 6603 | 1.03 | -4.14 |
| H-L | 1.31 | 1.14 | 3.49 | 1.13 | 2.26 | 10 | 18 | 273 | 1.01 | 4.80 |

| Random Forest | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 0.81 | 0.52 | 2.26 | 0.80 | 1.36 | 10 | 20 | 91 | 1.19 | 3.31 |
| 2 | 0.51 | 0.37 | 1.42 | 0.91 | 1.52 | 7 | 19 | 273 | 1.53 | 3.40 |
| 3 | 0.37 | 0.26 | 1.13 | 0.78 | 1.41 | 3 | 10 | 333 | 1.65 | 3.16 |
| 4 | 0.25 | 0.12 | 0.90 | 0.47 | 0.73 | 2 | 11 | 791 | 1.72 | 1.81 |
| 5 | 0.15 | 0.09 | 0.88 | 0.34 | 0.52 | 3 | 11 | 1765 | 1.72 | 1.46 |
| 6 | 0.04 | -0.01 | 0.97 | -0.03 | -0.04 | 5 | 20 | 1520 | 1.69 | -0.11 |
| 7 | -0.08 | -0.09 | 0.96 | -0.34 | -0.43 | 4 | 23 | 5202 | 1.69 | -1.21 |
| 8 | -0.21 | -0.23 | 1.00 | -0.81 | -0.97 | 4 | 55 | 6544 | 1.66 | -3.12 |
| 9 | -0.38 | -0.39 | 1.34 | -1.02 | -1.18 | 4 | 91 | 6697 | 1.55 | -3.95 |
| Low (L) | -0.77 | -0.62 | 2.15 | -1.00 | -1.20 | 9 | 146 | 6604 | 1.07 | -4.27 |
| H-L | 1.58 | 1.15 | 3.40 | 1.17 | 2.18 | 11 | 21 | 273 | 1.12 | 5.38 |
| Boosted Regression Trees | | | | | | | | | | |
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.32 | 0.66 | 3.17 | 0.72 | 1.58 | 10 | 21 | 273 | 1.18 | 2.78 |
| 2 | 0.39 | 0.35 | 1.29 | 0.93 | 1.62 | 4 | 12 | 273 | 1.49 | 3.52 |
| 3 | 0.27 | 0.16 | 1.07 | 0.52 | 0.90 | 3 | 9 | 639 | 1.59 | 2.18 |
| 4 | 0.18 | 0.07 | 1.18 | 0.20 | 0.27 | 6 | 21 | 488 | 1.64 | 0.75 |
| 5 | 0.09 | 0.05 | 1.23 | 0.14 | 0.19 | 7 | 26 | 3257 | 1.67 | 0.56 |
| 6 | 0 | 0 | 1.09 | 0 | 0 | 5 | 17 | 1615 | 1.65 | 0.01 |
| 7 | -0.09 | -0.10 | 1.08 | -0.34 | -0.41 | 7 | 36 | 4990 | 1.67 | -1.23 |
| 8 | -0.21 | -0.20 | 1.11 | -0.64 | -0.81 | 4 | 65 | 5387 | 1.61 | -2.43 |
| 9 | -0.37 | -0.37 | 1.46 | -0.88 | -1.03 | 7 | 90 | 5691 | 1.50 | -3.76 |
| Low (L) | -0.96 | -0.60 | 2.54 | -0.81 | -0.98 | 10 | 141 | 6454 | 1.09 | -3.08 |
| H-L | 2.28 | 1.26 | 3.79 | 1.15 | 2.45 | 10 | 21 | 151 | 1.10 | 4.81 |
| NN1 | | | | | | | | | | |
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.35 | 0.67 | 2.54 | 0.92 | 1.75 | 8 | 20 | 394 | 1.18 | 3.59 |
| 2 | 0.73 | 0.41 | 1.28 | 1.10 | 2.15 | 3 | 7 | 306 | 1.52 | 4.33 |
| 3 | 0.50 | 0.19 | 0.95 | 0.69 | 1.17 | 3 | 8 | 1155 | 1.64 | 2.82 |
| 4 | 0.34 | 0.10 | 0.93 | 0.36 | 0.59 | 3 | 13 | 910 | 1.68 | 1.41 |
| 5 | 0.20 | 0.01 | 1.02 | 0.02 | 0.03 | 4 | 16 | 3563 | 1.69 | 0.08 |
| 6 | 0.06 | -0.06 | 0.83 | -0.23 | -0.30 | 4 | 18 | 2621 | 1.69 | -0.94 |
| 7 | -0.08 | -0.13 | 1.05 | -0.43 | -0.54 | 4 | 40 | 5206 | 1.69 | -1.66 |
| 8 | -0.23 | -0.23 | 0.96 | -0.82 | -0.97 | 4 | 52 | 5873 | 1.65 | -3.62 |
| 9 | -0.44 | -0.40 | 1.26 | -1.10 | -1.21 | 4 | 98 | 5873 | 1.57 | -4.42 |
| Low (L) | -0.93 | -0.54 | 2.31 | -0.82 | -0.95 | 10 | 127 | 6664 | 1.24 | -3.57 |
| H-L | 2.28 | 1.22 | 3.57 | 1.18 | 2.39 | 11 | 17 | 153 | 1.19 | 5.34 |

| | NN2 | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.18 | 0.48 | 2.20 | 0.76 | 1.32 | 7 | 16 | 394 | 1.16 | 3.18 |
| 2 | 0.62 | 0.30 | 1.24 | 0.84 | 1.44 | 3 | 8 | 487 | 1.50 | 3.74 |
| 3 | 0.40 | 0.22 | 1.11 | 0.69 | 1.15 | 3 | 9 | 822 | 1.60 | 2.92 |
| 4 | 0.23 | 0.14 | 0.97 | 0.50 | 0.76 | 3 | 12 | 790 | 1.68 | 2.09 |
| 5 | 0.09 | -0.01 | 0.99 | -0.02 | -0.02 | 4 | 16 | 2527 | 1.69 | -0.09 |
| 6 | -0.05 | 0.02 | 0.90 | 0.09 | 0.12 | 4 | 17 | 3621 | 1.70 | 0.34 |
| 7 | -0.20 | -0.05 | 1.09 | -0.16 | -0.21 | 4 | 20 | 3624 | 1.70 | -0.67 |
| 8 | -0.36 | -0.16 | 1.14 | -0.50 | -0.64 | 4 | 45 | 5874 | 1.66 | -2.00 |
| 9 | -0.57 | -0.32 | 1.18 | -0.93 | -1.08 | 4 | 71 | 6726 | 1.57 | -4.06 |
| Low (L) | -1.07 | -0.62 | 2.37 | -0.91 | -1.04 | 10 | 147 | 5813 | 1.22 | -3.59 |
| H-L | 2.24 | 1.11 | 3.35 | 1.14 | 2.22 | 8 | 13 | 184 | 1.18 | 4.98 |
| | NN3 | | | | | | | | | |
| | Pred | Avg | Std | IR | SR | Max 1M Loss | Max DD | Max DD DUR | TO | *t*-stat |
| High (H) | 1.23 | 0.62 | 2.51 | 0.85 | 1.70 | 9 | 16 | 91 | 1.21 | 3.53 |
| 2 | 0.70 | 0.29 | 1.43 | 0.71 | 1.16 | 5 | 11 | 485 | 1.50 | 2.94 |
| 3 | 0.48 | 0.20 | 1.19 | 0.59 | 0.96 | 4 | 9 | 942 | 1.61 | 2.61 |
| 4 | 0.30 | 0.10 | 1.15 | 0.32 | 0.47 | 5 | 20 | 942 | 1.64 | 1.23 |
| 5 | 0.15 | 0.08 | 1.05 | 0.25 | 0.36 | 6 | 14 | 731 | 1.67 | 1.02 |
| 6 | 0.01 | -0.07 | 1.14 | -0.22 | -0.26 | 7 | 30 | 2588 | 1.67 | -0.84 |
| 7 | -0.13 | -0.16 | 1.13 | -0.49 | -0.57 | 9 | 43 | 5814 | 1.66 | -1.92 |
| 8 | -0.29 | -0.23 | 1.05 | -0.76 | -0.90 | 4 | 56 | 6727 | 1.63 | -3.46 |
| 9 | -0.49 | -0.37 | 1.55 | -0.82 | -0.95 | 8 | 86 | 6544 | 1.54 | -3.06 |
| Low (L) | -0.92 | -0.46 | 2.43 | -0.66 | -0.83 | 9 | 105 | 6544 | 1.20 | -2.70 |
| H-L | 2.15 | 1.08 | 3.49 | 1.07 | 2.08 | 10 | 15 | 333 | 1.19 | 4.69 |

*Table H.3.* Performance of equal-weighted decile portfolios sorted on out-of-sample return forecasts.

**Note**: Table H.3 reports the average predicted monthly returns (Pred), the average realized monthly returns (Avg), their realized standard deviations (Std), the information ratios (IR), the sortino ratios (SR), the most extreme negative monthly returns (Max 1M Loss), the maximum drawdowns (Max DD), the maximum drawdown durations (Max DD DUR), the average percentage changes in holdings (TO), and the *t*-statistics (*t*-stat) of equally-weighted decile portfolios sorted on out-of-sample return forecasts.

## Figures

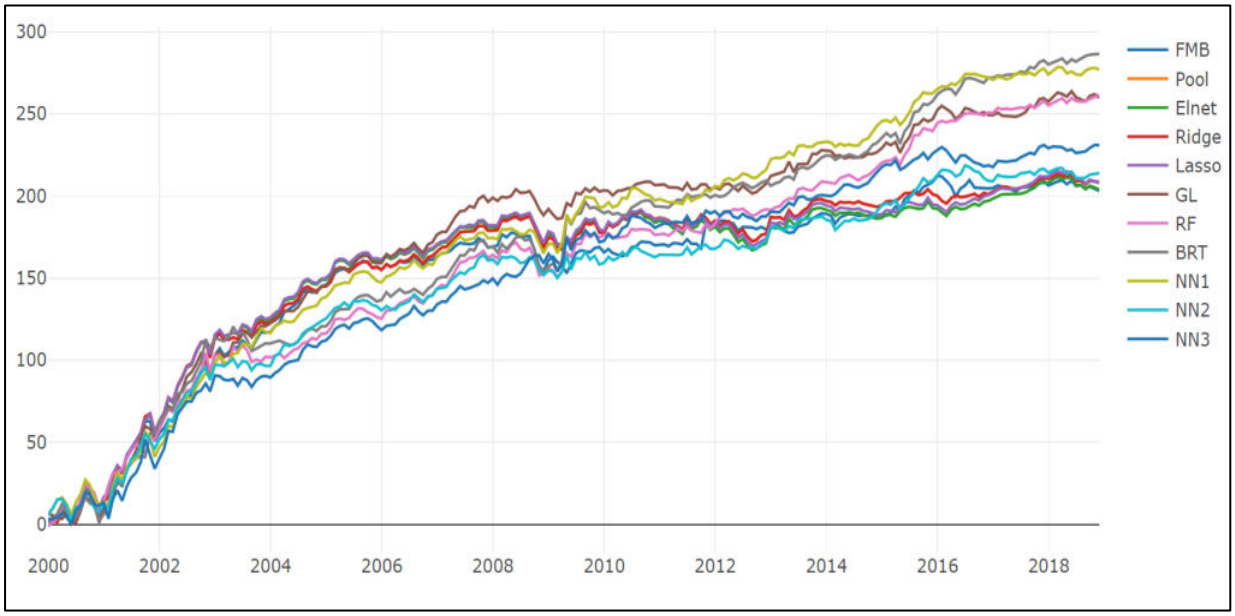



*Figure H.1.* Cumulative returns of equally-weighted decile portfolios sorted on out-of-sample return forecasts.